

Análisis Completo: Pensamiento Computacional + Historia

Proyecto: Evaluación PC e Historia — Colegio Juan Echegaray

Metadato	Valor
Centro	Colegio Juan Echegaray
Curso	5.º Primaria — 4 clases (A, B, C, D)
Test PC	28 ítems de respuesta múltiple (A/B/C/D)
Test Historia inicial	8 ítems de respuesta binaria (P1–P8)
Test Historia final	11 ítems de respuesta binaria (P1–P11)
Datos	<code>./data/{test_inicial_pc_echegaray.csv, especificacion_items.csv, autoperception_test_inicial_pc_echegaray.csv, Test_iniciales_v_finales.xlsx}</code>
Outputs	<code>./outputs/{tables/, figures/, logs/}</code>
Semilla	<code>numpy.random.seed(42)</code>

Objetivos del análisis

1. **Describir el rendimiento en PC inicial** de los alumnos de Echegaray: distribución global, por clase, género y NEE; análisis psicométrico de ítems por concepto computacional y tarea requerida.
2. **Examinar la autopercepción** de los alumnos sobre su rendimiento y su relación con el rendimiento real (a nivel agregado de grupo, dada la ausencia de ID individual).
3. **Clasificar las clases por nivel PC** (Bajo / Medio / Alto mediante terciles sobre la media de clase) para usarlo como variable moderadora en las secciones 7 y 8.
4. **Analizar el aprendizaje de Historia** (diseño pre–post, **muestras independientes** por clase): ganancia de Hake **grupal por clase** (sin emparejamiento individual) y efecto del nivel PC sobre la ganancia.
5. **Explorar las votaciones** de satisfacción con las actividades, cruzadas con el nivel PC de la clase, y su posible relación con la ganancia de aprendizaje.
6. **Exportar y verificar** todos los outputs mediante quality gates automáticos que garantizan la reproducibilidad del análisis.

Estructura del cuaderno

Sección	Contenido
0 · Setup	Librerías, semilla, rutas, logger, funciones utilitarias y helper de tablas
1 · Carga y Auditoría	Carga robusta de todos los datasets; tabla de auditoría de calidad exportada
2 · Test PC: Formato + Aciertos	Detección automática de formato de respuestas; score por alumno, tarea y concepto
3 · Descriptivo PC	Distribución global y por grupo; heatmap ítems × clase; barplots por concepto
4 · Contrastes PC	Diferencias significativas por Género y NEE; decisión metodológica automática
5 · Autopercepción	Contraste autopercepción vs. rendimiento real a nivel agregado de grupo
6 · Nivel PC por Clase	Clasificación en terciles (Bajo/Medio/Alto) como variable moderadora
7 · Historia	Análisis pre-post (muestras independientes); Hake grupal por clase; ANOVA; modelo OLS
8 · Votaciones	Satisfacción por actividad × nivel PC; ranking; correlación voto-ganancia
9 · Reflexión Metodológica	Los cinco pilares del PC aplicados al pipeline de análisis
10 · Quality Gates	Verificación automática de todos los outputs e invariantes de datos

Nota metodológica clave

Historia — diseño de muestras independientes: El índice `Alumno` en los datos representa el orden de respuesta, no identifica al mismo estudiante en ambos tests. Por tanto, **no es posible emparejar observaciones individuales**. La ganancia de Hake se calcula a nivel de **clase** (sobre medias de grupo):

$$g = \frac{\text{pct_fin} - \text{pct_ini}}{1 - \text{pct_ini}}$$

Nivel PC como moderador: El nivel (Bajo/Medio/Alto) se asigna a nivel de **clase**, no de alumno individual. Solo se aplica a las clases de Echegaray para no imputar niveles a centros sin datos de PC.

Autopercepción — limitación explícita: Sin columna ID en el dataset de autopercepción, la comparación con el rendimiento real solo es posible a nivel de media de grupo. No se realizan correlaciones individuales.

Nota de reproducibilidad

Este cuaderno es **autocontenido**. Instala las dependencias con `pip install -r requirements.txt`. Ejecutando todas las celdas en orden (con semilla `np.random.seed(42)`) se regeneran íntegramente todos los outputs en `./outputs/`. La **Sección 10 (Quality Gates)** verifica automáticamente la integridad de cada archivo de salida.

0. Setup y Configuración

Qué hace esta sección: Importa todas las librerías, fija la semilla aleatoria (`np.random.seed(42)`), crea los directorios de salida, configura el logger de calidad de datos y define las funciones utilitarias compartidas: lectura robusta de CSV/Excel, detección de normalidad, Hedges g, rank-biserial, IC bootstrap y prueba de permutación. Al final se añade el helper `style_table()` para tablas con estilo uniforme en todo el cuaderno.

Por qué: Centralizar configuración y funciones en el primer bloque garantiza reproducibilidad exacta (misma semilla → mismos resultados en bootstrap y permutaciones) y permite reutilizar las utilidades en cualquier sección sin duplicar código.

```
In [1]: # -----
# 0.1 Imports y semilla
# -----

import warnings
warnings.filterwarnings("ignore")

import os
import sys
# Ensure stdout/stderr use UTF-8 on Windows to avoid cp1252 issues
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
if hasattr(sys.stderr, "reconfigure"):
    sys.stderr.reconfigure(encoding="utf-8", errors="replace")

import logging
import pathlib
import re
import itertools

import numpy as np
import pandas as pd
import scipy.stats as stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.weightstats import DescrStatsW

import matplotlib
# Use Agg backend when running as plain script (non-interactive / CI)
# In a Jupyter kernel the inline magic overrides this automatically.
import sys as _sys
if not hasattr(_sys, "ps1") and "ipykernel" not in _sys.modules:
    matplotlib.use("Agg")
```

```

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

np.random.seed(42)

# -----
# 0.2 Opciones de pandas y estilo básico
# -----
pd.set_option("display.max_columns", 40)
pd.set_option("display.max_rows", 80)
pd.set_option("display.float_format", "{:.3f}".format)
pd.set_option("display.width", 120)

sns.set_theme(style="whitegrid", palette="muted", font_scale=1.1)
matplotlib.rcParams.update({"figure.dpi": 110, "figure.figsize": (9, 5)})

# -----
# 0.3 Rutas
# -----
ROOT = pathlib.Path(".")
DATA = ROOT / "data"
OUT = ROOT / "outputs"
FIGS = OUT / "figures"
TABS = OUT / "tables"
LOGS = OUT / "logs"

for d in [FIGS, TABS, LOGS]:
    d.mkdir(parents=True, exist_ok=True)

# -----
# 0.4 Logger
# -----
log_path = LOGS / "data_quality.log"
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler(log_path, mode="w", encoding="utf-8"),
        logging.StreamHandler(sys.stdout),
    ],
)
logger = logging.getLogger("pc_hist")
logger.info("=== Inicio del análisis PC + Historia ===")

# -----
# 0.5 Funciones utilitarias
# -----
def safe_read_csv(path, **kwargs):
    """Lee CSV con detección automática de separador y encoding."""
    path = pathlib.Path(path)
    for enc in ["utf-8", "latin-1", "cp1252"]:
        for sep in [",", ";", "\t"]:
            try:
                df = pd.read_csv(path, encoding=enc, sep=sep, **kwargs)
                if df.shape[1] > 1:
                    logger.info(f"CSV cargado: {path.name} | enc={enc} sep='{sep}'")
                    return df
            except Exception:
                pass

```

```

# Fallback: engine python, sep=None
df = pd.read_csv(path, sep=None, engine="python", **kwargs)
logger.info(f"CSV cargado (fallback python engine): {path.name} | {df.shape}")
return df

def safe_read_excel(path, sheet_name=0, **kwargs):
    """Lee hoja Excel con manejo de errores."""
    path = pathlib.Path(path)
    try:
        df = pd.read_excel(path, sheet_name=sheet_name, **kwargs)
        logger.info(f"Excel cargado: {path.name} hoja='{sheet_name}' | {df.shape}")
        return df
    except Exception as e:
        logger.error(f"Error leyendo {path.name} hoja={sheet_name}: {e}")
        return pd.DataFrame()

def ensure_numeric(series, col_name=""):
    """Convierte a numérico; valores inválidos → NaN."""
    result = pd.to_numeric(series, errors="coerce")
    n_bad = series.notna().sum() - result.notna().sum()
    if n_bad > 0:
        logger.warning(f" '{col_name}': {n_bad} valores no numéricos → NaN")
    return result

def coerce_01(series, col_name=""):
    """Fuerza a {0,1,NaN}; cualquier valor fuera → NaN."""
    s = ensure_numeric(series, col_name)
    mask_invalid = s.notna() & ~s.isin([0, 1])
    if mask_invalid.sum() > 0:
        logger.warning(f" '{col_name}': {mask_invalid.sum()} valores ∉ {{0,1}}")
        s.loc[mask_invalid] = np.nan
    return s

def log_anomaly(msg):
    """Registra una anomalía en el log."""
    logger.warning(f"[ANOMALÍA] {msg}")

def bootstrap_ci(data, stat_fn=np.mean, n_boot=2000, ci=95, seed=42):
    """IC bootstrap percentil para una estadística."""
    rng = np.random.default_rng(seed)
    data = np.asarray(data, dtype=float)
    data = data[~np.isnan(data)]
    if len(data) < 2:
        return (np.nan, np.nan)
    boots = [stat_fn(rng.choice(data, size=len(data), replace=True)) for _ in range(n_boot)]
    lo = np.percentile(boots, (100 - ci) / 2)
    hi = np.percentile(boots, 100 - (100 - ci) / 2)
    return (lo, hi)

def hedges_g(a, b):
    """Hedges g con IC 95 % aproximado (Borenstein 2009)."""
    a, b = np.asarray(a, float), np.asarray(b, float)
    a, b = a[~np.isnan(a)], b[~np.isnan(b)]
    n1, n2 = len(a), len(b)

```

```

if n1 < 2 or n2 < 2:
    return np.nan, (np.nan, np.nan)
d = (np.mean(a) - np.mean(b)) / np.sqrt(
    ((n1 - 1) * np.var(a, ddof=1) + (n2 - 1) * np.var(b, ddof=1)) / (n1 + n2
)
j = 1 - 3 / (4 * (n1 + n2 - 2) - 1)
g = d * j
se_g = np.sqrt((n1 + n2) / (n1 * n2) + g**2 / (2 * (n1 + n2 - 2)))
return g, (g - 1.96 * se_g, g + 1.96 * se_g)

def rank_biserial(a, b):
    """Correlación biserial de rangos (Cliff's delta) para Mann-Whitney."""
    a, b = np.asarray(a, float), np.asarray(b, float)
    a, b = a[~np.isnan(a)], b[~np.isnan(b)]
    n1, n2 = len(a), len(b)
    if n1 < 1 or n2 < 1:
        return np.nan
    stat, _ = stats.mannwhitneyu(a, b, alternative="two-sided")
    r = 1 - (2 * stat) / (n1 * n2)
    return r

def permutation_test(a, b, n_perm=5000, seed=42):
    """Prueba de permutación sobre diferencia de medias."""
    rng = np.random.default_rng(seed)
    a, b = np.asarray(a, float), np.asarray(b, float)
    a, b = a[~np.isnan(a)], b[~np.isnan(b)]
    if len(a) < 2 or len(b) < 2:
        return np.nan
    obs = np.abs(np.mean(a) - np.mean(b))
    combined = np.concatenate([a, b])
    count = 0
    for _ in range(n_perm):
        rng.shuffle(combined)
        count += np.abs(np.mean(combined[:len(a)]) - np.mean(combined[len(a):]))
    return count / n_perm

def shapiro_ok(x, alpha=0.05):
    """True si no se rechaza normalidad (N < 50) o si N >= 50 (no aplicable)."""
    x = np.asarray(x, float)
    x = x[~np.isnan(x)]
    if len(x) < 3:
        return False
    if len(x) >= 50:
        return False # Shapiro no fiable con N>=50; preferir MWU
    _, p = stats.shapiro(x)
    return p > alpha

def choose_test(a, b, alpha=0.05):
    """Selecciona t de Welch o Mann-Whitney según supuestos."""
    norm_a = shapiro_ok(a, alpha)
    norm_b = shapiro_ok(b, alpha)
    use_t = norm_a and norm_b
    return "welch_t" if use_t else "mann_whitney"

```

— *Helper de estilo de tablas* —

```
_HEADER_STYLE = [
    {"selector": "caption",
     "props": [("font-size", "13px"), ("font-weight", "bold"),
               ("color", "#1a1a2e"), ("margin-bottom", "6px"),
               ("text-align", "left")]},
    {"selector": "thead th",
     "props": [("background-color", "#2c3e50"), ("color", "white"),
               ("font-size", "12px"), ("padding", "7px 12px"),
               ("text-align", "center"), ("border", "none")]},
    {"selector": "tbody td",
     "props": [("padding", "5px 12px"), ("font-size", "12px"),
               ("border-bottom", "1px solid #e8e8e8")]},
    {"selector": "tbody tr:nth-child(even)",
     "props": [("background-color", "#f5f7fa")]},
    {"selector": "tbody tr:hover",
     "props": [("background-color", "#eaf4fb")]},
    {"selector": "table",
     "props": [("border-collapse", "collapse"), ("width", "auto"),
               ("margin-top", "4px")]},
]

def style_table(df, caption="", fmt_float="{:.3f}", fmt_cols=None,
               bar_col=None, bar_range=(0, 1), pval_cols=None,
               highlight_max_col=None, highlight_min_col=None,
               hide_index=True):
    """
    Devuelve un pandas Styler con estilos consistentes para display en el cuader
    Parámetros
    -----
    caption          : str    - Título que aparece sobre la tabla.
    fmt_float        : str    - Formato para columnas float (default 3 decimales)
    fmt_cols         : dict   - Sobreescribe formato para columnas específicas.
    bar_col          : str    - Nombre de columna donde añadir barra de progreso.
    bar_range        : tuple  - (vmin, vmax) para la barra de progreso.
    pval_cols        : list   - Columnas de p-valor; se resaltan en verde si < 0.
    highlight_max_col: str    - Columna cuyo máximo se resalta en amarillo.
    highlight_min_col: str    - Columna cuyo mínimo se resalta en rojo claro.
    hide_index       : bool   - Ocultar el índice del DataFrame (default True).
    """
    float_cols = df.select_dtypes(include="float").columns.tolist()
    fmt_dict = {col: fmt_float for col in float_cols}
    if fmt_cols:
        fmt_dict.update(fmt_cols)
    styler = (
        df.style
        .set_caption(caption)
        .format(fmt_dict, na_rep="-")
        .set_table_styles(_HEADER_STYLE)
    )
    if hide_index:
        styler = styler.hide(axis="index")
    if bar_col and bar_col in df.columns:
        styler = styler.bar(
            subset=[bar_col], vmin=bar_range[0], vmax=bar_range[1],
            color=["#d65f5f", "#5fba7d"],
        )
    if pval_cols:
        for pcol in pval_cols:
            if pcol in df.columns:
```

```

        styler = styler.map(
            lambda v: "background-color: #d4edda; color: #155724;"
                    if isinstance(v, float) and v < 0.05 else "",
            subset=[pcol],
        )
    if highlight_max_col and highlight_max_col in df.columns:
        styler = styler.highlight_max(subset=[highlight_max_col], color="#fff3cd")
    if highlight_min_col and highlight_min_col in df.columns:
        styler = styler.highlight_min(subset=[highlight_min_col], color="#fde8e8")
    return styler

print("Setup completo OK")
print(f"  Figures → {FIGS.resolve()}")
print(f"  Tables  → {TABS.resolve()}")
print(f"  Logs    → {LOGS.resolve()}")

```

2026-05-22 13:25:32,752 [INFO] === Inicio del análisis PC + Historia ===

Setup completo OK

Figures → H:\Proyectos\Analysis_PC_History\outputs\figures

Tables → H:\Proyectos\Analysis_PC_History\outputs\tables

Logs → H:\Proyectos\Analysis_PC_History\outputs\logs

1. Carga + Auditoría de Todos los Datos

Qué hace esta sección: Carga todos los archivos del proyecto —CSV de PC, especificación de ítems, autopercepción, Excel de Historia con todas sus hojas y `category_questions.xlsx` — usando funciones robustas con detección automática de separador y encoding. Muestra las primeras filas de cada dataset y llama a la función de auditoría.

Por qué: Antes de cualquier análisis es imprescindible conocer la calidad de los datos: dimensiones, tipo de columnas, presencia de NaN y valores atípicos. La carga robusta evita errores silenciosos por diferencias de separador (`,` vs `;`) o encoding (`utf-8` vs `latin-1`).

```

In [2]: # -----
# 1.1 Carga de archivos
# -----
logger.info("--- SECCIÓN 1: Carga de datos ---")

# PC test inicial
df_pc_raw = safe_read_csv(DATA / "test_inicial_pc_echegaray.csv")

# Especificación de ítems
df_spec = safe_read_csv(DATA / "especificacion_items.csv")

# Autopercepción
df_auto_raw = safe_read_csv(DATA / "autoperception_test_inicial_pc_echegaray.csv")

# Excel Historia
xl_path = DATA / "Test_iniciales_v_finales.xlsx"
xl = pd.ExcelFile(xl_path)
print(f"\nHojas en {xl_path.name}:")
for s in xl.sheet_names:
    print(f"  • {s}")

```

```
df_hist_ini = safe_read_excel(xl_path, sheet_name="Test inicial")
df_hist_fin = safe_read_excel(xl_path, sheet_name="Test final")

# Hojas de votaciones
voting_sheets = [s for s in xl.sheet_names if "Votaci" in s]
dfs_votes = {}
for sh in voting_sheets:
    dfs_votes[sh] = safe_read_excel(xl_path, sheet_name=sh)
print(f"\nHojas de votaciones: {voting_sheets}")

# Category questions
xl_cat = pd.ExcelFile(DATA / "category_questions.xlsx")
print(f"\nHojas category_questions.xlsx: {xl_cat.sheet_names}")
df_cat = safe_read_excel(DATA / "category_questions.xlsx", sheet_name=xl_cat.she
```

```
2026-05-22 13:25:32,774 [INFO] --- SECCIÓN 1: Carga de datos ---
2026-05-22 13:25:32,780 [INFO] CSV cargado: test_inicial_pc_echegaray.csv | enc=utf-8 sep=', ' | (97, 32)
2026-05-22 13:25:32,783 [INFO] CSV cargado: especificacion_items.csv | enc=utf-8 sep=', ' | (28, 7)
2026-05-22 13:25:32,786 [INFO] CSV cargado: autoperception_test_inicial_pc_echegaray.csv | enc=utf-8 sep=', ' | (97, 4)
```

Hojas en Test_iniciales_v_finales.xlsx:

- Test inicial
- Test inicial totales
- Test final totales
- Test final
- Votaciones actividad primer dia
- Votaciones actividad prehistori
- Votaciones actividad edad antig
- Votaciones actividad Edad Media
- Votaciones actividad Edad Moder
- Votaciones Edad Contemporanea
- Votaciones actividad ult dia

```
2026-05-22 13:25:33,308 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Test inicial' | (93, 13)
2026-05-22 13:25:33,356 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Test final' | (82, 16)
2026-05-22 13:25:33,401 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad primer dia' | (81, 5)
2026-05-22 13:25:33,465 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad prehistori' | (81, 5)
2026-05-22 13:25:33,555 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad edad antig' | (81, 5)
2026-05-22 13:25:33,596 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad Edad Media' | (81, 5)
2026-05-22 13:25:33,660 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad Edad Moder' | (81, 5)
2026-05-22 13:25:33,715 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones Edad Contemporanea' | (81, 5)
2026-05-22 13:25:33,819 [INFO] Excel cargado: Test_iniciales_v_finales.xlsx hoja='Votaciones actividad ult dia' | (81, 5)
```

Hojas de votaciones: ['Votaciones actividad primer dia', 'Votaciones actividad prehistori', 'Votaciones actividad edad antig', 'Votaciones actividad Edad Media', 'Votaciones actividad Edad Moder', 'Votaciones Edad Contemporanea', 'Votaciones actividad ult dia']

Hojas category_questions.xlsx: ['Hoja1']

```
2026-05-22 13:25:33,833 [INFO] Excel cargado: category_questions.xlsx hoja='Hoja 1' | (21, 3)
```

1.2 Auditoría de Datasets

Qué hace esta subsección: Para cada dataset cargado calcula estadísticos de calidad: número de filas/columnas, porcentaje de NaN por columna, valores únicos por variable y rango mínimo-máximo para las numéricas. Registra el resultado en `auditoria_datasets.csv` y en el log de calidad.

Por qué: La auditoría es el primer control de calidad del pipeline. Detectar columnas con exceso de NaN, duplicados inesperados o valores fuera de rango en esta fase evita que

errores de datos contaminen todos los análisis posteriores y hace trazable cualquier decisión de limpieza.

```
In [3]: def audit_df(df, name, key_cols=None):
        """Genera tabla de auditoría para un DataFrame."""
        n_rows, n_cols = df.shape
        pct_nan = (df.isna().mean() * 100).round(1)
        dups = df.duplicated(subset=key_cols).sum() if key_cols else np.nan
        audit_rows = []
        for col in df.columns:
            s = df[col]
            row = {
                "dataset": name,
                "columna": col,
                "dtype": str(s.dtype),
                "n_no_nulos": int(s.notna().sum()),
                "pct_nan": round(s.isna().mean() * 100, 1),
                "n_unicos": int(s.nunique()),
                "min": s.min() if pd.api.types.is_numeric_dtype(s) else None,
                "max": s.max() if pd.api.types.is_numeric_dtype(s) else None,
            }
            audit_rows.append(row)
        logger.info(
            f"[AUDITORÍA] {name}: {n_rows} filas × {n_cols} cols | "
            f"dup_key={dups} | NaN_max={pct_nan.max():.1f}%"
        )
        return pd.DataFrame(audit_rows)

audits = []
audits.append(audit_df(df_pc_raw, "PC_test_inicial",
                        key_cols=["ID"] if "ID" in df_pc_raw.columns else None))
audits.append(audit_df(df_spec, "Especificacion_items"))
audits.append(audit_df(df_auto_raw, "Autoperception"))
audits.append(audit_df(df_hist_ini, "Historia_inicial",
                        key_cols=["Alumno", "Clase", "Colegio"]))
audits.append(audit_df(df_hist_fin, "Historia_final",
                        key_cols=["Alumno", "Clase", "Colegio"]))
audits.append(audit_df(df_cat, "category_questions"))

df_audit = pd.concat(audits, ignore_index=True)
df_audit.to_csv(TABS / "auditoria_datasets.csv", index=False, encoding="utf-8-si")
print(f"Auditoría guardada → {TABS / 'auditoria_datasets.csv'}")
print("\nResumen por dataset:")
display(style_table(
    df_audit.groupby("dataset")[["pct_nan"]].agg(["max", "mean"]).round(1).reset
    caption="% NaN por dataset (máximo y media)",
))
```

```

2026-05-22 13:25:33,891 [INFO] [AUDITORÍA] PC_test_inicial: 97 filas x 32 cols |
dup_key=0 | NaN_max=6.2%
2026-05-22 13:25:33,896 [INFO] [AUDITORÍA] Especificacion_items: 28 filas x 7 col
s | dup_key=nan | NaN_max=85.7%
2026-05-22 13:25:33,901 [INFO] [AUDITORÍA] Autoperception: 97 filas x 4 cols | du
p_key=nan | NaN_max=0.0%
2026-05-22 13:25:33,909 [INFO] [AUDITORÍA] Historia_inicial: 93 filas x 13 cols |
dup_key=0 | NaN_max=1.1%
2026-05-22 13:25:33,922 [INFO] [AUDITORÍA] Historia_final: 82 filas x 16 cols | d
up_key=0 | NaN_max=2.4%
2026-05-22 13:25:33,925 [INFO] [AUDITORÍA] category_questions: 21 filas x 3 cols
| dup_key=nan | NaN_max=0.0%
Auditoría guardada → outputs\tables\auditoria_datasets.csv

```

Resumen por dataset:

% NaN por dataset (máximo y media)

dataset	pct_nan	
	max	mean
Autoperception	0.000	0.000
Especificacion_items	85.700	23.500
Historia_final	2.400	0.900
Historia_inicial	1.100	0.100
PC_test_inicial	6.200	2.000
category_questions	0.000	0.000

2. Test Inicial PC: Detección de Formato + Aciertos

Qué hace esta sección: Estandariza los nombres de columnas, detecta automáticamente el formato de respuestas del test PC (binario `{0,1}` o letras `A/B/C/D`), convierte las respuestas a acierto/fallo comparando con la clave de corrección (`especificacion_items.csv`) y calcula el score total por alumno, además de scores desglosados por **tarea requerida** (Secuenciación, Completamiento, Depuración) y **concepto computacional** (Direcciones, Bucles, Condicionales, Funciones).

Por qué: El CSV de PC puede venir en dos formatos distintos según el proceso de exportación. Detectar el formato automáticamente hace el cuaderno robusto ante diferentes versiones del archivo. El desglose por tarea y concepto permite identificar en qué dimensión del pensamiento computacional muestran más o menos dominio los alumnos, orientando posibles intervenciones didácticas.

```

In [4]: # -----
# 2.1 Estandarizar nombres de columnas
# -----
logger.info("--- SECCIÓN 2: Detección de formato PC + cálculo de aciertos ---")

df_pc = df_pc_raw.copy()
# Limpiar nombres (quitar \n y espacios sobrantes)
df_pc.columns = [c.strip().replace("\n", "") for c in df_pc.columns]

```

```

# Columna Clase
col_clase_pc = [c for c in df_pc.columns if "clase" in c.lower()]
assert len(col_clase_pc) >= 1, "No se encontró columna 'Clase' en PC"
col_clase_pc = col_clase_pc[0]
df_pc.rename(columns={col_clase_pc: "Clase"}, inplace=True)

# Columna NEE
col_nee = [c for c in df_pc.columns if "necesidad" in c.lower() or "nee" in c.lower()]
assert len(col_nee) >= 1, "No se encontró columna NEE en PC"
df_pc.rename(columns={col_nee[0]: "NEE"}, inplace=True)

# Estandarizar Género
genero_map = {
    "chico": "Chico", "masculino": "Chico", "hombre": "Chico", "m": "Chico",
    "chica": "Chica", "femenino": "Chica", "mujer": "Chica", "f": "Chica",
}
df_pc["Género"] = df_pc["Género"].str.strip().str.lower().map(genero_map).fillna("")

# Estandarizar NEE
nee_map = {"si": "Sí", "sí": "Sí", "yes": "Sí", "1": "Sí",
           "no": "No", "0": "No"}
df_pc["NEE"] = df_pc["NEE"].str.strip().str.lower().map(nee_map).fillna(df_pc["NEE"].isna().fillna(""))

# Estandarizar Clase
df_pc["Clase"] = df_pc["Clase"].str.strip().str.upper()

print("Distribución de grupos:")
print("  Clase:", df_pc["Clase"].value_counts().to_dict())
print("  Género:", df_pc["Género"].value_counts().to_dict())
print("  NEE:", df_pc["NEE"].value_counts().to_dict())

# -----
# 2.2 Detectar columnas de Preguntas (1..28)
# -----
pregunta_cols = []
for i in range(1, 29):
    matches = [c for c in df_pc.columns if re.search(rf"pregunta\s*{i}$", c, re.IGNORECASE)]
    if matches:
        pregunta_cols.append(matches[0])
    else:
        log_anomaly(f"No se encontró 'Pregunta {i}' en el CSV de PC")

logger.info(f"Columnas de preguntas detectadas: {len(pregunta_cols)}/28")
print(f"Columnas de preguntas encontradas: {len(pregunta_cols)}")
print(f"  Ejemplo de valores: {df_pc[pregunta_cols[:3]].iloc[0].to_dict()}")

# -----
# 2.3 Detección de formato: {0,1} o Letras
# -----
sample_vals = df_pc[pregunta_cols].stack().dropna().unique()
is_binary = all(v in [0, 1, "0", "1"] for v in sample_vals)

if is_binary:
    fmt_detected = "BINARIO (0/1)"
    logger.info(f"Formato PC detectado: {fmt_detected}")
    for col in pregunta_cols:
        df_pc[col] = coerce_01(df_pc[col], col)
    n_conversiones = 0
else:
    fmt_detected = "LETRAS (A/B/C/D)"

```

```

logger.info(f"Formato PC detectado: {fmt_detected} → convirtiendo a 0/1")
spec_clean = df_spec.copy()
spec_clean["Item_num"] = spec_clean["Item"].str.extract(r"(\d+)").astype(int)
spec_clean = spec_clean.sort_values("Item_num")
respuestas_correctas = dict(zip(spec_clean["Item_num"], spec_clean["Respuest

n_conversiones = 0
for i, col in enumerate(pregunta_cols, start=1):
    correcto = respuestas_correctas.get(i)
    if correcto is None:
        log_anomaly(f"No hay respuesta correcta para Pregunta {i}")
        df_pc[col] = np.nan
        continue
    original = df_pc[col].str.strip().str.upper()
    df_pc[col] = (original == correcto.upper()).astype(float)
    n_conversiones += (df_pc[col].notna()).sum()

logger.info(f" Conversiones realizadas: {n_conversiones} celdas (letras→0/1

print(f"\nFormato detectado: {fmt_detected}")
print(f"Conversiones realizadas: {n_conversiones}")

# -----
# 2.4 Construir variables de score
# -----
pc_mat = df_pc[pregunta_cols].apply(pd.to_numeric, errors="coerce")

df_pc["score_pc"] = pc_mat.sum(axis=1)
df_pc["pct_pc"] = df_pc["score_pc"] / len(pregunta_cols)

# Preparar spec limpio (usado en ambos Loops)
spec_clean = df_spec.copy()
spec_clean["Item_num"] = spec_clean["Item"].str.extract(r"(\d+)").astype(int)
spec_clean = spec_clean.sort_values("Item_num").reset_index(drop=True)

# - Por Tarea requerida (Secuenciación / Completamiento / Depuración) -
tarea_pct_cols = [] # lista de columnas creadas para este análisis
tareas = spec_clean["Tarea_requerida"].str.strip().unique()
for tarea in tareas:
    items_idx = spec_clean.loc[
        spec_clean["Tarea_requerida"].str.strip() == tarea, "Item_num"
    ].tolist()
    cols_t = [pregunta_cols[i - 1] for i in items_idx if i - 1 < len(pregunta_co
    if cols_t:
        col_name = "pct_" + re.sub(r"\W+", "_", tarea.strip())
        df_pc[col_name] = df_pc[cols_t].apply(pd.to_numeric, errors="coerce").me
        tarea_pct_cols.append(col_name)

# - Por Concepto Computacional (Direcciones / Bucles / Condicionales / Funciones
concepto_pct_cols = [] # lista de columnas creadas para este análisis
for concepto in ["Direcciones", "Bucles", "Condicionales", "Funciones"]:
    mask_c = spec_clean[concepto].notna()
    items_idx = spec_clean.loc[mask_c, "Item_num"].tolist()
    cols_c = [pregunta_cols[i - 1] for i in items_idx if i - 1 < len(pregunta_co
    if cols_c:
        col_name = f"pct_{concepto}"
        df_pc[col_name] = df_pc[cols_c].apply(pd.to_numeric, errors="coerce").me
        concepto_pct_cols.append(col_name)

print("\nVariables construidas:")

```

```

print(f" Tarea_requerida : {tarea_pct_cols}")
print(f" Conceptos      : {concepto_pct_cols}")
print(f"\nResumen score_pc:")
display(style_table(
    df_pc["score_pc"].describe().round(2).to_frame().reset_index().rename(column
caption="Estadísticos descriptivos: score PC inicial",
))

# -----
# 2.5 Exportar tabla limpia PC
# -----

df_pc.to_csv(TABS / "pc_limpio_por_alumno.csv", index=False, encoding="utf-8-sig
logger.info(f"Exportado: pc_limpio_por_alumno.csv | {df_pc.shape}")
print(f"\nExportado → {TABS / 'pc_limpio_por_alumno.csv'}")

```

2026-05-22 13:25:34,076 [INFO] --- SECCIÓN 2: Detección de formato PC + cálculo d e aciertos ---

Distribución de grupos:

Clase: {'C': 26, 'A': 24, 'D': 24, 'B': 23}

Género: {'Chico': 52, 'Chica': 45}

NEE: {'No': 89, 'Sí': 8}

2026-05-22 13:25:34,089 [INFO] Columnas de preguntas detectadas: 28/28

Columnas de preguntas encontradas: 28

Ejemplo de valores: {'Pregunta 1': 'B', 'Pregunta 2': 'C', 'Pregunta 3': 'D'}

2026-05-22 13:25:34,102 [INFO] Formato PC detectado: LETRAS (A/B/C/D) → convirtie ndo a 0/1

2026-05-22 13:25:34,128 [INFO] Conversiones realizadas: 2716 celdas (letras→0/ 1)

Formato detectado: LETRAS (A/B/C/D)

Conversiones realizadas: 2716

Variables construidas:

Tarea_requerida : ['pct_Secuenciación', 'pct_Completamiento', 'pct_Depuración']

Conceptos : ['pct_Direcciones', 'pct_Bucles', 'pct_Condicionales', 'pct_F unciones']

Resumen score_pc:

Estadísticos descriptivos:

score PC inicial

estadístico	score_pc
count	97.000
mean	17.040
std	4.960
min	4.000
25%	14.000
50%	18.000
75%	21.000
max	26.000

2026-05-22 13:25:34,192 [INFO] Exportado: pc_limpio_por_alumno.csv | (97, 41)

Exportado → outputs\tables\pc_limpio_por_alumno.csv

3. Descriptivo del PC Inicial

Qué hace esta sección: Genera el análisis descriptivo completo del rendimiento en el test PC: histogramas con KDE del score global, boxplots/violines por clase, género y NEE con IC bootstrap al 95%, tablas de estadísticos descriptivos globales y por grupo, heatmap de % de acierto por pregunta × clase, y barplots por tarea requerida y concepto computacional.

Por qué: La estadística descriptiva es el paso previo imprescindible antes de cualquier inferencia: permite ver la distribución, detectar outliers, comparar grupos visualmente e identificar qué conceptos o tareas son más difíciles para el alumnado antes de aplicar pruebas formales de significación.

```
In [ ]: # -----
# 3.1 Histograma + KDE de score_pc y pct_pc (global)
# -----
logger.info("--- SECCIÓN 3: Descriptivo PC ---")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, col, xlabel in zip(axes,
                           ["score_pc", "pct_pc"],
                           ["Puntuación (0-28)", "% aciertos (0-1)"]):
    sns.histplot(df_pc[col].dropna(), kde=True, ax=ax, bins=14, color="steelblue")
    ax.axvline(df_pc[col].mean(), color="crimson", ls="--", lw=1.5, label=f"Mediana")
    ax.axvline(df_pc[col].median(), color="orange", ls=":", lw=1.5, label=f"Mediana")
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Frecuencia")
    ax.legend(fontsize=9)
    ax.set_title(f"Distribución {col}")

fig.suptitle("Distribución global del Test Inicial PC", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "pc_histograma_global.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_histograma_global.png")

# -----
# 3.2 Boxplot/Violin por Clase, Género y NEE
# -----
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
grouping_vars = [
    ("Clase", "Clase (A/B/C/D)"),
    ("Género", "Género"),
    ("NEE", "Necesidad Educativa Especial"),
]
palette = {"A": "#4E79A7", "B": "#F28E2B", "C": "#E15759", "D": "#76B7B2",
           "Chico": "#59A14F", "Chica": "#B07AA1",
           "Sí": "#FF9DA7", "No": "#9C755F"}

for ax, (gvar, glabel) in zip(axes, grouping_vars):
    order = sorted(df_pc[gvar].dropna().unique())
    data_sub = df_pc.dropna(subset=[gvar, "score_pc"])
    color_list = [palette.get(g, "steelblue") for g in order]
    vp = ax.violinplot(
        [data_sub.loc[data_sub[gvar] == g, "score_pc"].values for g in order],
```

```

        positions=range(len(order)),
        showmedians=True,
    )
    for body, color in zip(vp["bodies"], color_list):
        body.set_facecolor(color)
        body.set_alpha(0.7)
    ax.set_xticks(range(len(order)))
    ax.set_xticklabels(order)
    ax.set_xlabel(glabel)
    ax.set_ylabel("Score PC (0-28)")
    ax.set_title(f"Score PC por {gvar}")

fig.suptitle("Score PC inicial por grupos", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "pc_violin_por_grupo.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_violin_por_grupo.png")

# -----
# 3.3 Tabla resumen global y por grupos
# -----
def resumen_grupo(df, grupo_col=None, val_col="score_pc", n_boot=2000):
    """Calcula estadísticos + IC95% bootstrap."""
    rows = []
    if grupo_col is None:
        subsets = [("Global", df)]
    else:
        subsets = [(g, df.loc[df[grupo_col] == g]) for g in sorted(df[grupo_col])]
    for name, sub in subsets:
        x = sub[val_col].dropna().values
        if len(x) == 0:
            continue
        ci_lo, ci_hi = bootstrap_ci(x, np.mean, n_boot=n_boot)
        q1, q3 = np.percentile(x, 25), np.percentile(x, 75)
        rows.append({
            "Grupo": "Global" if grupo_col is None else "Global": name,
            "N": len(x),
            "Media": np.mean(x),
            "Mediana": np.median(x),
            "SD": np.std(x, ddof=1),
            "IQR": q3 - q1,
            "Min": np.min(x),
            "Max": np.max(x),
            "IC95_lo": ci_lo,
            "IC95_hi": ci_hi,
        })
    return pd.DataFrame(rows)

df_res_global = resumen_grupo(df_pc, grupo_col=None)
df_res_clase = resumen_grupo(df_pc, grupo_col="Clase")
df_res_genero = resumen_grupo(df_pc, grupo_col="Género")
df_res_nee = resumen_grupo(df_pc, grupo_col="NEE")

df_desc_global = pd.concat([df_res_global], ignore_index=True)
df_desc_grupos = pd.concat([
    df_res_clase.assign(Variable="Clase"),
    df_res_genero.assign(Variable="Género"),
    df_res_nee.assign(Variable="NEE"),
], ignore_index=True)

```

```

df_desc_global.to_csv(TABS / "pc_descriptivo_global.csv", index=False, encoding=
df_desc_grupos.to_csv(TABS / "pc_descriptivo_por_grupo.csv", index=False, encodi

print("Descriptivo global:")
display(style_table(df_desc_global, caption="Estadísticos descriptivos globales
print("\nDescriptivo por grupos:")
display(style_table(df_desc_grupos, caption="Descriptivos PC por grupo (Clase /

# -----
# 3.4 Tabla por ítem: %acierto global + por Clase
# -----
item_rows = []
for i, col in enumerate(pregunta_cols, start=1):
    vals_num = pd.to_numeric(df_pc[col], errors="coerce")
    row = {"Pregunta": i, "Global_%acierto": vals_num.mean()}
    for clase in sorted(df_pc["Clase"].dropna().unique()):
        mask = df_pc["Clase"] == clase
        row[f"Clase_{clase}%acierto"] = vals_num[mask].mean()
    item_rows.append(row)
df_items = pd.DataFrame(item_rows)
df_items.to_csv(TABS / "pc_por_item_y_clase.csv", index=False, encoding="utf-8-s
print(f"\nTabla de ítems guardada → {TABS / 'pc_por_item_y_clase.csv'}")
display(style_table(df_items, caption="% Acierto por ítem y clase", highlight_mi

# -----
# 3.5 Heatmap %acierto (Preguntas × Clases)
# -----
heatmap_cols = [c for c in df_items.columns if c.startswith("Clase_")]
df_heat = df_items.set_index("Pregunta")[heatmap_cols]
df_heat.columns = [c.replace("Clase_", "").replace("_%acierto", "") for c in df_

fig, ax = plt.subplots(figsize=(8, 10))
sns.heatmap(
    df_heat, annot=True, fmt=".0%", cmap="RdYlGn",
    vmin=0, vmax=1, linewidths=0.4, ax=ax,
    cbar_kws={"label": "% acierto"},
)
ax.set_title("Heatmap % acierto por Pregunta × Clase", fontweight="bold")
ax.set_xlabel("Clase")
ax.set_ylabel("Pregunta")
fig.tight_layout()
fig.savefig(FIGS / "pc_heatmap_items_clase.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_heatmap_items_clase.png")

# -----
# 3.6a Tabla % acierto por Tarea_requerida
#       (Secuenciación / Completamiento / Depuración)
# -----
def _make_dim_table(df, cols, group_col_name):
    """Construye tabla descriptiva para una lista de columnas pct_*."""
    rows = []
    for col in cols:
        # Recuperar etiqueta original desde el spec
        label = col.replace("pct_", "").replace("_", " ").strip()
        vals = df[col].dropna()
        rows.append({
            group_col_name: label,
            "N": len(vals),

```

```

        "Media": round(vals.mean(), 3),
        "SD": round(vals.std(ddof=1), 3),
        "Min": round(vals.min(), 3),
        "Max": round(vals.max(), 3),
    })
    return pd.DataFrame(rows)

df_tarea = _make_dim_table(df_pc, tarea_pct_cols, "Tarea_requerida")
df_tarea.to_csv(TABS / "pc_por_tarea_requerida.csv", index=False, encoding="utf-8")
print(f"\nTabla por Tarea_requerida → {TABS / 'pc_por_tarea_requerida.csv'}")
display(style_table(df_tarea, caption="Score PC por tarea requerida"))

# -----
# 3.6b Tabla % acierto por Concepto Computacional
#       (Direcciones / Bucles / Condicionales / Funciones)
# -----
df_concepto = _make_dim_table(df_pc, concepto_pct_cols, "Concepto")
df_concepto.to_csv(TABS / "pc_por_concepto_computacional.csv", index=False, encoding="utf-8")
print(f"\nTabla por Concepto Computacional → {TABS / 'pc_por_concepto_computacional.csv'}")
display(style_table(df_concepto, caption="Score PC por concepto computacional"))

# archivo combinado para compatibilidad con quality gates
df_dim = pd.concat([
    df_tarea.rename(columns={"Tarea_requerida": "Dimensión"}),
    df_concepto.rename(columns={"Concepto": "Dimensión"}),
], ignore_index=True)
df_dim.to_csv(TABS / "pc_por_dimension.csv", index=False, encoding="utf-8-sig")

# -----
# 3.7a Barplot % acierto por Tarea_requerida
# -----
fig, ax = plt.subplots(figsize=(8, 5))
df_t_sorted = df_tarea.sort_values("Media", ascending=False)
sns.barplot(data=df_t_sorted, x="Tarea_requerida", y="Media", palette="Blues_d",
ax.set_ylim(0, 1)
ax.set_ylabel("% acierto medio")
ax.set_xlabel("")
ax.set_title("% Acierto por Tarea Requerida (Test PC Inicial)", fontweight="bold")
plt.xticks(rotation=20, ha="right")
fig.tight_layout()
fig.savefig(FIGS / "pc_acierto_por_tarea_requerida.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_acierto_por_tarea_requerida.png")

# -----
# 3.7b Barplot % acierto por Concepto Computacional
# -----
fig, ax = plt.subplots(figsize=(8, 5))
df_c_sorted = df_concepto.sort_values("Media", ascending=False)
sns.barplot(data=df_c_sorted, x="Concepto", y="Media", palette="Oranges_d", ax=ax)
ax.set_ylim(0, 1)
ax.set_ylabel("% acierto medio")
ax.set_xlabel("")
ax.set_title("% Acierto por Concepto Computacional (Test PC Inicial)\n"
            "Direcciones · Bucles · Condicionales · Funciones",
            fontweight="bold")
plt.xticks(rotation=15, ha="right")
fig.tight_layout()
fig.savefig(FIGS / "pc_acierto_por_concepto.png", dpi=120, bbox_inches="tight")
plt.show()

```

```

print("Figura guardada: pc_acierto_por_concepto.png")

import shutil as _shutil
_shutil.copy(FIGS / "pc_acierto_por_tarea_requerida.png", FIGS / "pc_acierto_por

# -----
# 3.8 Histograma score en escala 0-10 (nota de examen)
# -----
# nota_10 = pct_pc * 10 (equivalente a score_pc / max_score * 10)
df_pc["nota_10"] = df_pc["pct_pc"] * 10

fig, ax = plt.subplots(figsize=(8, 4))
sns.histplot(df_pc["nota_10"].dropna(), kde=True, bins=14, color="steelblue", ax
ax.axvline(df_pc["nota_10"].mean(), color="crimson", ls="--", lw=1.5,
            label=f"Media = {df_pc['nota_10'].mean():.2f}")
ax.axvline(df_pc["nota_10"].median(), color="orange", ls=":", lw=1.5,
            label=f"Mediana = {df_pc['nota_10'].median():.2f}")
ax.axvspan(0, 5, alpha=0.06, color="red", zorder=0)
ax.axvspan(5, 10, alpha=0.06, color="green", zorder=0)
ax.axvline(5, color="grey", ls="-", lw=1, alpha=0.5, label="Aprobado (5)")
ax.set_xlim(0, 10)
ax.set_xlabel("Nota (escala 0-10)")
ax.set_ylabel("Frecuencia")
ax.legend(fontsize=9)
ax.set_title("Distribución del Test Inicial PC – Escala 0-10", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "pc_histograma_notas10.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_histograma_notas10.png")

# -----
# 3.9 Violin plot por Clase, Género y NEE – escala 0-10
# -----
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
for ax, (gvar, glabel) in zip(axes, grouping_vars):
    order = sorted(df_pc[gvar].dropna().unique())
    data_sub = df_pc.dropna(subset=[gvar, "nota_10"])
    color_list = [palette.get(g, "steelblue") for g in order]
    vp = ax.violinplot(
        [data_sub.loc[data_sub[gvar] == g, "nota_10"].values for g in order],
        positions=range(len(order)),
        showmedians=True,
    )
    for body, color in zip(vp["bodies"], color_list):
        body.set_facecolor(color)
        body.set_alpha(0.7)
    ax.axhline(5, color="red", ls="--", lw=1, alpha=0.6, label="Aprobado (5)")
    ax.set_xticks(range(len(order)))
    ax.set_xticklabels(order)
    ax.set_ylim(0, 10)
    ax.set_xlabel(glabel)
    ax.set_ylabel("Nota (0-10)")
    ax.set_title(f"Nota PC (0-10) por {gvar}")
    ax.legend(fontsize=8)

fig.suptitle("Score PC inicial en escala 0-10 por grupos", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "pc_violin_notas10_por_grupo.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_violin_notas10_por_grupo.png")

```

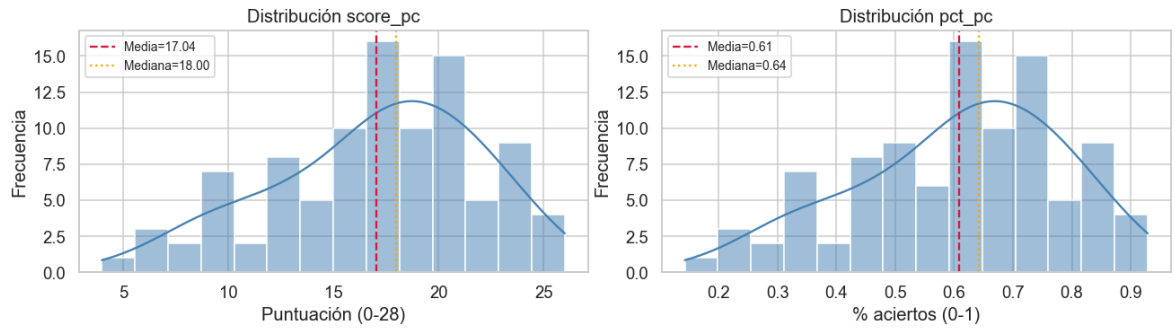
Distribución global del Test Inicial PC

Figura guardada: pc_histograma_global.png

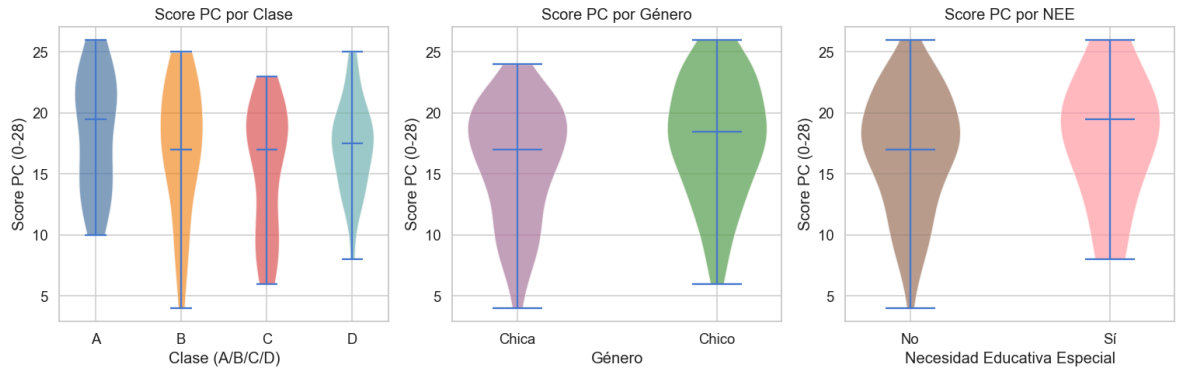
Score PC inicial por grupos

Figura guardada: pc_violin_por_grupo.png

Descriptivo global:

Estadísticos descriptivos globales — PC inicial

Global	N	Media	Mediana	SD	IQR	Min	Max	IC95_lo	IC95_hi
Global	97	17.041	18.000	4.956	7.000	4.000	26.000	16.031	18.010

Descriptivo por grupos:

Descriptivos PC por grupo (Clase / Género / NEE)

Grupo	N	Media	Mediana	SD	IQR	Min	Max	IC95_lo	IC95_hi
A	24	18.500	19.500	4.845	7.250	10.000	26.000	16.542	20.334
B	23	17.087	17.000	5.526	6.000	4.000	25.000	14.696	19.217
C	26	15.462	17.000	5.248	8.750	6.000	23.000	13.422	17.462
D	24	17.250	17.500	3.848	4.500	8.000	25.000	15.792	18.792
Chica	45	16.000	17.000	4.819	7.000	4.000	24.000	14.511	17.378
Chico	52	17.942	18.500	4.940	7.000	6.000	26.000	16.557	19.231
No	89	16.966	17.000	4.932	7.000	4.000	26.000	15.899	17.955
Sí	8	17.875	19.500	5.489	5.000	8.000	26.000	14.250	21.375

Tabla de ítems guardada → outputs\tables\pc_por_item_y_clase.csv

% Acierto por ítem y clase

Pregunta	Global_%acierto	Clase_A_%acierto	Clase_B_%acierto	Clase_C_%acierto	Clase_D_%acierto
1	0.928	0.875	0.913	0.923	0.923
2	0.887	0.875	0.870	0.808	0.808
3	0.825	0.917	0.826	0.692	0.692
4	0.381	0.500	0.217	0.308	0.308
5	0.918	0.917	0.913	0.923	0.923
6	0.897	0.958	0.913	0.846	0.846
7	0.649	0.708	0.696	0.654	0.654
8	0.443	0.542	0.522	0.269	0.269
9	0.918	0.917	0.913	0.846	0.846
10	0.784	0.833	0.739	0.808	0.808
11	0.732	0.833	0.652	0.692	0.692
12	0.247	0.292	0.217	0.231	0.231
13	0.732	0.625	0.826	0.692	0.692
14	0.588	0.583	0.696	0.538	0.538
15	0.299	0.375	0.348	0.308	0.308
16	0.454	0.458	0.522	0.462	0.462
17	0.526	0.625	0.478	0.385	0.385
18	0.660	0.750	0.522	0.654	0.654
19	0.505	0.542	0.478	0.423	0.423
20	0.660	0.625	0.826	0.538	0.538
21	0.598	0.667	0.652	0.577	0.577
22	0.412	0.500	0.348	0.462	0.462
23	0.165	0.167	0.348	0.038	0.038
24	0.660	0.667	0.652	0.538	0.538
25	0.423	0.583	0.391	0.346	0.346
26	0.557	0.708	0.478	0.500	0.500
27	0.598	0.792	0.609	0.462	0.462
28	0.598	0.667	0.522	0.538	0.538

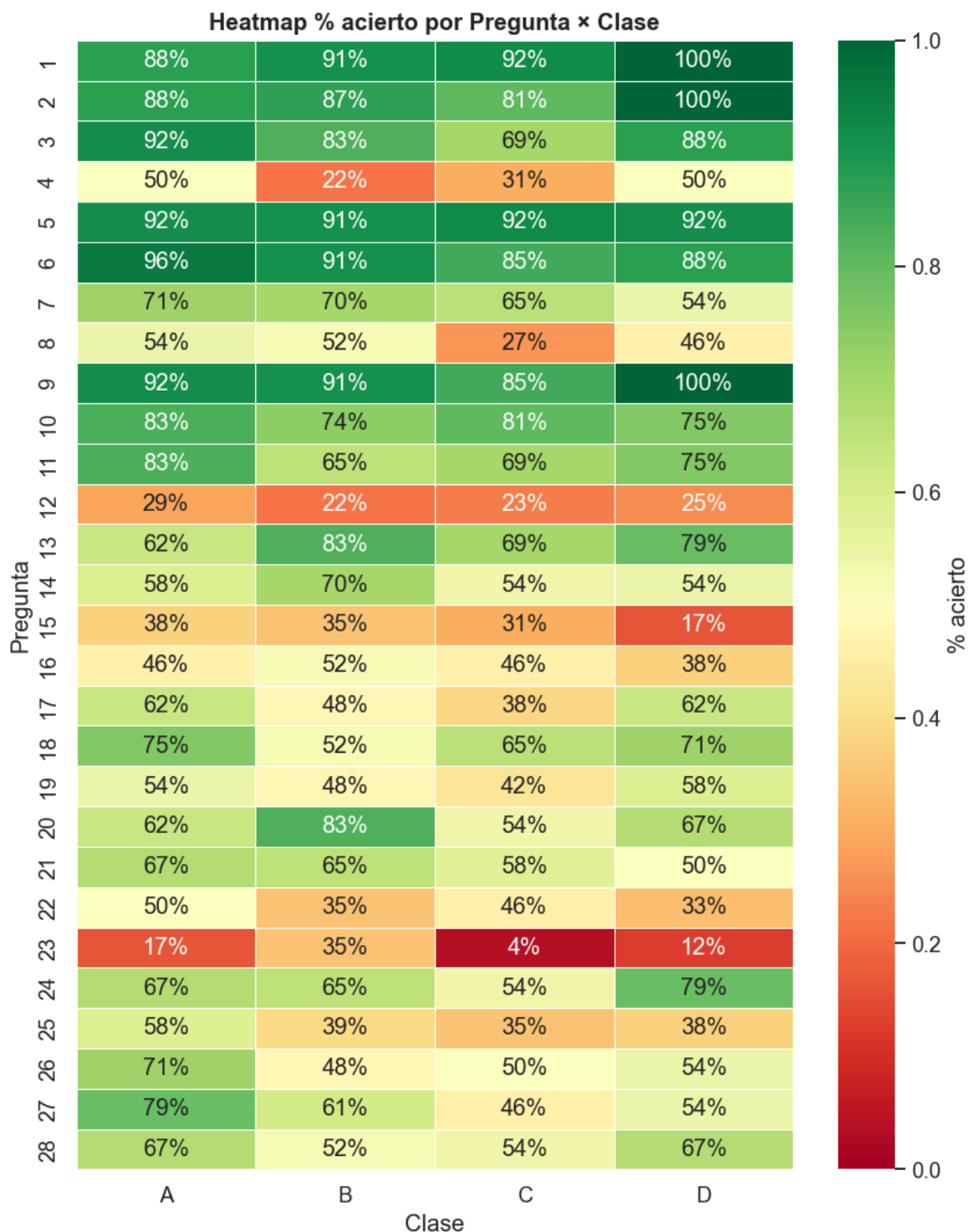


Figura guardada: pc_heatmap_items_clase.png

Tabla por Tarea_requerida → outputs\tables\pc_por_tarea_requerida.csv

Score PC por tarea requerida

Tarea_requerida	N	Media	SD	Min	Max
Secuenciación	97	0.598	0.186	0.071	1.000
Completamiento	97	0.612	0.198	0.000	1.000
Depuración	97	0.633	0.270	0.000	1.000

Tabla por Concepto Computacional → outputs\tables\pc_por_concepto_computacional.csv

Score PC por concepto computacional

Concepto	N	Media	SD	Min	Max
Direcciones	97	0.609	0.177	0.143	0.929
Bucles	97	0.600	0.186	0.136	0.955
Condicionales	97	0.521	0.223	0.083	0.917
Funciones	97	0.544	0.247	0.000	1.000

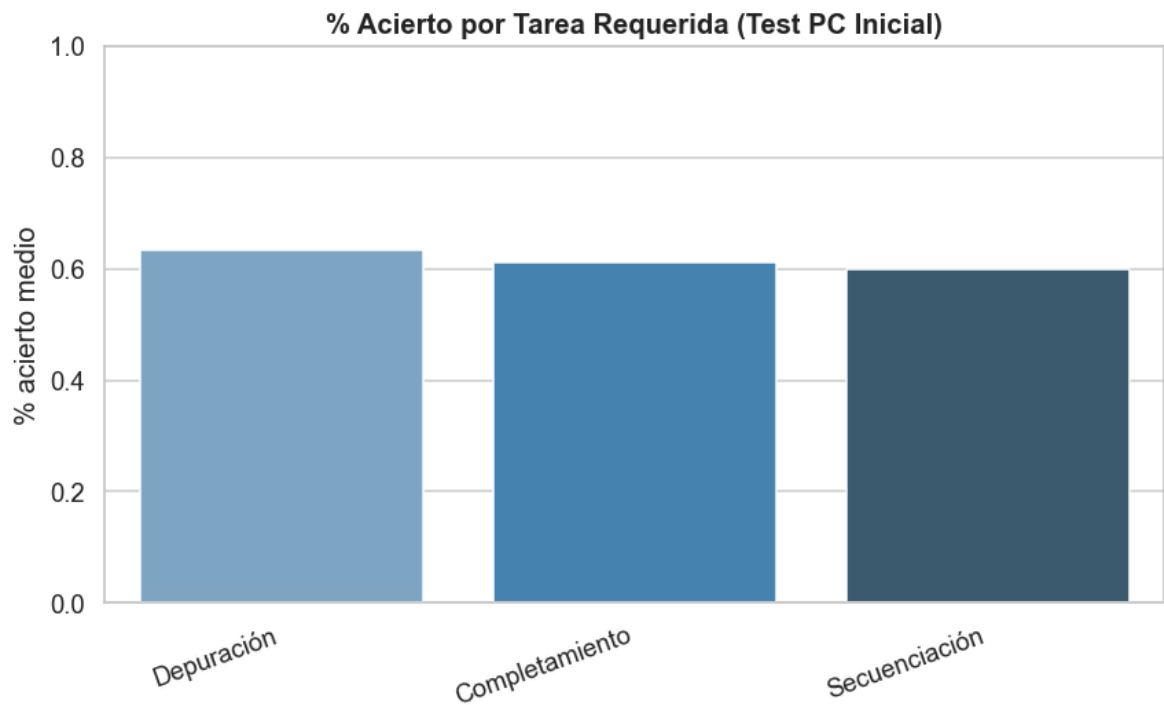


Figura guardada: pc_acierto_por_tarea_requerida.png

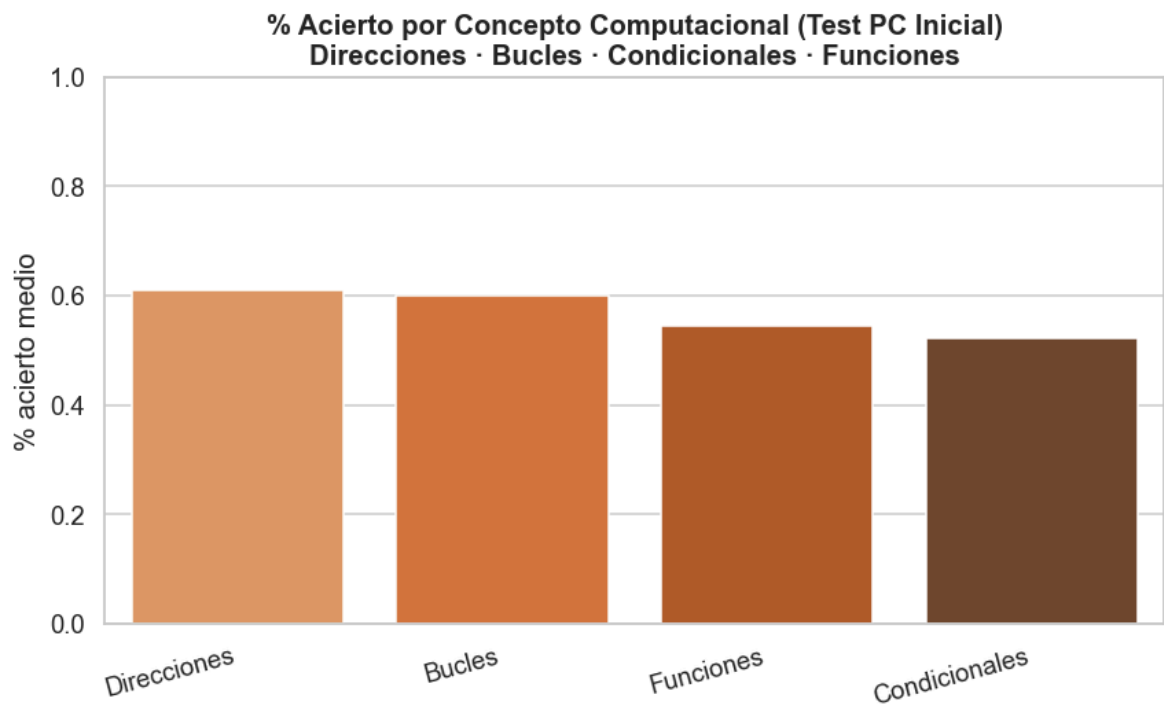


Figura guardada: pc_acierto_por_concepto.png

Out[]: WindowsPath('outputs/figures/pc_acierto_por_dimension.png')

4. Diferencias Significativas PC: Género y NEE

Qué hace esta sección: Contrasta el score PC entre Chicos y Chicas, y entre alumnos con y sin NEE. La selección del test estadístico (t de Welch o Mann-Whitney U) se realiza automáticamente según normalidad (Shapiro-Wilk) y homocedasticidad (Levene). Calcula tamaños de efecto (Hedges g o rank-biserial) con IC95% bootstrap y prueba de permutación para robustez. Documenta la decisión metodológica en una tabla exportable.

Por qué: Verificar si existen diferencias por género o NEE es una cuestión de equidad educativa. La selección automática de test garantiza que se aplique el método estadísticamente adecuado en cada caso, lo que es especialmente crítico cuando el grupo NEE tiene muy pocos alumnos ($N < 10$), situación en la que el p-valor es altamente inestable y se emite una advertencia explícita.

```
In [6]: # -----
# 4.1 Función genérica de contraste con decisión metodológica
# -----
logger.info("--- SECCIÓN 4: Contrastes género y NEE ---")

def contraste_dos_grupos(df, grupo_col, val_col="score_pc",
                        group_labels=None, n_perm=5000, n_boot=2000):
    """
    Contraste automático entre dos grupos:
    - Normalidad por Shapiro
    - Levene para varianzas
    - Welch-t si normal; Mann-Whitney si no
    - Hedges g (si t) o rank-biserial (si MWU)
    - Prueba de permutación
    Retorna un dict con todos los resultados.
    """
    grupos = df[grupo_col].dropna().unique()
    if group_labels is None:
        group_labels = sorted(grupos)

    # Extraer grupos
    data_groups = {}
    for g in group_labels:
        vals = df.loc[df[grupo_col] == g, val_col].dropna().values
        data_groups[g] = vals

    result = {"variable": val_col, "grupo": grupo_col}
    for g, vals in data_groups.items():
        result[f"N_{g}"] = len(vals)
        result[f"media_{g}"] = np.mean(vals) if len(vals) > 0 else np.nan

    if len(data_groups) != 2:
        result["nota"] = "Requiere exactamente 2 grupos"
        return result

    a, b = [data_groups[g] for g in group_labels]
    g1, g2 = group_labels

    # Shapiro
    sw_p_a = stats.shapiro(a).pvalue if len(a) >= 3 else np.nan
    sw_p_b = stats.shapiro(b).pvalue if len(b) >= 3 else np.nan
    result["shapiro_p_" + str(g1)] = sw_p_a
    result["shapiro_p_" + str(g2)] = sw_p_b
```

```

# Decisión de test
test_chosen = choose_test(a, b)
result["test_elegido"] = test_chosen

if test_chosen == "welch_t":
    stat, p = stats.ttest_ind(a, b, equal_var=False)
    eff, eff_ci = hedges_g(a, b)
    result.update({
        "estadístico": stat,
        "p_valor": p,
        "efecto": eff,
        "IC95_efecto_lo": eff_ci[0],
        "IC95_efecto_hi": eff_ci[1],
        "tipo_efecto": "Hedges_g",
    })
else:
    stat, p = stats.mannwhitneyu(a, b, alternative="two-sided")
    eff = rank_biserial(a, b)
    ci_lo, ci_hi = bootstrap_ci(
        np.concatenate([a, b]),
        lambda x: rank_biserial(x[:len(a)], x[len(a):]),
        n_boot=n_boot,
    )
    result.update({
        "estadístico": stat,
        "p_valor": p,
        "efecto": eff,
        "IC95_efecto_lo": ci_lo,
        "IC95_efecto_hi": ci_hi,
        "tipo_efecto": "rank_biserial",
    })

# Prueba de permutación
p_perm = permutation_test(a, b, n_perm=n_perm)
result["p_perm"] = p_perm

# Levene
lev_stat, lev_p = stats.levene(a, b)
result["levене_p"] = lev_p

logger.info(
    f"Contraste {grupo_col} ({g1} vs {g2}): test={test_chosen} "
    f"p={p:.4f} p_perm={p_perm:.4f} efecto={eff:.3f}"
)
return result

# -----
# 4.1 Contraste Género
# -----
res_genero = contraste_dos_grupos(df_pc, "Género", val_col="score_pc",
                                  group_labels=["Chico", "Chica"])
df_contraste_genero = pd.DataFrame([res_genero])
df_contraste_genero.to_csv(TABS / "pc_contraste_genero.csv", index=False, encoding='utf-8')

print("=== Contraste Género ===")
for k, v in res_genero.items():
    print(f" {k}: {v:.4f}" if isinstance(v, float) else f" {k}: {v}")

```

```

# QQ-plot Género
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, gen in zip(axes, ["Chico", "Chica"]):
    vals = df_pc.loc[df_pc["Género"] == gen, "score_pc"].dropna()
    sm.qqplot(vals, line="s", ax=ax, alpha=0.6, markersize=4)
    ax.set_title(f"QQ-plot: {gen} (N={len(vals)})")
fig.suptitle("Normalidad score_pc por Género", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "pc_qqplot_genero.png", dpi=120, bbox_inches="tight")
plt.show()

# -----
# 4.2 Contraste NEE (con advertencia si N<10)
# -----

n_nee_si = (df_pc["NEE"] == "Sí").sum()
n_nee_no = (df_pc["NEE"] == "No").sum()
print(f"\n=== Contraste NEE ===")
print(f"    N(NEE=Sí)={n_nee_si}    N(NEE=No)={n_nee_no}")

decision_nee = {}
if n_nee_si < 10:
    logger.warning(
        f"[NEE] Desequilibrio muestral severo: N(Sí)={n_nee_si}. "
        "Se prioriza descriptivo + bootstrapping. Conclusiones de p-valor con CA
    )
    advertencia_nee = (
        f"[!] **ADVERTENCIA:** El grupo NEE=Sí tiene solo {n_nee_si} alumnos. "
        "Los p-valores son extremadamente poco fiables en muestras tan pequeñas. "
        "Se reportan descriptivos, tamaño de efecto con IC bootstrap y prueba de "
        "NO deben extraerse conclusiones fuertes sobre significación estadística
    )
    print("\n" + advertencia_nee)
    decision_nee["advertencia"] = advertencia_nee
    decision_nee["metodo"] = "MWU + permutación + IC bootstrap (N<10 en NEE=Sí)"
else:
    advertencia_nee = None
    decision_nee["metodo"] = "Selección automática Shapiro + Levene"

res_nee = contraste_dos_grupos(df_pc, "NEE", val_col="score_pc",
                              group_labels=["Sí", "No"])
res_nee["metodo_elegido"] = decision_nee["metodo"]
if advertencia_nee:
    res_nee["advertencia"] = advertencia_nee

df_contraste_nee = pd.DataFrame([res_nee])
df_contraste_nee.to_csv(TABS / "pc_contraste_nee.csv", index=False, encoding="utf-8")

print("\nResultados NEE:")
for k, v in res_nee.items():
    if k != "advertencia":
        print(f"    {k}: {v:.4f}" if isinstance(v, float) else f"    {k}: {v}")

# QQ-plot NEE
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, nee_val in zip(axes, ["Sí", "No"]):
    vals = df_pc.loc[df_pc["NEE"] == nee_val, "score_pc"].dropna()
    if len(vals) >= 3:
        sm.qqplot(vals, line="s", ax=ax, alpha=0.6, markersize=4)
        ax.set_title(f"QQ-plot: NEE={nee_val} (N={len(vals)})")
fig.suptitle("Normalidad score_pc por NEE", fontweight="bold")

```

```

fig.tight_layout()
fig.savefig(FIGS / "pc_qqplot_nee.png", dpi=120, bbox_inches="tight")
plt.show()

# -----
# 4.3 Tabla de decisión metodológica
# -----
df_decision = pd.DataFrame([
    {
        "Contraste": "Género (Chico vs Chica)",
        "N_grupo1": res_genero.get("N_Chico", np.nan),
        "N_grupo2": res_genero.get("N_Chica", np.nan),
        "Test_elegido": res_genero.get("test_elegido", ""),
        "p_valor": res_genero.get("p_valor", np.nan),
        "p_perm": res_genero.get("p_perm", np.nan),
        "Efecto": res_genero.get("efecto", np.nan),
        "Tipo_efecto": res_genero.get("tipo_efecto", ""),
        "Decisión": "Selección automática (Shapiro + Levene)",
    },
    {
        "Contraste": "NEE (Sí vs No)",
        "N_grupo1": n_nee_si,
        "N_grupo2": n_nee_no,
        "Test_elegido": res_nee.get("test_elegido", ""),
        "p_valor": res_nee.get("p_valor", np.nan),
        "p_perm": res_nee.get("p_perm", np.nan),
        "Efecto": res_nee.get("efecto", np.nan),
        "Tipo_efecto": res_nee.get("tipo_efecto", ""),
        "Decisión": decision_nee["metodo"],
    },
])
df_decision.to_csv(TABS / "pc_decision_metodologica.csv", index=False, encoding=
print(f"\nTabla decisión metodológica → {TABS / 'pc_decision_metodologica.csv'}")
display(style_table(df_decision, caption="Decisión metodológica: Género y NEE",

# -----
# 4.4 Boxplot comparativo con IC bootstrap
# -----
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, (gcol, glabel) in zip(axes, [("Género", "Género"), ("NEE", "NEE")]):
    order = sorted(df_pc[gcol].dropna().unique())
    means, ci_los, ci_his, labels = [], [], [], []
    for g in order:
        x = df_pc.loc[df_pc[gcol] == g, "score_pc"].dropna().values
        if len(x) == 0:
            continue
        labels.append(f"{g}\n(N={len(x)})")
        means.append(np.mean(x))
        lo, hi = bootstrap_ci(x, np.mean, n_boot=2000)
        ci_los.append(lo)
        ci_his.append(hi)
    # Boxplot base
    data_list = [df_pc.loc[df_pc[gcol] == g, "score_pc"].dropna().values for g in order]
    bp = ax.boxplot(data_list, labels=labels, patch_artist=True,
                    medianprops=dict(color="red", lw=2))
    colors = ["#4E79A7", "#F28E2B", "#E15759"]
    for patch, col in zip(bp["boxes"], colors):
        patch.set_facecolor(col)
        patch.set_alpha(0.5)
    # Añadir medias + IC

```

```

for xi, (m, lo, hi) in enumerate(zip(means, ci_los, ci_his), start=1):
    ax.plot(xi, m, "D", color="navy", ms=6, zorder=5)
    ax.vlines(xi, lo, hi, color="navy", lw=2, zorder=5)
ax.set_title(f"Score PC por {glabel}")
ax.set_ylabel("Score PC (0-28)")

fig.suptitle("Contraste Score PC por Género y NEE\n(* = media; barras = IC95% bo
fig.tight_layout()
fig.savefig(FIGS / "pc_contraste_grupos.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_contraste_grupos.png")

```

2026-05-22 13:25:40,628 [INFO] --- SECCIÓN 4: Contrastes género y NEE ---

2026-05-22 13:25:43,826 [INFO] Contraste Género (Chico vs Chica): test=mann_whitn
 ey p=0.0694 p_perm=0.0556 efecto=-0.215

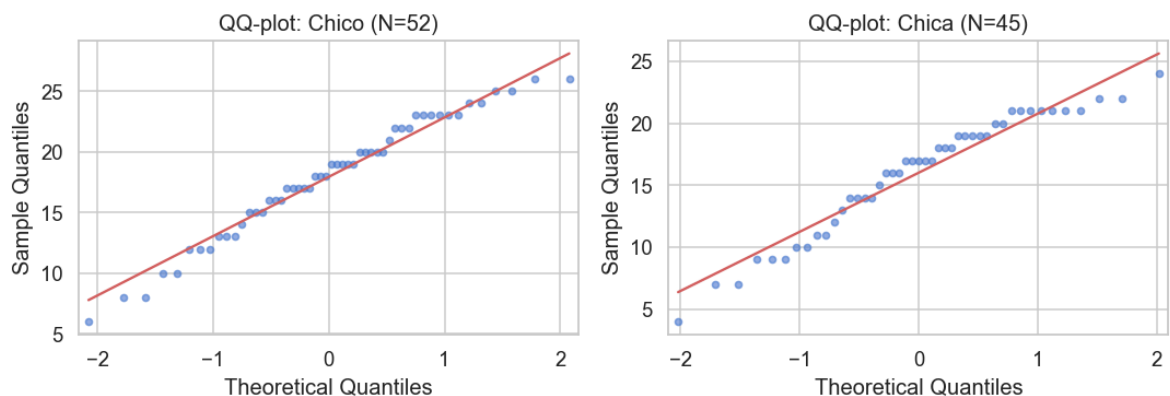
=== Contraste Género ===

```

variable: score_pc
grupo: Género
N_Chico: 52
media_Chico: 17.9423
N_Chica: 45
media_Chica: 16.0000
shapiro_p_Chico: 0.1801
shapiro_p_Chica: 0.0285
test_elegido: mann_whitney
estadístico: 1421.0000
p_valor: 0.0694
efecto: -0.2145
IC95_efecto_lo: -0.2210
IC95_efecto_hi: 0.2184
tipo_efecto: rank_biserial
p_perm: 0.0556
levene_p: 0.8222

```

Normalidad score_pc por Género



=== Contraste NEE ===

N(NEE=Sí)=8 N(NEE=No)=89

2026-05-22 13:25:44,487 [WARNING] [NEE] Desequilibrio muestral severo: N(Sí)=8. Se prioriza descriptivo + bootstrapping. Conclusiones de p-valor con CAUTELA EXTREMA.

[!] ****ADVERTENCIA:**** El grupo NEE=Sí tiene solo 8 alumnos. Los p-valores son extremadamente poco fiables en muestras tan pequeñas. Se reportan descriptivos, tamaño de efecto con IC bootstrap y prueba de permutación. NO deben extraerse conclusiones fuertes sobre significación estadística.

2026-05-22 13:25:47,596 [INFO] Contraste NEE (Sí vs No): test=mann_whitney p=0.6221 p_perm=0.6410 efecto=-0.107

Resultados NEE:

variable: score_pc

grupo: NEE

N_Sí: 8

media_Sí: 17.8750

N_No: 89

media_No: 16.9663

shapiro_p_Sí: 0.7445

shapiro_p_No: 0.0427

test_elegido: mann_whitney

estadístico: 394.0000

p_valor: 0.6221

efecto: -0.1067

IC95_efecto_lo: -0.4087

IC95_efecto_hi: 0.4143

tipo_efecto: rank_biserial

p_perm: 0.6410

levene_p: 0.9510

metodo_elegido: MWU + permutación + IC bootstrap (N<10 en NEE=Sí)

Normalidad score_pc por NEE

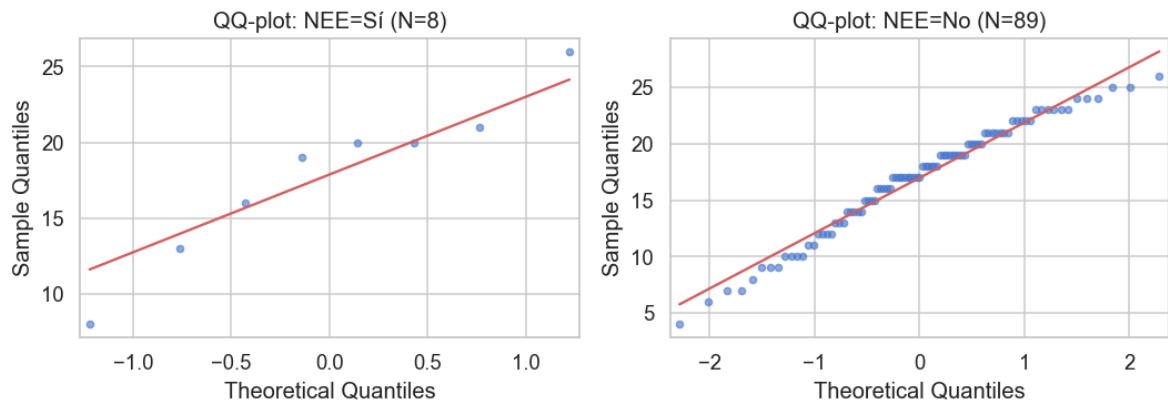


Tabla decisión metodológica → outputs\tables\pc_decision_metodologica.csv

Decisión metodológica: Género y NEE

Contraste	N_grupo1	N_grupo2	Test_elegido	p_valor	p_perm	Efecto	Tipo_efe
Género (Chico vs Chica)	52	45	mann_whitney	0.069	0.056	-0.215	rank_bise
NEE (Sí vs No)	8	89	mann_whitney	0.622	0.641	-0.107	rank_bise

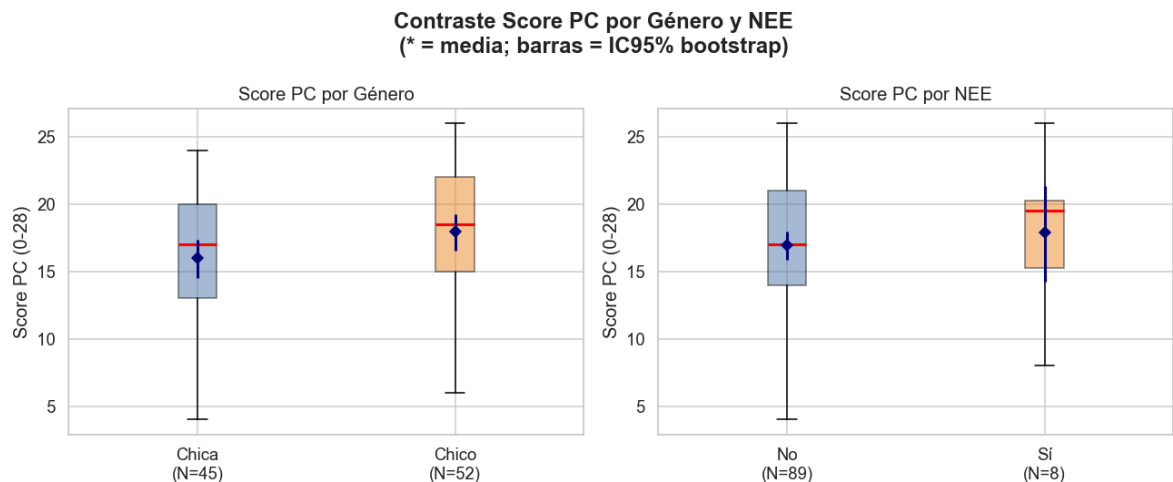


Figura guardada: pc_contraste_grupos.png

5. Autopercepción vs Rendimiento Real (PC)

Qué hace esta sección: Contrasta la autopercepción de los alumnos sobre su rendimiento en PC con su rendimiento real. Como el dataset de autopercepción no tiene columna ID individual, la comparación se realiza a nivel **agregado de grupo** (por género, NEE o nivel de PC): se calcula la diferencia entre la autopercepción media del grupo y su score PC medio real, identificando si el grupo se sobreestima, subestima o está bien calibrado.

Por qué: La calibración metacognitiva (conocer bien el propio nivel) es un factor clave en el aprendizaje. Alumnos que se sobrestiman pueden no esforzarse lo suficiente; alumnos que se subestiman pueden presentar baja autoeficacia. Esta sección identifica grupos con desajustes sistemáticos de autopercepción.

Limitación explícita: Sin ID individual no se pueden calcular correlaciones alumno a alumno. Todos los análisis se realizan a nivel de media de grupo y así se reportan.

```
In [7]: # -----
# 5.1 Preparar dataset de autopercepción
# -----
logger.info("--- SECCIÓN 5: Autopercepción ---")
```

```

df_auto = df_auto_raw.copy()
df_auto.columns = [c.strip() for c in df_auto.columns]

# Detectar columnas clave
col_auto_score = [c for c in df_auto.columns if "salido" in c.lower() or "consi
col_auto_dif = [c for c in df_auto.columns if "difícil" in c.lower() or "difi
col_auto_genero = [c for c in df_auto.columns if "género" in c.lower() or "gener
col_auto_nee = [c for c in df_auto.columns if "necesidad" in c.lower() or "ne

col_auto_score = col_auto_score[0] if col_auto_score else None
col_auto_dif = col_auto_dif[0] if col_auto_dif else None
col_auto_genero = col_auto_genero[0] if col_auto_genero else None
col_auto_nee = col_auto_nee[0] if col_auto_nee else None

df_auto.rename(columns={
    col_auto_score: "autopercepcion",
    col_auto_dif: "dificultad",
    col_auto_genero: "Género",
    col_auto_nee: "NEE",
}, inplace=True, errors="ignore")

# Limpiar y validar rangos 0-10
for col in ["autopercepcion", "dificultad"]:
    if col in df_auto.columns:
        df_auto[col] = ensure_numeric(df_auto[col], col)
        out_range = df_auto[col].notna() & ~df_auto[col].between(0, 10)
        if out_range.sum() > 0:
            log_anomaly(f"Autopercepción '{col}': {out_range.sum()} valores fuer
            df_auto.loc[out_range, col] = np.nan

# Estandarizar Género y NEE en autopercepción
df_auto["Género"] = df_auto["Género"].str.strip().str.lower().map(genero_map).fi
df_auto["NEE"] = df_auto["NEE"].str.strip().str.lower().map(neo_map).fillna(d

print(f"Autopercepción: {df_auto.shape}")
display(style_table(
    df_auto[["autopercepcion", "dificultad", "Género", "NEE"]].describe().round(
    caption="Estadísticos descriptivos: autopercepción",
))

# -----
# 5.2 Verificar si hay ID común con PC
# -----

id_cols_pc = set(df_pc.columns)
id_cols_auto = set(df_auto.columns)
common_id = id_cols_pc & id_cols_auto - {"Género", "NEE"}

has_id = any(c.lower() in ["id", "alumno", "número", "numero"] for c in common_id)
if has_id:
    logger.info("[AUTOPERCEPCIÓN] Se encontró ID común → emparejamiento individu
    id_col_name = [c for c in common_id if c.lower() in ["id", "alumno", "número
    df_merged = df_pc.merge(df_auto, on=id_col_name, how="inner",
                            suffixes=("_pc", "_auto"))
    logger.info(f" Emparejados: {len(df_merged)} alumnos")
    CAN_LINK = True
else:
    logger.warning(
        "[AUTOPERCEPCIÓN] No hay columna ID común con PC. "
        "No se realizará emparejamiento individual. "
        "Análisis EXCLUSIVAMENTE agregado por grupo. "

```

```

        "LIMITACIÓN: no es posible relacionar autopercepción con rendimiento a n
    )
    print("\n[!] Sin ID común → Análisis AGREGADO por grupo (no individual).")
    print("    LIMITACIÓN: La comparación autopercepción vs rendimiento real")
    print("    se realiza a nivel de media de grupo, NO a nivel de alumno.")
    CAN_LINK = False

# -----
# 5.3 Descriptivos de autopercepción
# -----
desc_auto_rows = []
for gcol in [None, "Género", "NEE"]:
    if gcol is None:
        subsets_a = [("Global", df_auto)]
    else:
        subsets_a = [(g, df_auto[df_auto[gcol] == g]) for g in sorted(df_auto[gcol].unique())]
    for label, sub in subsets_a:
        for vcol in ["autopercepcion", "dificultad"]:
            if vcol not in sub.columns:
                continue
            x = sub[vcol].dropna().values
            ci_lo, ci_hi = bootstrap_ci(x, np.mean, n_boot=2000)
            desc_auto_rows.append({
                "Agrupación": gcol or "Global",
                "Grupo": label,
                "Variable": vcol,
                "N": len(x),
                "Media": np.mean(x) if len(x) else np.nan,
                "Mediana": np.median(x) if len(x) else np.nan,
                "SD": np.std(x, ddof=1) if len(x) > 1 else np.nan,
                "IC95_lo": ci_lo,
                "IC95_hi": ci_hi,
            })

df_desc_auto = pd.DataFrame(desc_auto_rows)
df_desc_auto.to_csv(TABS / "autopercepcion_descriptivo.csv", index=False, encoding='utf-8')
print(f"\nDescriptivos autopercepción → {TABS / 'autopercepcion_descriptivo.csv'}")
display(style_table(df_desc_auto, caption="Descriptivos autopercepción por grupo"))

# -----
# 5.4 Análisis agregado: media autopercepción vs media pct_pc por grupo
# -----
agg_rows = []
for gcol in ["Género", "NEE"]:
    grupos_pc = df_pc.groupby(gcol)["pct_pc"].mean()
    grupos_auto = df_auto.groupby(gcol)["autopercepcion"].mean() / 10 # normalizada
    for g in grupos_pc.index:
        if g in grupos_auto.index:
            agg_rows.append({
                "Variable_agrupacion": gcol,
                "Grupo": g,
                "media_pct_pc": grupos_pc[g],
                "media_autopercepcion_norm": grupos_auto[g],
                "diferencia_agregada": grupos_auto[g] - grupos_pc[g],
                "nota": "Comparación a nivel de grupo, NO individual",
            })

df_agg_auto = pd.DataFrame(agg_rows)
print("\nComparación agregada por grupo (sin emparejamiento individual):")
display(style_table(df_agg_auto, caption="Comparación agregada autopercepción vs"))

```

```

# -----
# 5.5 Contrastes de autopercepción/dificultad por Género y NEE
# -----
contrast_auto_rows = []
for gcol in ["Género", "NEE"]:
    for vcol in ["autopercepcion", "dificultad"]:
        if vcol not in df_auto.columns:
            continue
        groups = sorted(df_auto[gcol].dropna().unique())
        if len(groups) == 2:
            res = contraste_dos_grupos(df_auto, gcol, val_col=vcol, group_labels=groups)
            res["variable"] = vcol
            contrast_auto_rows.append(res)

df_contraste_auto = pd.DataFrame(contrast_auto_rows)
df_contraste_auto.to_csv(TABS / "autopercepcion_contrastes.csv", index=False, encoding='utf-8')
print(f"\nContrastes autopercepción → {TABS / 'autopercepcion_contrastes.csv'}")

# -----
# 5.6 Figuras autopercepción
# -----
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, vcol in zip(axes, ["autopercepcion", "dificultad"]):
    if vcol not in df_auto.columns:
        continue
    order = sorted(df_auto["Género"].dropna().unique())
    data_list = [df_auto.loc[df_auto["Género"] == g, vcol].dropna().values for g in order]
    bp = ax.boxplot(data_list, labels=[f"{g}\n(N={len(d)})" for g, d in zip(order, data_list)],
                    patch_artist=True)
    for patch, col in zip(bp["boxes"], ["#4E79A7", "#F28E2B"]):
        patch.set_facecolor(col)
        patch.set_alpha(0.6)
    ax.set_title(f"'{vcol}' por Género")
    ax.set_ylabel("Puntuación (0-10)")
    ax.set_ylim(0, 10.5)

fig.suptitle("Autopercepción y Dificultad por Género", fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "autopercepcion_por_genero.png", dpi=120, bbox_inches="tight")
plt.show()

# Comparación agregada barplot
if not df_agg_auto.empty:
    fig2, ax2 = plt.subplots(figsize=(8, 4))
    df_agg_g = df_agg_auto[df_agg_auto["Variable_agrupacion"] == "Género"].copy()
    x = np.arange(len(df_agg_g))
    w = 0.35
    ax2.bar(x - w/2, df_agg_g["media_pct_pc"], w, label="Rendimiento real (pct_pc)")
    ax2.bar(x + w/2, df_agg_g["media_autopercepcion_norm"], w, label="Autopercepción (media)")
    ax2.set_xticks(x)
    ax2.set_xticklabels(df_agg_g["Grupo"])
    ax2.set_ylim(0, 1)
    ax2.set_ylabel("Proporción (0-1)")
    ax2.set_title("Autopercepción vs Rendimiento Real (medias por grupo)")
    ax2.legend()
    note = "[!] Comparación agregada por grupo, NO individual"
    ax2.text(0.01, -0.15, note, transform=ax2.transAxes, fontsize=8, color="gray")
    fig2.tight_layout()
    fig2.savefig(FIGS / "autopercepcion_vs_rendimiento_agregado.png", dpi=120, b

```

```

plt.show()

print("Figuras autopercepción guardadas OK")

# Análisis individual si hay ID
if CAN_LINK:
    df_merged["autopercepcion_norm"] = df_merged["autopercepcion"] / 10
    df_merged["error_calibracion"] = df_merged["autopercepcion_norm"] - df_merged["pct_pc"]
    r_pearson, p_pearson = stats.pearsonr(
        df_merged["autopercepcion_norm"].dropna(),
        df_merged["pct_pc"].dropna()
    )
    r_spearman, p_spearman = stats.spearmanr(
        df_merged["autopercepcion_norm"].dropna(),
        df_merged["pct_pc"].dropna()
    )
    print(f"\nCorrelaciones (N={len(df_merged)}):")
    print(f"   Pearson r={r_pearson:.3f} (p={p_pearson:.4f})")
    print(f"   Spearman p={r_spearman:.3f} (p={p_spearman:.4f})")

    fig3, ax3 = plt.subplots(figsize=(6, 5))
    ax3.scatter(df_merged["pct_pc"], df_merged["autopercepcion_norm"], alpha=0.5)
    m, b = np.polyfit(df_merged["pct_pc"].dropna(), df_merged["autopercepcion_norm"].dropna(), 1)
    x_line = np.linspace(0, 1, 100)
    ax3.plot(x_line, m * x_line + b, "r-", lw=1.5, label=f"r={r_pearson:.2f}")
    ax3.set_xlabel("Rendimiento real (pct_pc)")
    ax3.set_ylabel("Autopercepción normalizada")
    ax3.set_title("Autopercepción vs Rendimiento Real (individual)")
    ax3.legend()
    fig3.tight_layout()
    fig3.savefig(FIGS / "autopercepcion_scatter_individual.png", dpi=120, bbox_inches='tight')
    plt.show()

```

2026-05-22 13:25:49,548 [INFO] --- SECCIÓN 5: Autopercepción ---

Autopercepción: (97, 4)

Estadísticos descriptivos: autopercepción

estadístico	autopercepcion	dificultad
count	97.000	97.000
mean	7.270	5.910
std	1.880	2.650
min	0.000	0.000
25%	6.000	4.000
50%	7.000	6.000
75%	8.000	8.000
max	10.000	10.000

2026-05-22 13:25:49,602 [WARNING] [AUTOPERCEPCIÓN] No hay columna ID común con P C. No se realizará emparejamiento individual. Análisis EXCLUSIVAMENTE agregado por grupo. LIMITACIÓN: no es posible relacionar autopercepción con rendimiento a nivel individuo.

[!] Sin ID común → Análisis AGREGADO por grupo (no individual).
LIMITACIÓN: La comparación autopercepción vs rendimiento real se realiza a nivel de media de grupo, NO a nivel de alumno.

Descriptivos autopercepción → outputs\tables\autopercepcion_descriptivo.csv

Descriptivos autopercepción por grupo

Agrupación	Grupo	Variable	N	Media	Mediana	SD	IC95_lo	IC95_h
Global	Global	autopercepcion	97	7.268	7.000	1.879	6.876	7.610
Global	Global	dificultad	97	5.907	6.000	2.650	5.381	6.450
Género	Chica	autopercepcion	45	6.933	7.000	1.671	6.400	7.400
Género	Chica	dificultad	45	6.867	7.000	2.191	6.200	7.510
Género	Chico	autopercepcion	52	7.558	8.000	2.014	6.981	8.070
Género	Chico	dificultad	52	5.077	5.000	2.750	4.346	5.820
NEE	No	autopercepcion	89	7.202	7.000	1.896	6.787	7.580
NEE	No	dificultad	89	6.022	6.000	2.598	5.483	6.550
NEE	Sí	autopercepcion	8	8.000	8.000	1.604	7.000	9.000
NEE	Sí	dificultad	8	4.625	4.000	3.068	2.750	6.620

Comparación agregada por grupo (sin emparejamiento individual):

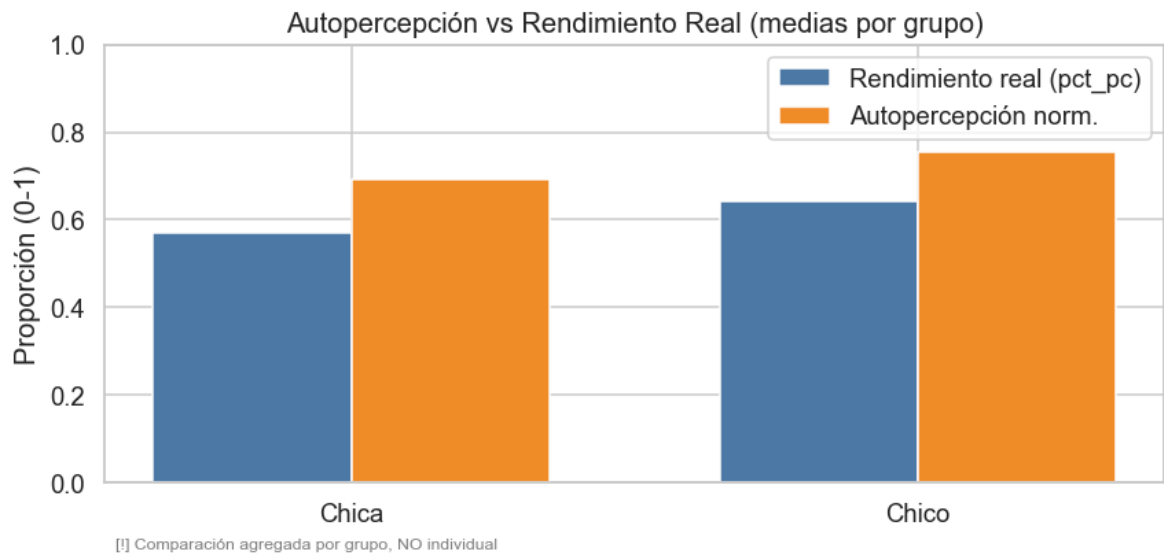
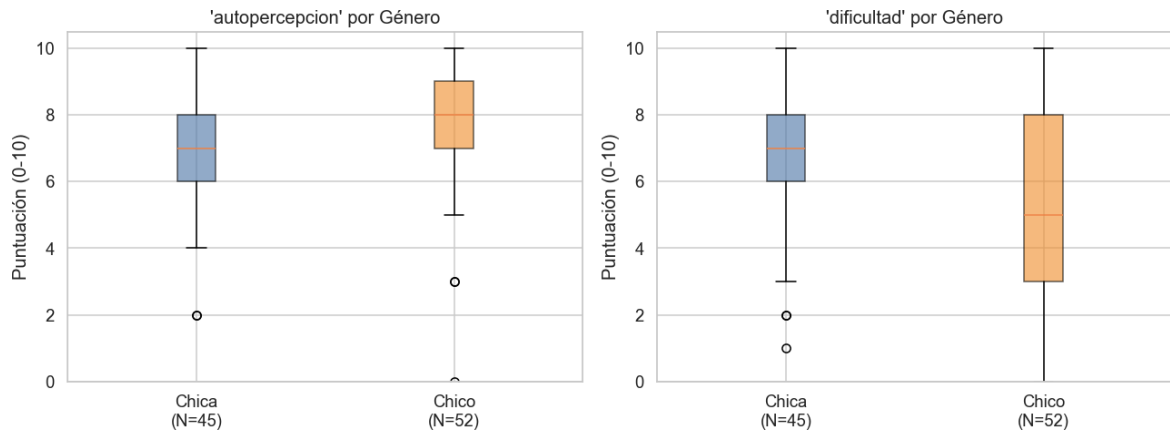
Comparación agregada autopercepción vs rendimiento real

Variable_agrupacion	Grupo	media_pct_pc	media_autopercepcion_norm	diferencia_agreg
Género	Chica	0.571	0.693	0.122
Género	Chico	0.641	0.756	0.115
NEE	No	0.606	0.720	0.114
NEE	Sí	0.638	0.800	0.162

2026-05-22 13:25:53,978 [INFO] Contraste Género (Chica vs Chico): test=mann_whitney p=0.0322 p_perm=0.1042 efecto=0.249
 2026-05-22 13:25:56,661 [INFO] Contraste Género (Chica vs Chico): test=mann_whitney p=0.0015 p_perm=0.0004 efecto=-0.372
 2026-05-22 13:25:59,097 [INFO] Contraste NEE (No vs Sí): test=mann_whitney p=0.3128 p_perm=0.2886 efecto=0.213
 2026-05-22 13:26:01,541 [INFO] Contraste NEE (No vs Sí): test=mann_whitney p=0.1632 p_perm=0.1626 efecto=-0.298

Contrastes autopercepción → outputs\tables\autopercepcion_contrastes.csv

Autopercepción y Dificultad por Género



[!] Comparación agregada por grupo, NO individual

Figuras autopercepción guardadas OK

6. Nivel PC por Clase: Bajo / Medio / Alto (Terciles)

Qué hace esta sección: Calcula la media de `pct_pc` de cada clase, las ordena y les asigna un nivel categórico (**Bajo / Medio / Alto**) mediante terciles con `pd.qcut`. Si hay empates, recurre a un ranking denso con cortes manuales. El nivel queda unido al dataset de alumnos PC y se convierte en la variable moderadora de las secciones 7 (Historia) y 8 (Votaciones).

Por qué: Con solo 4 clases, el nivel PC no puede usarse como variable cuantitativa continua entre grupos. La categorización en terciles permite analizar si las clases con mayor rendimiento en PC también aprenden más Historia o valoran las actividades de

forma distinta. La asignación solo se aplica a las clases de Echegaray para no imputar niveles a otros centros sin datos de PC.

```
In [8]: # -----
# 6.1 Media de pct_pc por clase
# -----
logger.info("---- SECCIÓN 6: Nivel PC por clase ----")

df_nivel_clase = (
    df_pc.groupby("Clase")["pct_pc"]
    .agg(["mean", "count", "std"])
    .reset_index()
    .rename(columns={"mean": "mean_pct_pc", "count": "N", "std": "sd_pct_pc"})
    .sort_values("Clase")
)

# Añadir Centro si existe en el CSV de PC
if "Centro" in df_pc.columns or "Colegio" in df_pc.columns:
    centro_col = "Centro" if "Centro" in df_pc.columns else "Colegio"
    centros = df_pc.groupby("Clase")[centro_col].first().reset_index()
    df_nivel_clase = df_nivel_clase.merge(centros, on="Clase", how="left")
    logger.info(f"Centro detectado en PC: {centros[centro_col].unique()}")
else:
    df_nivel_clase["Centro"] = "Echegaray" # único centro en este dataset PC
    logger.info("Columna 'Centro' no presente en PC CSV → asignado 'Echegaray'")

print("Media pct_pc por Clase:")
display(style_table(df_nivel_clase, caption="Media pct_pc por clase"))

# -----
# 6.2 Asignación de nivel con terciles
# -----
n_clases = len(df_nivel_clase)
if n_clases < 3:
    logger.warning("[NIVEL_PC] Pocas clases para terciles; usando ranking denso")
    df_nivel_clase["rank_pc"] = df_nivel_clase["mean_pct_pc"].rank(method="dense")
    max_rank = df_nivel_clase["rank_pc"].max()
    cut1 = max_rank / 3
    cut2 = 2 * max_rank / 3
    df_nivel_clase["nivel_pc"] = pd.cut(
        df_nivel_clase["rank_pc"],
        bins=[-np.inf, cut1, cut2, np.inf],
        labels=["Bajo", "Medio", "Alto"],
    )
else:
    try:
        df_nivel_clase["nivel_pc"] = pd.qcut(
            df_nivel_clase["mean_pct_pc"],
            q=3,
            labels=["Bajo", "Medio", "Alto"],
            duplicates="drop",
        )
    except ValueError:
        logger.warning("[NIVEL_PC] qcut falló (valores repetidos). Usando ranking")
        df_nivel_clase["rank_pc"] = df_nivel_clase["mean_pct_pc"].rank(method="dense")
        max_rank = df_nivel_clase["rank_pc"].max()
        df_nivel_clase["nivel_pc"] = pd.cut(
            df_nivel_clase["rank_pc"],
            bins=[-np.inf, max_rank / 3, 2 * max_rank / 3, np.inf],
```

```

        labels=["Bajo", "Medio", "Alto"],
    )

logger.info("Asignación nivel PC por clase:")
for _, row in df_nivel_clase.iterrows():
    logger.info(f" Clase={row['Clase']} mean_pct={row['mean_pct_pc']:.3f} nivel

df_nivel_clase.to_csv(TABS / "nivel_pc_por_clase.csv", index=False, encoding="ut
print(f"\nNivel PC por clase → {TABS / 'nivel_pc_por_clase.csv'}")
display(style_table(df_nivel_clase, caption="Nivel PC asignado por clase (tercil

# -----
# 6.3 Unir nivel_pc al dataset de alumnos PC
# -----

df_pc = df_pc.merge(df_nivel_clase[["Clase", "nivel_pc"]], on="Clase", how="left
print("\nDistribución nivel_pc en alumnos PC:")
display(style_table(
    df_pc["nivel_pc"].value_counts().to_frame().reset_index(),
    caption="Alumnos PC por nivel de clase",
))

# Figura barplot niveles
fig, ax = plt.subplots(figsize=(7, 4))
colors_nivel = {"Bajo": "#E15759", "Medio": "#F28E2B", "Alto": "#59A14F"}
df_nivel_clase_plot = df_nivel_clase.sort_values("mean_pct_pc")
ax.barh(
    df_nivel_clase_plot["Clase"],
    df_nivel_clase_plot["mean_pct_pc"],
    color=[colors_nivel.get(str(n), "gray") for n in df_nivel_clase_plot["nivel_
)
ax.set_xlabel("Media % aciertos PC")
ax.set_title("Nivel PC por Clase (terciles sobre media de clase)", fontweight="b
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=v, label=k) for k, v in colors_nivel.items()]
ax.legend(handles=legend_elements, title="Nivel PC")
for i, row in df_nivel_clase_plot.iterrows():
    ax.text(row["mean_pct_pc"] + 0.005, list(df_nivel_clase_plot["Clase"]).index
            f"{row['mean_pct_pc']:.2f}", va="center", fontsize=9)
fig.tight_layout()
fig.savefig(FIGS / "pc_nivel_por_clase.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: pc_nivel_por_clase.png")

```

2026-05-22 13:26:02,491 [INFO] --- SECCIÓN 6: Nivel PC por clase ---

2026-05-22 13:26:02,496 [INFO] Columna 'Centro' no presente en PC CSV → asignado 'Echegaray'

Media pct_pc por Clase:

Media pct_pc por clase

Clase	mean_pct_pc	N	sd_pct_pc	Centro
A	0.661	24	0.173	Echegaray
B	0.610	23	0.197	Echegaray
C	0.552	26	0.187	Echegaray
D	0.616	24	0.137	Echegaray

2026-05-22 13:26:02,509 [INFO] Asignación nivel PC por clase:
 2026-05-22 13:26:02,511 [INFO] Clase=A mean_pct=0.661 nivel=Alto
 2026-05-22 13:26:02,513 [INFO] Clase=B mean_pct=0.610 nivel=Bajo
 2026-05-22 13:26:02,516 [INFO] Clase=C mean_pct=0.552 nivel=Bajo
 2026-05-22 13:26:02,518 [INFO] Clase=D mean_pct=0.616 nivel=Medio

Nivel PC por clase → outputs\tables\nivel_pc_por_clase.csv

Nivel PC asignado por clase (terciles)

Clase	mean_pct_pc	N	sd_pct_pc	Centro	nivel_pc
A	0.661	24	0.173	Echegaray	Alto
B	0.610	23	0.197	Echegaray	Bajo
C	0.552	26	0.187	Echegaray	Bajo
D	0.616	24	0.137	Echegaray	Medio

Distribución nivel_pc en alumnos PC:

Alumnos PC por nivel de clase

nivel_pc	count
Bajo	49
Medio	24
Alto	24

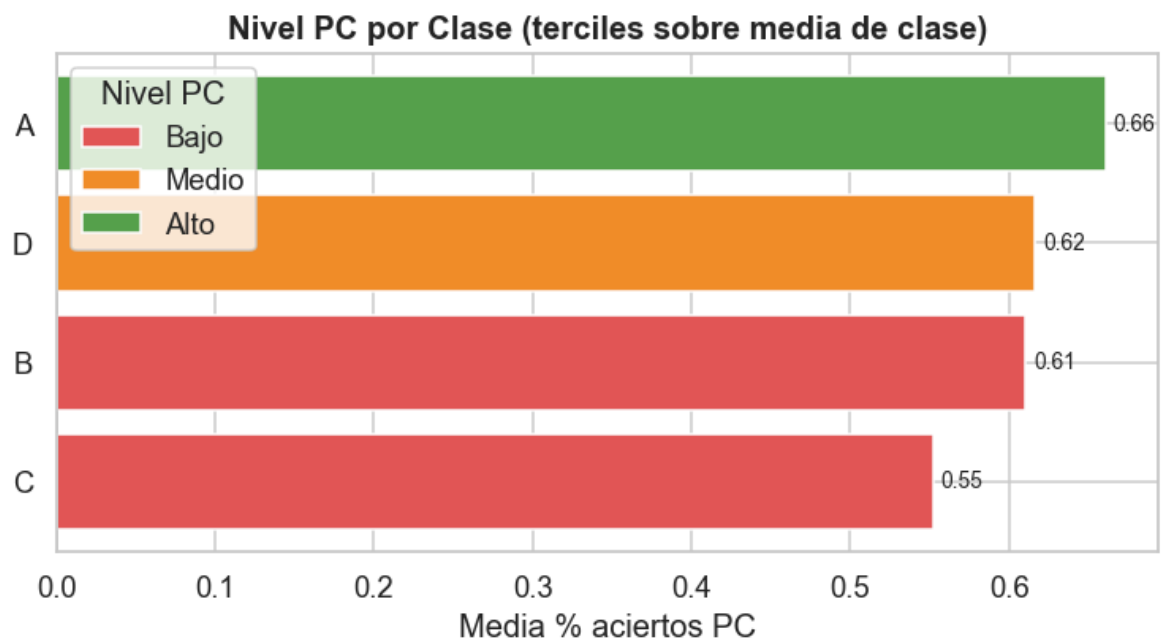


Figura guardada: pc_nivel_por_clase.png

7. Historia (Excel): Análisis Enriquecido + Nivel PC + Categorías

Qué hace esta sección: Realiza el análisis completo del aprendizaje de Historia en diseño pre-post con **muestras independientes**. El índice **Alumno** en los datos representa el orden de respuesta —no identifica al mismo estudiante en tests inicial y final—, por lo que no es posible emparejar observaciones individuales.

Subsección	Contenido
7.1–7.2	Preparación y limpieza de tests inicial y final; diseño de muestras independientes por clase
7.3	Cálculo de ganancia absoluta por clase y Hake grupal (sobre medias de clase, no individual)
7.4	Enriquecimiento con nivel PC de la clase (solo Echegaray) y categorías temáticas de ítems
7.5	Descriptivos globales y por clase; boxplots, histogramas, heatmap categoría × nivel PC
7.6–7.8	Inferencia: Welch-t / Mann-Whitney U (inicial vs final, indep.); ANOVA/KW por nivel PC; modelo OLS
7.9–7.14	Psicometría por ítem (dificultad, discriminación, co-errores); comparaciones entre clases

Por qué: Integrar el nivel PC como moderador de la ganancia en Historia permite responder si las clases con mejor pensamiento computacional también aprenden más Historia, lo que revelaría una posible transferencia de capacidades cognitivas entre dominios. La ganancia de Hake corrige el efecto techo y permite comparar clases con distintos niveles de partida.

```
In [9]: # -----
# 7.1 Preparar Historia Inicial y Final
# -----
logger.info("--- SECCIÓN 7: Historia (muestras independientes) ---")

def prep_historia(df_raw, tipo="Inicial"):
    """Limpia y prepara un test de historia."""
    df = df_raw.copy()
    df.columns = [str(c).strip() for c in df.columns]
    p_cols = [c for c in df.columns if re.match(r"^\d+$", c)]
    total_col_raw = [c for c in df.columns if "TOTAL" in c.upper()]
    total_col = total_col_raw[0] if total_col_raw else None
    if total_col and total_col != "TOTAL":
        df.rename(columns={total_col: "TOTAL"}, inplace=True)
        logger.info(f" Historia {tipo}: columna TOTAL renombrada desde '{total_col}'")
    for col in p_cols:
        df[col] = coerce_01(df[col], col_name=f"{tipo}/{col}")
    if "TOTAL" in df.columns and p_cols:
        total_calc = df[p_cols].sum(axis=1)
        total_orig = ensure_numeric(df["TOTAL"], "TOTAL")
        discrepantes = ((total_calc - total_orig).abs() > 0.5) & total_orig.notna()
        if discrepantes.sum() > 0:
            log_anomaly(f"Historia {tipo}: {discrepantes.sum()} filas con TOTAL")
            df.loc[discrepantes, "TOTAL"] = total_calc[discrepantes]
    elif p_cols:
        df["TOTAL"] = df[p_cols].sum(axis=1)
    df["Clase"] = df["Clase"].astype(str).strip()
    df["Colegio"] = df["Colegio"].astype(str).strip() if "Colegio" in df.columns else None
    df["Alumno"] = df["Alumno"].astype(str).strip()
    df["ID_hist"] = df["Colegio"] + "-" + df["Clase"] + "-" + df["Alumno"]
    df["Tipo"] = tipo
    logger.info(f" Historia {tipo}: {df.shape[0]} filas, {len(p_cols)} ítems")
    return df, p_cols
```

```

df_ini, p_cols_ini = prep_historia(df_hist_ini, "Inicial")
df_fin, p_cols_fin = prep_historia(df_hist_fin, "Final")
n_items_ini = len(p_cols_ini)
n_items_fin = len(p_cols_fin)
print(f"Historia Inicial: {df_ini.shape} | ítems: {p_cols_ini}")
print(f"Historia Final: {df_fin.shape} | ítems: {p_cols_fin}")

df_ini["pct_hist"] = df_ini["TOTAL"] / n_items_ini
df_fin["pct_hist"] = df_fin["TOTAL"] / n_items_fin

# -----
# 7.2 Muestras INDEPENDIENTES – asignar nivel_pc por clase
# -----
# Δ DECISIÓN METODOLÓGICA:
# Los índices de alumno (columna "Alumno") en el test inicial y en el test
# final son indicadores internos de recogida de información, NO permiten
# identificar al mismo estudiante en ambos tests.
# → Se tratan como DOS MUESTRAS INDEPENDIENTES del mismo grupo-clase.
#
# Consecuencias:
# • No se calculan ganancias individuales (pct_fin_i - pct_ini_i).
# • Las comparaciones se realizan a nivel de GRUPO/CLASE (diferencias de media)
# • La ganancia de Hake se calcula a nivel de clase:
#    $g = (mean\_fin\_clase - mean\_ini\_clase) / (1 - mean\_ini\_clase)$ 
# • Los tests estadísticos empleados son de dos muestras independientes
#   (Welch-t o Mann-Whitney U).
# -----

df_nivel_map = df_nivel_clase[["Clase", "nivel_pc", "Centro"]].copy()
df_nivel_map["Clase_hist"] = "5" + df_nivel_map["Clase"].str.upper()
df_ini["Clase_hist"] = df_ini["Clase"].astype(str).str.strip()
df_fin["Clase_hist"] = df_fin["Clase"].astype(str).str.strip()

def map_nivel_pc(df, nivel_map, clase_col="Clase_hist", colegio_col="Colegio"):
    """Une nivel_pc por clase. Si otro centro, deja NaN."""
    df = df.copy()
    df["nivel_pc"] = None
    mask_echegaray = df[colegio_col].str.contains("Echegaray", case=False, na=False)
    clase_to_nivel = dict(zip(nivel_map["Clase_hist"], nivel_map["nivel_pc"]))
    mapped = df.loc[mask_echegaray, clase_col].map(clase_to_nivel)
    df.loc[mask_echegaray, "nivel_pc"] = mapped.values
    n_asig = df["nivel_pc"].notna().sum()
    n_nan = df["nivel_pc"].isna().sum()
    logger.info(f" nivel_pc asignado: {n_asig} | sin asignar: {n_nan}")
    return df

df_ini = map_nivel_pc(df_ini, df_nivel_map)
df_fin = map_nivel_pc(df_fin, df_nivel_map)

print(f"\nHistoria Inicial: N={len(df_ini)} | media={df_ini['pct_hist'].mean():.3f} | SD={df_ini['pct_hist'].std():.3f}")
print(f"Historia Final: N={len(df_fin)} | media={df_fin['pct_hist'].mean():.3f} | SD={df_fin['pct_hist'].std():.3f}")
print("\nΔ Diseño: MUESTRAS INDEPENDIENTES – sin emparejamiento individual por alumno")

print(f"\nnivel_pc – Inicial:")
display(style_table(df_ini["nivel_pc"].value_counts(dropna=False).to_frame().reset_index()))

```

```

print(f"\nnivel_pc - Final:")
display(style_table(df_fin["nivel_pc"].value_counts(dropna=False).to_frame()).res

# -----
# 7.3 Categorías de preguntas (category_questions.xlsx)
# -----
df_cat_clean = df_cat.copy()
df_cat_clean.columns = [c.strip() for c in df_cat_clean.columns]
cat_ini_map = (
    df_cat_clean[df_cat_clean["Test"] == "Inicial"]
    .set_index("Pregunta")["Tematica"].to_dict()
)
cat_fin_map = (
    df_cat_clean[df_cat_clean["Test"] == "Final"]
    .set_index("Pregunta")["Tematica"].to_dict()
)
print("\nMapa categorías Inicial:", cat_ini_map)
print("Mapa categorías Final: ", cat_fin_map)

# -----
# 7.4 Formato largo (nivel_pc ya asignado - no requiere nuevo join)
# -----
def to_long(df, p_cols, cat_map, momento):
    """Convierte a formato largo. nivel_pc ya incluido en df."""
    rows = []
    for _, row in df.iterrows():
        for p in p_cols:
            rows.append({
                "ID_hist": row.get("ID_hist"),
                "Centro": row.get("Colegio"),
                "Clase": row.get("Clase"),
                "Alumno": row.get("Alumno"),
                "nivel_pc": row.get("nivel_pc"),
                "Momento": momento,
                "Pregunta": p.upper(),
                "Acierto": row.get(p),
                "Categoria": cat_map.get(p.upper(), "Desconocida"),
            })
    return pd.DataFrame(rows)

df_long_ini = to_long(df_ini, p_cols_ini, cat_ini_map, "Inicial")
df_long_fin = to_long(df_fin, p_cols_fin, cat_fin_map, "Final")
df_long = pd.concat([df_long_ini, df_long_fin], ignore_index=True)
df_long["Acierto"] = pd.to_numeric(df_long["Acierto"], errors="coerce")
print(f"\nDataset largo historia: {df_long.shape}")

# -----
# 7.5 Métricas por categoría x nivel_pc (heatmap)
# -----
df_cat_stats = (
    df_long.groupby(["Momento", "Categoria", "nivel_pc"])["Acierto"]
    .agg(["mean", "count", "sum"])
    .reset_index()
    .rename(columns={"mean": "pct_acierto", "count": "N", "sum": "n_aciertos"})
)
df_cat_stats.to_csv(TABS / "historia_categoria_stats.csv", index=False, encoding="utf-8")
print(f"\nTabla categoríaxnivel_pc → {TABS / 'historia_categoria_stats.csv'}")

for momento in ["Inicial", "Final"]:

```

```

sub = df_cat_stats[df_cat_stats["Momento"] == momento].dropna(subset=["nivel"])
if sub.empty:
    continue
pivot = sub.pivot_table(index="Categoria", columns="nivel_pc",
                          values="pct_acierto", aggfunc="mean")
fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(pivot, annot=True, fmt=".0%", cmap="RdYlGn", vmin=0, vmax=1,
            linewidths=0.4, ax=ax, cbar_kws={"label": "% acierto"})
ax.set_title(f"% Acierto por Categoría x Nivel PC – {momento}", fontweight="bold")
ax.set_xlabel("Nivel PC")
fig.tight_layout()
fig.savefig(FIGS / f"historia_heatmap_cat_nivel_{momento.lower()}.png",
            dpi=120, bbox_inches="tight")

plt.show()
print(f"Figura guardada: historia_heatmap_cat_nivel_{momento.lower()}.png")

# -----
# 7.6 Descriptivos por Momento x grupo (muestras independientes)
# -----
hist_resumen_rows = []
for momento, df_m in [("Inicial", df_ini), ("Final", df_fin)]:
    for gcol in [None, "Clase", "nivel_pc"]:
        if gcol is None:
            subsets = [("Global", df_m)]
        else:
            subsets = [(g, df_m[df_m[gcol] == g])
                       for g in sorted(df_m[gcol].dropna().unique())]
        for label, sub in subsets:
            x = sub["pct_hist"].dropna().values
            ci_lo, ci_hi = bootstrap_ci(x, np.mean, n_boot=2000)
            hist_resumen_rows.append({
                "Momento": momento,
                "Agrupación": gcol or "Global",
                "Grupo": label,
                "N": len(x),
                "Media": round(np.mean(x), 3) if len(x) else np.nan,
                "SD": round(np.std(x, ddof=1), 3) if len(x) > 1 else np.nan,
                "IC95_lo": round(ci_lo, 3),
                "IC95_hi": round(ci_hi, 3),
            })

df_hist_resumen = pd.DataFrame(hist_resumen_rows)
df_hist_resumen.to_csv(TABS / "historia_resumen_por_grupo.csv", index=False, encoding="utf-8")
print(f"\nResumen historia → {TABS / 'historia_resumen_por_grupo.csv'}")
display(style_table(df_hist_resumen[df_hist_resumen["Agrupación"] == "Global"],
                    ["Momento", "Grupo", "N", "Media", "SD", "IC95_lo", "IC95_hi"])))

# -----
# 7.7 Figuras básicas – distribución y comparación por grupo
# -----
niv_order = ["Bajo", "Medio", "Alto"]

# Histogramas superpuestos (dos distribuciones independientes)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, (df_m, label, color) in zip(axes, [
    (df_ini, "Inicial", "steelblue"),
    (df_fin, "Final", "coral"),
]):
    vals = df_m["pct_hist"].dropna()
    sns.histplot(vals, kde=True, ax=ax, bins=12, color=color)
    ax.axvline(vals.mean(), color="crimson", ls="--", lw=1.5,
               label=f"Media: {vals.mean():.2f}")

```

```

        label=f"Media={vals.mean():.2f}")
    ax.set_xlabel("% aciertos")
    ax.set_title(f"Historia {label} (N={len(vals)})")
    ax.legend()
fig.suptitle("Distribución Test Historia – Inicial vs Final (muestras independiente
            fontweight="bold")
fig.tight_layout()
fig.savefig(FIGS / "historia_distribucion.png", dpi=120, bbox_inches="tight")
plt.show()

# Boxplot ini vs fin por nivel_pc (muestras independientes)
df_long_compare = pd.concat([
    df_ini[["Clase", "nivel_pc", "pct_hist"]].assign(Momento="Inicial"),
    df_fin[["Clase", "nivel_pc", "pct_hist"]].assign(Momento="Final"),
], ignore_index=True)
df_long_compare_niv = df_long_compare.dropna(subset=["nivel_pc"])

fig2, ax2 = plt.subplots(figsize=(9, 5))
sns.boxplot(
    data=df_long_compare_niv, x="nivel_pc", y="pct_hist", hue="Momento",
    order=niv_order, palette={"Inicial": "steelblue", "Final": "coral"}, ax=ax2,
)
ax2.set_xlabel("Nivel PC (asignado por clase)")
ax2.set_ylabel("Puntuación normalizada (0-1)")
ax2.set_title("Historia Inicial vs Final por Nivel PC\n(muestras independientes)
            fontweight="bold")
ax2.legend(title="Momento")
ax2.set_ylim(-0.05, 1.05)
fig2.tight_layout()
fig2.savefig(FIGS / "historia_ganancia_por_nivel_pc.png", dpi=120, bbox_inches="
plt.show()

# Barplot medias por clase (muestras independientes)
clases_hist = sorted(set(df_ini["Clase"].unique()) | set(df_fin["Clase"].unique(
x_pos = np.arange(len(clases_hist))
w = 0.35
fig3, ax3 = plt.subplots(figsize=(10, 5))
for i, (df_m, label, color) in enumerate([
    (df_ini, "Inicial", "#4E79A7"),
    (df_fin, "Final", "#F28E2B"),
]):
    means_c = [df_m.loc[df_m["Clase"] == c, "pct_hist"].mean() for c in clases_h
    ax3.bar(x_pos + (i - 0.5) * w, means_c, w, label=label, color=color, alpha=0
ax3.set_xticks(x_pos)
ax3.set_xticklabels(clases_hist)
ax3.set_ylabel("% aciertos medio")
ax3.set_ylim(0, 1)
ax3.set_title("Rendimiento Historia por Clase – Inicial vs Final\n(muestras inde
            fontweight="bold")
ax3.legend()
fig3.tight_layout()
fig3.savefig(FIGS / "historia_rendimiento_por_clase.png", dpi=120, bbox_inches="
plt.show()
print("Figuras 7.7 guardadas OK")

# -----
# 7.8 Inferencia – muestras independientes
# -----
logger.info(" Inferencia historia (muestras independientes)...")
contraste_hist_rows = []

```

```

ini_all = df_ini["pct_hist"].dropna().values
fin_all = df_fin["pct_hist"].dropna().values

def _bootstrap_2samp_ci(x, y, stat_func, n_boot=2000):
    """Bootstrap CI para estadísticos de dos muestras independientes."""
    rng = np.random.default_rng(42)
    boots = [
        stat_func(
            rng.choice(x, len(x), replace=True),
            rng.choice(y, len(y), replace=True),
        )
        for _ in range(n_boot)
    ]
    boots = [b for b in boots if b is not None and not np.isnan(b)]
    return (np.percentile(boots, 2.5), np.percentile(boots, 97.5)) if boots else

# 7.8a Contraste global ini vs fin
sw_ini_p = stats.shapiro(ini_all).pvalue if len(ini_all) >= 3 else 0.0
sw_fin_p = stats.shapiro(fin_all).pvalue if len(fin_all) >= 3 else 0.0
use_t = sw_ini_p > 0.05 and sw_fin_p > 0.05
if use_t:
    stat_g, p_g = stats.ttest_ind(fin_all, ini_all, equal_var=False)
    test_name_g = "Welch-t"
else:
    stat_g, p_g = stats.mannwhitneyu(fin_all, ini_all, alternative="two-sided")
    test_name_g = "Mann-Whitney U"

diff_means_g = fin_all.mean() - ini_all.mean()
ci_lo_g, ci_hi_g = _bootstrap_2samp_ci(
    ini_all, fin_all, lambda x, y: y.mean() - x.mean()
)
contraste_hist_rows.append({
    "Contraste": "Global ini vs fin (indep.)",
    "Test": test_name_g,
    "N_ini": len(ini_all), "N_fin": len(fin_all),
    "media_ini": round(ini_all.mean(), 3), "media_fin": round(fin_all.mean(), 3)
    "diff_medias": round(diff_means_g, 3),
    "IC95_lo": round(ci_lo_g, 3), "IC95_hi": round(ci_hi_g, 3),
    "Estadístico": round(float(stat_g), 4), "p_valor": round(p_g, 4),
})
print(f"\nTest global ini vs fin ({test_name_g}): stat={stat_g:.4f} p={p_g:.4f}")
print(f" Δ medias (fin-ini): {diff_means_g:+.3f} IC95=[{ci_lo_g:.3f}, {ci_hi_g:.3f}]")

# 7.8b Contraste pct_hist (ini y fin por separado) × nivel_pc
for momento, df_m in [("Final", df_fin), ("Inicial", df_ini)]:
    niveles = [n for n in niv_order if n in df_m["nivel_pc"].values]
    groups = [df_m.loc[df_m["nivel_pc"] == n, "pct_hist"].dropna().values for n
    valid = [(n, g) for n, g in zip(niveles, groups) if len(g) >= 2]
    if len(valid) < 2:
        continue
    data_v = [g for _, g in valid]
    all_norm = all((stats.shapiro(g).pvalue > 0.05 if len(g) >= 3 else False) for
    if all_norm:
        stat_om, p_om = stats.f_oneway(*data_v)
        method_om = "ANOVA"
        grand_m = np.concatenate(data_v).mean()
        ss_b = sum(len(g) * (g.mean() - grand_m) ** 2 for g in data_v)

```

```

ss_t = sum(((x - grand_m) ** 2).sum() for g in data_v for x in g)
ef_om = round(ss_b / ss_t, 3) if ss_t > 0 else np.nan
tipo_ef = "eta^2"
else:
    stat_om, p_om = stats.kruskal(*data_v)
    method_om = "Kruskal-Wallis"
    n_tot = sum(len(g) for g in data_v)
    ef_om = round((stat_om - len(data_v) + 1) / (n_tot - len(data_v)), 3)
    tipo_ef = "epsilon^2"
contraste_hist_rows.append({
    "Contraste": f"{momento} por Nivel PC (omnibus)",
    "Test": method_om,
    "N_ini": len(ini_all), "N_fin": len(fin_all),
    "Estadístico": round(float(stat_om), 3), "p_valor": round(p_om, 4),
    "efecto": ef_om, "tipo_efecto": tipo_ef,
})
print(f"\n{method_om} pct_hist({momento}) x Nivel PC: stat={stat_om:.4f}"
      f" p={p_om:.4f} ef={ef_om:.3f}")
if p_om < 0.05 and len(valid) >= 3:
    for (n1, g1), (n2, g2) in itertools.combinations(valid, 2):
        _, p_ph = stats.mannwhitneyu(g1, g2, alternative="two-sided")
        p_corr = min(p_ph * len(list(itertools.combinations(valid, 2))), 1)
        print(f" Post-hoc {n1} vs {n2}: MWU p_raw={p_ph:.4f} p_bonf={p_corr:.4f}")

# 7.8c OLS: pct_hist(Final) ~ nivel_pc + Colegio
df_fin_model = df_fin.dropna(subset=["pct_hist", "nivel_pc"]).copy()
if len(df_fin_model) >= 10 and df_fin_model["nivel_pc"].nunique() >= 2:
    try:
        df_fin_model["nivel_pc_num"] = df_fin_model["nivel_pc"].map({"Bajo": 0,
                                                                        "Medio": 1,
                                                                        "Alto": 2})
        mod_fin = smf.ols("pct_hist ~ nivel_pc_num + C(Colegio)", data=df_fin_model)
        print("\n=== Modelo Lineal: pct_hist(Final) ~ nivel_pc + Colegio ===")
        print(mod_fin.summary2().tables[1].round(4))
        logger.info(f"Modelo lineal historia Final: R²={mod_fin.rsquared:.3f}")
        contraste_hist_rows.append({
            "Contraste": "OLS pct_hist(Final)~nivel_pc+Colegio",
            "Test": "OLS", "N_fin": len(df_fin_model),
            "R2": round(mod_fin.rsquared, 3), "p_valor": round(mod_fin.f_pvalue, 4)
        })
    except Exception as e:
        logger.warning(f"Modelo lineal historia (Final) falló: {e}")
else:
    logger.warning("Insuficientes datos para modelo lineal historia Final")

df_hist_contrastes = pd.DataFrame(contraste_hist_rows)
df_hist_contrastes.to_csv(TABS / "historia_contrastes.csv", index=False, encoding="utf-8")
print(f"\nContrastes historia → {TABS / 'historia_contrastes.csv'}")

# -----
# 7.9 Ganancia de Hake GRUPAL por clase
# g = (mean_pct_fin - mean_pct_ini) / (1 - mean_pct_ini)
# Válido para muestras independientes: opera sobre medias de grupo.
# -----
df_ini_cls = (
    df_ini.groupby("Clase")["pct_hist"]
    .agg(n_ini="count", mean_pct_ini="mean", std_ini="std")
    .reset_index()
)
df_fin_cls = (
    df_fin.groupby("Clase")["pct_hist"]
    .agg(n_fin="count", mean_pct_fin="mean", std_fin="std")
)

```

```

        .reset_index()
    )
    df_hake_clase = df_ini_cls.merge(df_fin_cls, on="Clase", how="outer")
    df_hake_clase["ganancia_abs"] = df_hake_clase["mean_pct_fin"] - df_hake_clase["mean_pct_ini"]
    ceiling_h = 1 - df_hake_clase["mean_pct_ini"]
    df_hake_clase["hake_g"] = np.where(
        ceiling_h > 0, df_hake_clase["ganancia_abs"] / ceiling_h, np.nan
    )
    bins_hake = [-np.inf, 0.30, 0.70, np.inf]
    labels_hake = ["Baja (<0.30)", "Media (0.30-0.70)", "Alta (>0.70)"]
    df_hake_clase["categoria_hake"] = pd.cut(df_hake_clase["hake_g"], bins=bins_hake)
    df_hake_clase.to_csv(TABS / "historia_hake_por_clase.csv", index=False, encoding="utf-8")

    global_ini_h = df_ini["pct_hist"].mean()
    global_fin_h = df_fin["pct_hist"].mean()
    global_hake_h = (global_fin_h - global_ini_h) / (1 - global_ini_h) \
        if (1 - global_ini_h) > 0 else np.nan

    print("\n== Ganancia de Hake grupal por clase (Historia - muestras indep.) ==")
    display(style_table(df_hake_clase, caption="Ganancia de Hake grupal por clase - Historia"))
    print(f"\n Global: N_ini={len(df_ini)} N_fin={len(df_fin)} | "
          f" pct_ini={global_ini_h:.3f} | pct_fin={global_fin_h:.3f} | Hake g={global_hake_h:.3f}")

    # Barplot Hake grupal
    fig_hk, ax_hk = plt.subplots(figsize=(8, 4))
    colors_hk = ["steelblue" if g >= 0 else "salmon" for g in df_hake_clase["hake_g"]]
    bars_hk = ax_hk.bar(df_hake_clase["Clase"], df_hake_clase["hake_g"],
                       color=colors_hk, edgecolor="white")
    ax_hk.axhline(0, color="black", lw=0.8)
    ax_hk.axhline(0.30, color="grey", lw=0.8, ls="--", label="umbral bajo (0.30)")
    ax_hk.axhline(0.70, color="grey", lw=0.8, ls="-. ", label="umbral alto (0.70)")
    for bar, val in zip(bars_hk, df_hake_clase["hake_g"]):
        if not np.isnan(val):
            ax_hk.text(
                bar.get_x() + bar.get_width() / 2,
                val + 0.01 * np.sign(val + 0.001),
                f"{val:+.2f}", ha="center",
                va="bottom" if val >= 0 else "top", fontsize=10,
            )
    ax_hk.set_xlabel("Clase")
    ax_hk.set_ylabel("Ganancia de Hake g")
    ax_hk.set_title("Ganancia de Hake grupal por clase - Historia\n(muestras independientes)")
    ax_hk.legend(fontsize=8)
    plt.tight_layout()
    fig_hk.savefig(FIGS / "historia_hake_por_clase.png", dpi=120, bbox_inches="tight")
    plt.show()
    print("Figura guardada: historia_hake_por_clase.png")

    # -----
    # 7.10 Boxplots ini vs fin por clase + media ± SD + histograma global
    # -----
    df_long_hist = pd.concat([
        df_ini[["Clase", "pct_hist"]].rename(columns={"pct_hist": "pct"}).assign(Mom="ini"),
        df_fin[["Clase", "pct_hist"]].rename(columns={"pct_hist": "pct"}).assign(Mom="fin")
    ], ignore_index=True)
    order_clases_h = sorted(df_long_hist["Clase"].dropna().unique())

    fig_bx, ax_bx = plt.subplots(figsize=(12, 5))
    sns.boxplot(

```

```

    data=df_long_hist, x="Clase", y="pct", hue="Momento", order=order_clases_h,
    palette={"Inicial": "steelblue", "Final": "coral"}, ax=ax_bx,
)
sns.stripplot(
    data=df_long_hist, x="Clase", y="pct", hue="Momento", order=order_clases_h,
    palette={"Inicial": "steelblue", "Final": "coral"},
    dodge=True, size=3, alpha=0.4, ax=ax_bx, legend=False,
)
ax_bx.set_title("Distribución Historia por clase – Inicial vs Final\n(muestras i
                fontweight="bold")
ax_bx.set_xlabel("Clase")
ax_bx.set_ylabel("Puntuación normalizada (0-1)")
ax_bx.legend(title="Momento")
ax_bx.set_ylim(-0.05, 1.05)
plt.tight_layout()
fig_bx.savefig(FIGS / "historia_boxplot_ini_fin_clase.png", dpi=120, bbox_inches
plt.show()
print("Figura guardada: historia_boxplot_ini_fin_clase.png")

# Barras media ± SD por clase
fig_mn, ax_mn = plt.subplots(figsize=(9, 5))
x_cls = np.arange(len(df_hake_clase))
w_cls = 0.35
ax_mn.bar(x_cls - w_cls / 2, df_hake_clase["mean_pct_ini"], w_cls,
          yerr=df_hake_clase["std_ini"], capsize=4,
          label="Media Inicial", color="steelblue", alpha=0.85)
ax_mn.bar(x_cls + w_cls / 2, df_hake_clase["mean_pct_fin"], w_cls,
          yerr=df_hake_clase["std_fin"], capsize=4,
          label="Media Final", color="coral", alpha=0.85)
ax_mn.set_xticks(x_cls)
ax_mn.set_xticklabels(df_hake_clase["Clase"])
ax_mn.set_ylabel("Puntuación normalizada media (± SD)")
ax_mn.set_title("Media Historia Inicial vs Final por clase (± SD)\n(muestras ind
                fontweight="bold")
ax_mn.legend()
ax_mn.set_ylim(0, 1)
plt.tight_layout()
fig_mn.savefig(FIGS / "historia_medias_ini_fin_clase.png", dpi=120, bbox_inches=
plt.show()
print("Figura guardada: historia_medias_ini_fin_clase.png")

# Histograma global superpuesto
fig_hg, ax_hg = plt.subplots(figsize=(9, 4))
ax_hg.hist(df_ini["pct_hist"].dropna(), bins=12, alpha=0.6,
          label=f"Inicial (N={len(df_ini)})", color="steelblue", edgecolor="whi
ax_hg.hist(df_fin["pct_hist"].dropna(), bins=12, alpha=0.6,
          label=f"Final (N={len(df_fin)})", color="coral", edgecolor="white")
ax_hg.set_xlabel("Puntuación normalizada (0-1)")
ax_hg.set_ylabel("Frecuencia")
ax_hg.set_title("Histograma global Historia – Inicial vs Final\n(muestras indepe
                fontweight="bold")
ax_hg.legend()
plt.tight_layout()
fig_hg.savefig(FIGS / "historia_histograma_global.png", dpi=120, bbox_inches="ti
plt.show()
print("Figura guardada: historia_histograma_global.png")

# -----
# 7.11 Cohen's d (muestras independientes) y forest plot
# -----

```

```

def _cohens_d_indep(x, y):
    """Cohen's d para muestras independientes (pooled SD)."""
    nx, ny = len(x), len(y)
    if nx < 2 or ny < 2:
        return np.nan
    pooled_var = ((nx - 1) * np.var(x, ddof=1) + (ny - 1) * np.var(y, ddof=1)) /
    pooled_sd = np.sqrt(pooled_var)
    return (np.mean(y) - np.mean(x)) / pooled_sd if pooled_sd > 0 else np.nan

effect_rows = []
# Global
d_g_eff = _cohens_d_indep(ini_all, fin_all)
ci_lo_e, ci_hi_e = _bootstrap_2samp_ci(ini_all, fin_all, _cohens_d_indep)
effect_rows.append({
    "grupo": "GLOBAL",
    "N_ini": len(ini_all), "N_fin": len(fin_all),
    "media_ini": round(ini_all.mean(), 3), "media_fin": round(fin_all.mean(), 3)
    "Cohen_d": round(d_g_eff, 3),
    "IC95_lo": round(ci_lo_e, 3), "IC95_hi": round(ci_hi_e, 3),
})
# Por clase
for clase in sorted(set(df_ini["Clase"].unique()) & set(df_fin["Clase"].unique())):
    x_c = df_ini.loc[df_ini["Clase"] == clase, "pct_hist"].dropna().values
    y_c = df_fin.loc[df_fin["Clase"] == clase, "pct_hist"].dropna().values
    d_c = _cohens_d_indep(x_c, y_c)
    lo_c, hi_c = _bootstrap_2samp_ci(x_c, y_c, _cohens_d_indep)
    effect_rows.append({
        "grupo": f"Clase {clase}",
        "N_ini": len(x_c), "N_fin": len(y_c),
        "media_ini": round(x_c.mean(), 3), "media_fin": round(y_c.mean(), 3),
        "Cohen_d": round(d_c, 3),
        "IC95_lo": round(lo_c, 3), "IC95_hi": round(hi_c, 3),
    })

df_effects = pd.DataFrame(effect_rows)
df_effects.to_csv(TABS / "historia_efectos.csv", index=False, encoding="utf-8-si")
print("\n=== Cohen's d Historia (muestras independientes: Final vs Inicial) ===")
display(style_table(df_effects, caption="Tamaños de efecto – Historia (Cohen's d)"))

# Forest plot
fig_ef, ax_ef = plt.subplots(figsize=(10, max(4, len(df_effects) * 0.7)))
y_pos_e = range(len(df_effects))
colors_e = ["steelblue" if d >= 0 else "salmon" for d in df_effects["Cohen_d"]]
ax_ef.barh(list(y_pos_e), df_effects["Cohen_d"],
            color=colors_e, edgecolor="white", height=0.6)
lo_err = df_effects["Cohen_d"] - df_effects["IC95_lo"]
hi_err = df_effects["IC95_hi"] - df_effects["Cohen_d"]
ax_ef.errorbar(df_effects["Cohen_d"], list(y_pos_e),
               xerr=[lo_err, hi_err], fmt="none", color="black", capsize=4, lw=1)
ax_ef.axvline(0, color="black", lw=0.8)
for thresh, ls, lbl in [
    (0.2, "--", "pequeño (0.2)"),
    (0.5, "-.", "mediano (0.5)"),
    (0.8, ":", "grande (0.8)"),
]:
    ax_ef.axvline(thresh, color="grey", lw=0.6, ls=ls, label=lbl)
    ax_ef.axvline(-thresh, color="grey", lw=0.6, ls=ls)
ax_ef.set_yticks(list(y_pos_e))
ax_ef.set_yticklabels(df_effects["grupo"], fontsize=9)

```

```

ax_ef.set_xlabel("Cohen's d (muestras indep.; positivo = Final > Inicial)")
ax_ef.set_title("Tamaños de efecto Historia ini→fin (IC95% bootstrap)", fontweig
ax_ef.legend(loc="lower right", fontsize=8)
plt.tight_layout()
fig_ef.savefig(FIGS / "historia_effect_sizes.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: historia_effect_sizes.png")

# -----
# 7.12 Comparaciones entre clases (ANOVA / Kruskal-Wallis)
# -----
def _run_omnibus_h(df, col, group_col, label):
    groups_d = [grp[col].dropna().values for _, grp in df.groupby(group_col)]
    all_norm = all(stats.shapiro(g).pvalue > 0.05 for g in groups_d if len(g) >
    method = "ANOVA" if all_norm else "Kruskal-Wallis"
    stat, p = stats.f_oneway(*groups_d) if all_norm else stats.kruskal(*groups
    return {"Contraste": label, "Método": method,
            "Estadístico": round(float(stat), 3), "p": round(p, 4), "Sig": p < 0

row_ini_omni = _run_omnibus_h(df_ini, "pct_hist", "Clase", "Inicial – entre clas
row_fin_omni = _run_omnibus_h(df_fin, "pct_hist", "Clase", "Final – entre clases

df_omnibus_h = pd.DataFrame([row_ini_omni, row_fin_omni])
df_omnibus_h.to_csv(TABS / "historia_omnibus_clases.csv", index=False, encoding=
print("\n=== Comparaciones entre clases (Historia) ===")
display(style_table(df_omnibus_h, caption="Comparaciones entre clases – Historia

# Post-hoc si ómnibus significativo
for row_omni, df_m, lbl_ph in [
    (row_ini_omni, df_ini, "Inicial"),
    (row_fin_omni, df_fin, "Final"),
]:
    if row_omni["Sig"]:
        print(f"\n=== Post-hoc Mann-Whitney + Bonferroni: {lbl_ph} ===")
        clases_ph = sorted(df_m["Clase"].dropna().unique())
        ph_rows = []
        for i in range(len(clases_ph)):
            for j in range(i + 1, len(clases_ph)):
                g1_ = df_m.loc[df_m["Clase"] == clases_ph[i], "pct_hist"].dropna
                g2_ = df_m.loc[df_m["Clase"] == clases_ph[j], "pct_hist"].dropna
                _, p_raw = stats.mannwhitneyu(g1_, g2_, alternative="two-sided")
                ph_rows.append({"grupo1": clases_ph[i], "grupo2": clases_ph[j],
                                "p_raw": round(p_raw, 4)})
        df_ph_h = pd.DataFrame(ph_rows)
        df_ph_h["p_bonferroni"] = (df_ph_h["p_raw"] * len(df_ph_h)).clip(upper=1
        df_ph_h["sig"] = df_ph_h["p_bonferroni"] < 0.05
        display(style_table(df_ph_h, caption="Post-hoc entre clases (Bonferroni)
    else:
        print(f"\n→ {lbl_ph}: p ≥ 0.05 – sin diferencias significativas entre cl

# Boxplots comparativos Inicial / Final (entre clases)
order_cl_h = sorted(set(df_ini["Clase"].unique()) | set(df_fin["Clase"].unique()
fig_cl, axes_cl = plt.subplots(1, 2, figsize=(12, 5))
for ax, df_m, title, color in zip(
    axes_cl,
    [df_ini, df_fin],
    [f"Inicial (p={row_ini_omni['p']:.4f})", f"Final (p={row_fin_omni['p']:.4f})
    ["steelblue", "coral"]],
):

```

```

sns.boxplot(data=df_m, x="Clase", y="pct_hist", order=order_cl_h,
            color=color, ax=ax, width=0.5)
sns.swarmplot(data=df_m, x="Clase", y="pct_hist", order=order_cl_h,
              color="black", alpha=0.5, size=4, ax=ax)
ax.set_title(title)
ax.set_xlabel("Clase")
ax.set_ylabel("pct_hist")
ax.set_ylim(-0.05, 1.05)
fig_cl.suptitle("Comparación entre clases – Historia (Inicial / Final)",
               fontsize=13, fontweight="bold")
plt.tight_layout()
fig_cl.savefig(FIGS / "historia_comparacion_clases.png", dpi=120, bbox_inches="t
plt.show()
print("Figura guardada: historia_comparacion_clases.png")

# -----
# 7.13 Análisis psicométrico por ítem – Historia
# Dificultad p, discriminación pbis, índice D27, co-errores
# -----
def _item_stats_hist(df, item_cols, total_col, label, cat_map):
    """Dificultad y discriminación para ítems de Historia."""
    df_num = df[item_cols].apply(pd.to_numeric, errors="coerce")
    tot = pd.to_numeric(df[total_col], errors="coerce")
    diff_ = df_num.mean().rename("dificultad_p")
    pbis_d = {}
    for col in item_cols:
        valid = df_num[col].notna() & tot.notna()
        if valid.sum() > 2:
            r, _ = stats.pointbiserialr(df_num.loc[valid, col], tot[valid])
            pbis_d[col] = round(r, 3)
        else:
            pbis_d[col] = np.nan
    pbis_ = pd.Series(pbis_d, name="disc_pbis")
    n27 = max(1, int(len(df) * 0.27))
    upper_ = df.nlargest(n27, total_col)
    lower_ = df.nsmallest(n27, total_col)
    d27_ = (
        upper_[item_cols].apply(pd.to_numeric, errors="coerce").mean()
        - lower_[item_cols].apply(pd.to_numeric, errors="coerce").mean()
    ).rename("disc_D27").round(3)
    result = pd.concat([diff_, pbis_, d27_], axis=1)
    result.index.name = "Pregunta"
    result["test"] = label
    result["Categoria"] = result.index.map(cat_map).fillna("Desconocida")
    return result.reset_index()

df_items_ini_h = _item_stats_hist(df_ini, p_cols_ini, "TOTAL", "Inicial", cat_in
df_items_fin_h = _item_stats_hist(df_fin, p_cols_fin, "TOTAL", "Final", cat_fi
df_items_hist = pd.concat([df_items_ini_h, df_items_fin_h], ignore_index=True)
df_items_hist.to_csv(TABS / "historia_analisis_items.csv", index=False, encoding
print("\n=== Análisis psicométrico por ítem – Historia ===")
display(style_table(df_items_hist, caption="Análisis psicométrico por ítem – His

# Heatmap dificultad y discriminación (Inicial / Final)
fig_it, axes_it = plt.subplots(1, 2, figsize=(14, 5))
for ax, df_itm, title in zip(
    axes_it,
    [df_items_ini_h.set_index("Pregunta"), df_items_fin_h.set_index("Pregunta")],
    ["Test Historia Inicial", "Test Historia Final"],

```

```

):
    pivot_it = df_itm[["dificultad_p", "disc_pbis", "disc_D27"]]
    sns.heatmap(pivot_it.T, annot=True, fmt=".2f", cmap="RdYlGn",
                linewidths=0.5, ax=ax, vmin=-0.4, vmax=1.0)
    ax.set_title(title, fontweight="bold")
    ax.set_ylabel("Indicador")
    ax.set_xlabel("Ítem")
    fig_it.suptitle("Dificultad y Discriminación por Ítem – Historia",
                   fontsize=13, fontweight="bold")
    plt.tight_layout()
    fig_it.savefig(FIGS / "historia_heatmap_items.png", dpi=120, bbox_inches="tight")
    plt.show()
    print("Figura guardada: historia_heatmap_items.png")

# Matriz de co-errores
def _coerror_matrix_h(df, item_cols):
    errors = 1 - df[item_cols].apply(pd.to_numeric, errors="coerce").fillna(0)
    coerr = errors.T @ errors
    arr = coerr.to_numpy(dtype=float, copy=True)
    np.fill_diagonal(arr, 0)
    return pd.DataFrame(arr, index=coerr.index, columns=coerr.columns)

fig_ce, axes_ce = plt.subplots(1, 2, figsize=(14, 5))
for ax, df_h, item_cols, title in zip(
    axes_ce, [df_ini, df_fin], [p_cols_ini, p_cols_fin],
    ["Co-errores Inicial", "Co-errores Final"],
):
    cm = _coerror_matrix_h(df_h, item_cols)
    sns.heatmap(cm, annot=True, fmt=".0f", cmap="Blues",
                linewidths=0.3, ax=ax, square=True)
    ax.set_title(title, fontweight="bold")
    fig_ce.suptitle("Matriz de Co-errores – Historia (veces que dos ítems fallan jun
                    fontsize=12)
    plt.tight_layout()
    fig_ce.savefig(FIGS / "historia_coerror_matrix.png", dpi=120, bbox_inches="tight")
    plt.show()
    print("Figura guardada: historia_coerror_matrix.png")

```

```

2026-05-22 13:26:03,160 [INFO] --- SECCIÓN 7: Historia (muestras independientes)
---
2026-05-22 13:26:03,177 [INFO]   Historia Inicial: 93 filas, 8 ítems
2026-05-22 13:26:03,198 [INFO]   Historia Final: 82 filas, 11 ítems
Historia Inicial: (93, 15) | ítems: ['P1', 'P2', 'P3', 'P4', 'P5', 'P6', 'P7', 'P
8']
Historia Final:   (82, 18) | ítems: ['P1', 'P2', 'P3', 'P4', 'P5', 'P6', 'P7', 'P
8', 'P9', 'P10', 'P11']
2026-05-22 13:26:03,208 [INFO]   nivel_pc asignado: 93 | sin asignar: 0
2026-05-22 13:26:03,213 [INFO]   nivel_pc asignado: 82 | sin asignar: 0

Historia Inicial: N=93 | media=0.781 SD=0.177
Historia Final:   N=82 | media=0.796 SD=0.139

```

△ Diseño: MUESTRAS INDEPENDIENTES – sin emparejamiento individual por alumno

nivel_pc – Inicial:

Distribución nivel

PC — Historia Inicial

nivel_pc	count
Bajo	46
Medio	24
Alto	23

nivel_pc – Final:

Distribución nivel

PC — Historia Final

nivel_pc	count
Bajo	43
Alto	21
Medio	18

Mapa categorías Inicial: {'P1': 'Algoritmia', 'P2': 'Prehistoria', 'P3': 'Edad Moderna', 'P4': 'Edad Antigua', 'P5': 'Edad Contemporanea', 'P6': 'Edad Media', 'P7': 'Programacion/Robotica', 'P8': 'Programacion/Robotica'}

Mapa categorías Final: {'P1': 'Algoritmia', 'P2': 'Prehistoria', 'P3': 'Prehistoria', 'P4': 'Edad Antigua', 'P5': 'Edad Antigua', 'P6': 'Edad Media', 'P7': 'Edad Media', 'P8': 'Edad Moderna', 'P9': 'Edad Moderna', 'P10': 'Edad Contemporanea', 'P11': 'Edad Contemporanea', 'P12': 'Respuesta libre', 'P13': 'Respuesta libre'}

Dataset largo historia: (1646, 9)

Tabla categoríaxnivel_pc → outputs\tables\historia_categoria_stats.csv

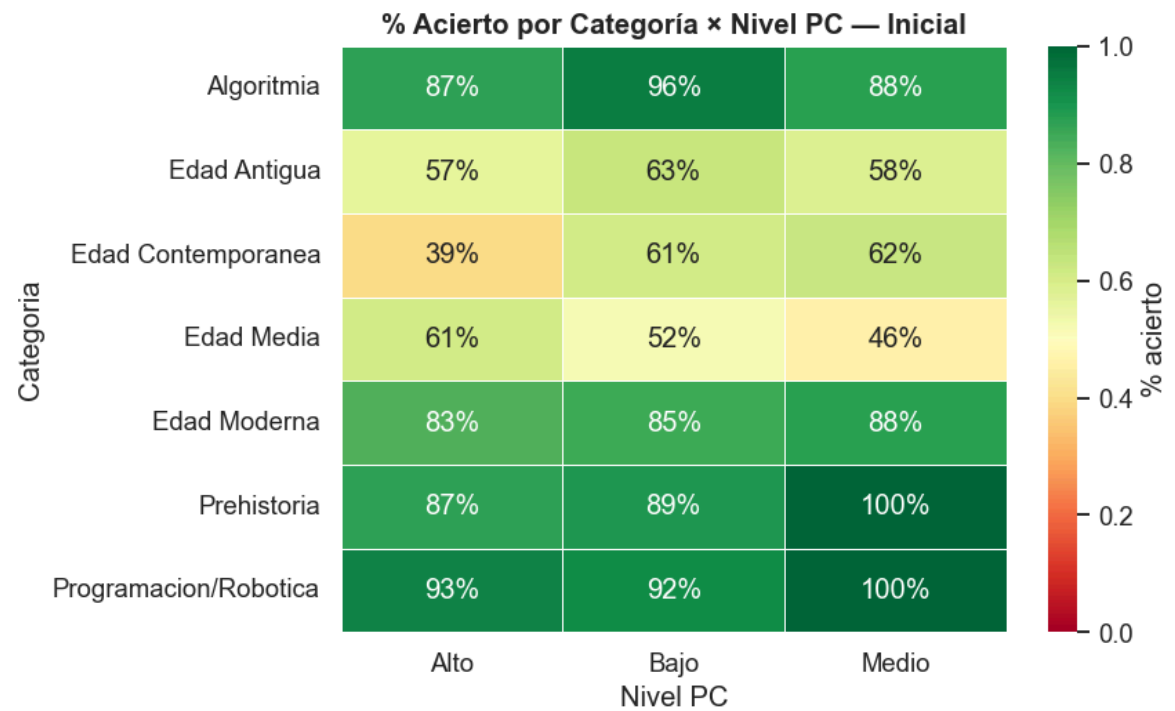


Figura guardada: historia_heatmap_cat_nivel_inicial.png

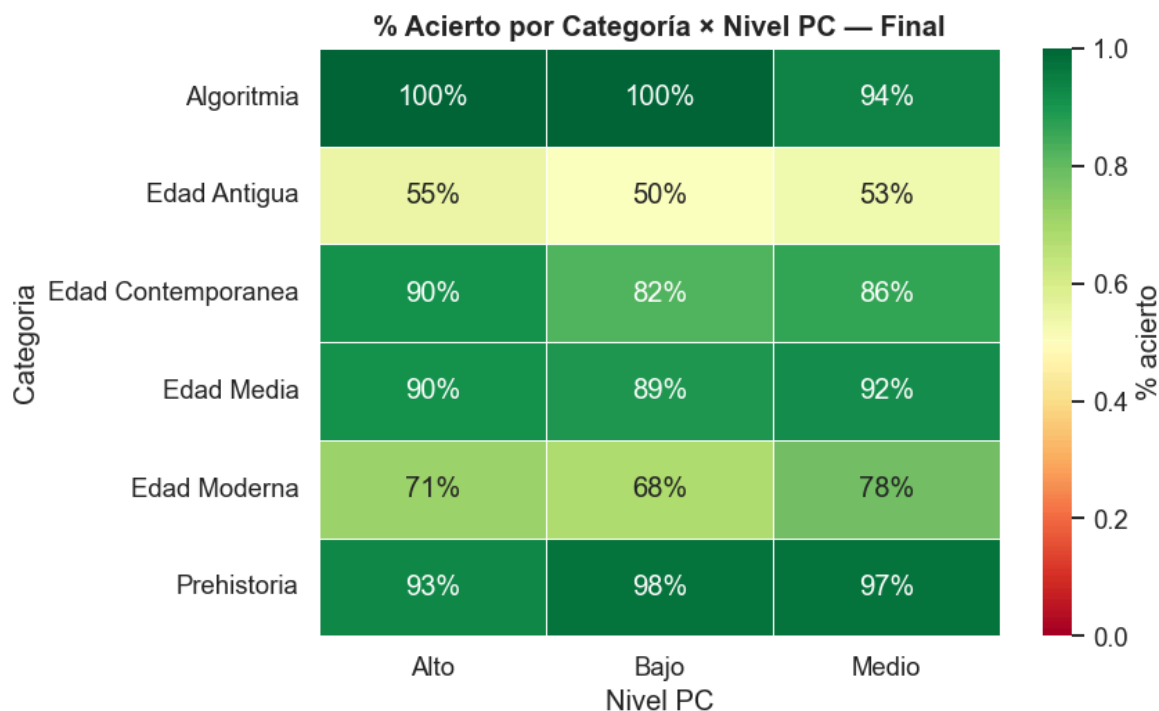


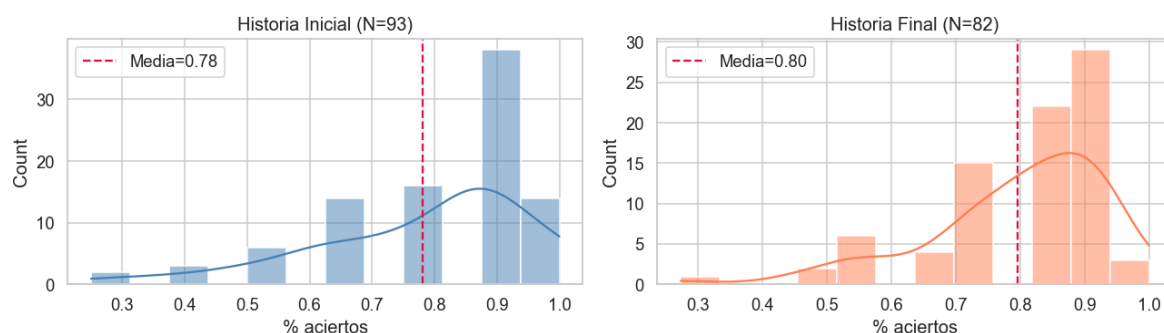
Figura guardada: historia_heatmap_cat_nivel_final.png

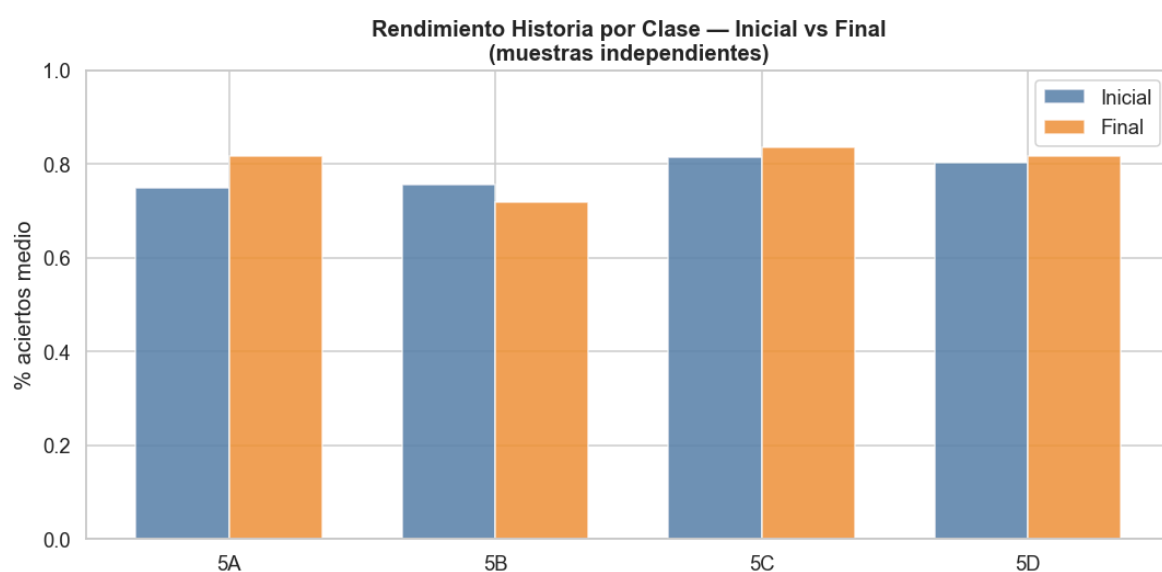
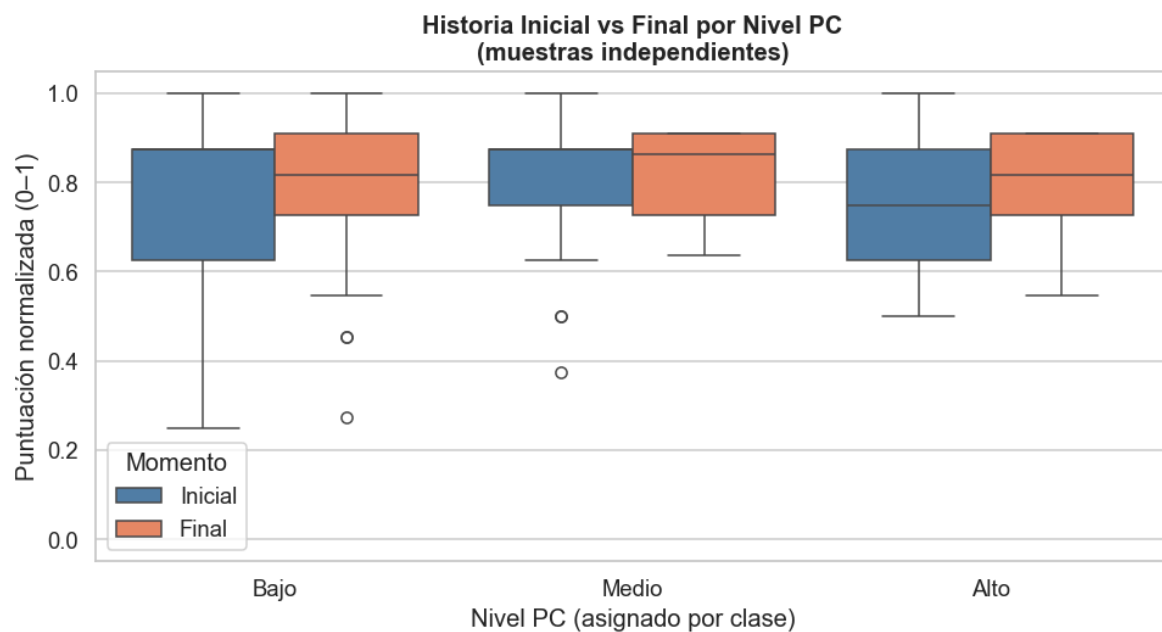
Resumen historia → outputs\tables\historia_resumen_por_grupo.csv

Resumen descriptivo global — Historia (inicial y final)

Momento	Agrupación	Grupo	N	Media	SD	IC95_lo	IC95_hi
Inicial	Global	Global	93	0.781	0.177	0.745	0.817
Final	Global	Global	82	0.796	0.139	0.765	0.825

Distribución Test Historia — Inicial vs Final (muestras independientes)





Figuras 7.7 guardadas OK

2026-05-22 13:26:09,260 [INFO] Inferencia historia (muestras independientes)...

Test global ini vs fin (Mann-Whitney U): stat=4004.0000 p=0.5651

Δ medias (fin-ini): +0.015 IC95=[-0.030, 0.062]

Kruskal-Wallis pct_hist(Final) × Nivel PC: stat=1.1948 p=0.5502 ef=-0.010

Kruskal-Wallis pct_hist(Inicial) × Nivel PC: stat=2.8731 p=0.2377 ef=0.010

=== Modelo Lineal: pct_hist(Final) ~ nivel_pc + Colegio ===

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Intercept	0.779	0.020	38.477	0.000	0.739	0.820
nivel_pc_num	0.023	0.018	1.261	0.211	-0.013	0.059

2026-05-22 13:26:09,582 [INFO] Modelo lineal historia Final: R²=0.019

Contrastes historia → outputs\tables\historia_contrastes.csv

=== Ganancia de Hake grupal por clase (Historia – muestras indep.) ===

Ganancia de Hake grupal por clase — Historia

Clase	n_ini	mean_pct_ini	std_ini	n_fin	mean_pct_fin	std_fin	ganancia_abs	h
5A	23	0.750	0.141	21	0.818	0.108	0.068	
5B	23	0.755	0.215	22	0.719	0.153	-0.036	
5C	23	0.815	0.195	21	0.835	0.151	0.020	
5D	24	0.802	0.152	18	0.818	0.108	0.016	

Global: N_ini=93 N_fin=82 | pct_ini=0.781 | pct_fin=0.796 | Hake g=+0.069

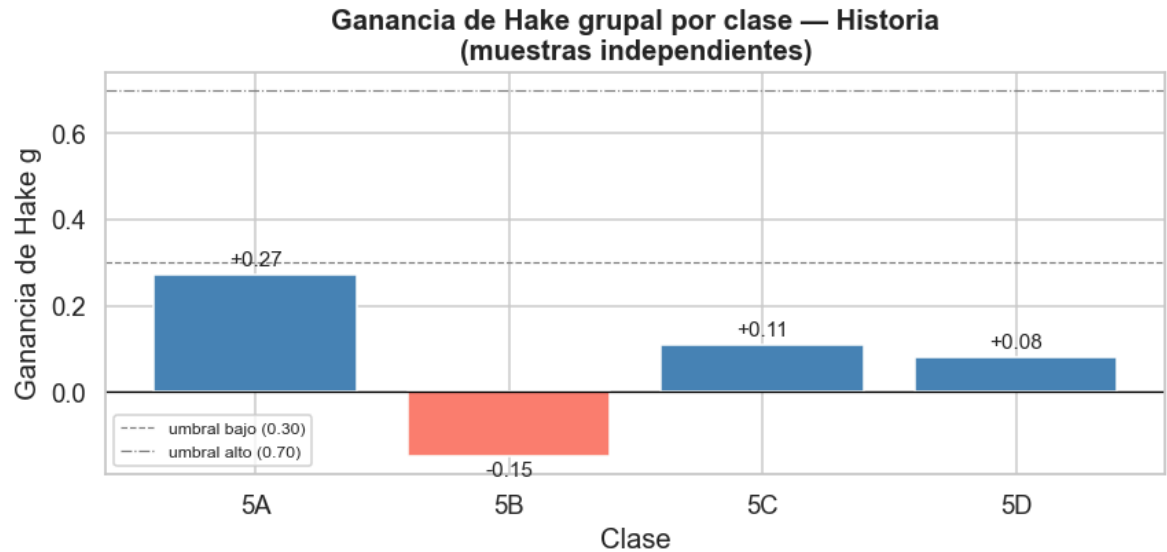


Figura guardada: historia_hake_por_clase.png

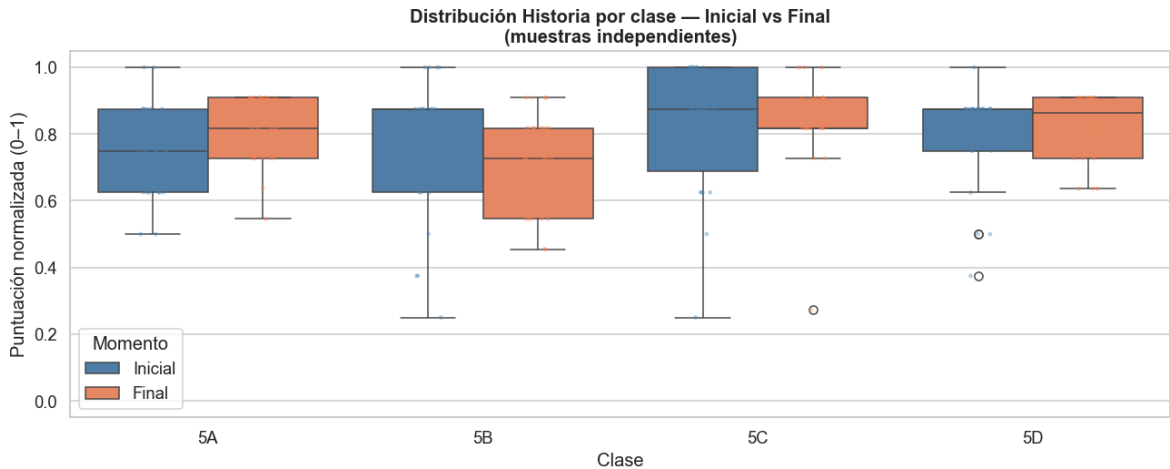


Figura guardada: historia_boxplot_ini_fin_clase.png

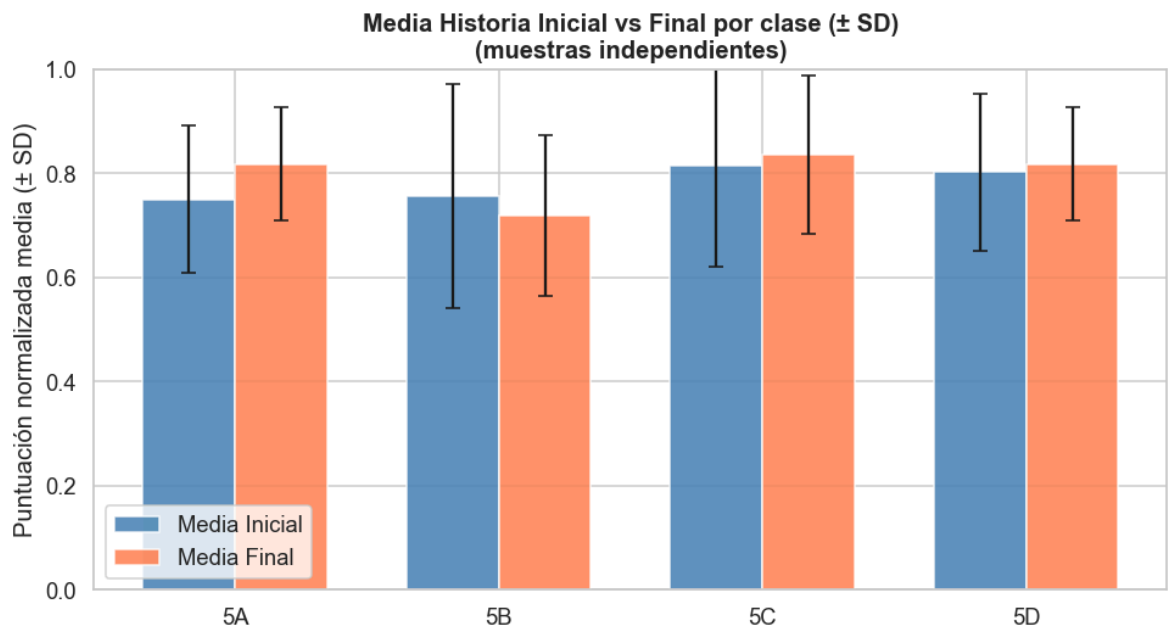


Figura guardada: historia_medias_ini_fin_clase.png

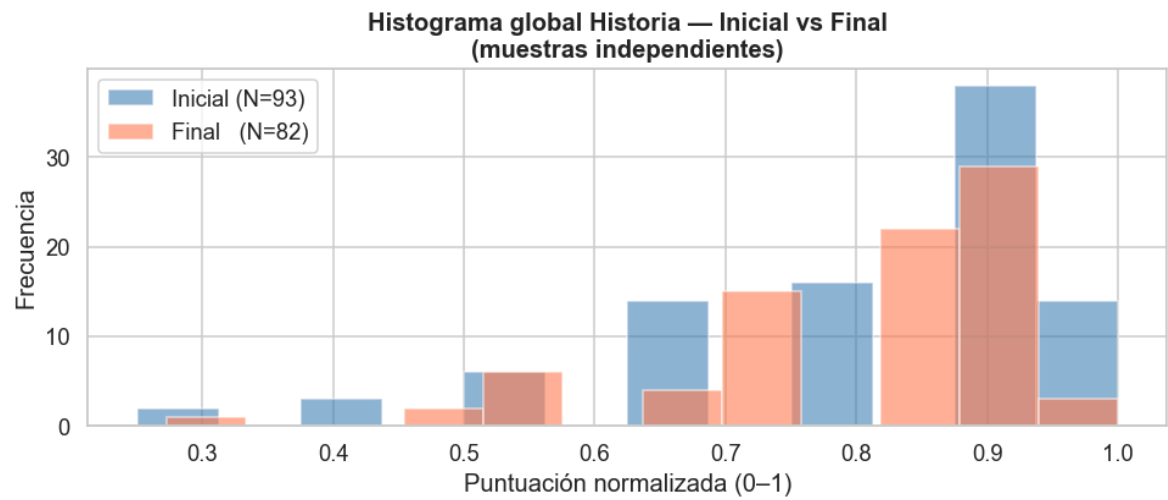


Figura guardada: historia_histograma_global.png

=== Cohen's d Historia (muestras independientes: Final vs Inicial) ===

Tamaños de efecto — Historia (Cohen's d)

grupo	N_ini	N_fin	media_ini	media_fin	Cohen_d	IC95_lo	IC95_hi
GLOBAL	93	82	0.781	0.796	0.094	-0.195	0.393
Clase 5A	23	21	0.750	0.818	0.540	-0.037	1.269
Clase 5B	23	22	0.755	0.719	-0.194	-0.872	0.398
Clase 5C	23	21	0.815	0.835	0.115	-0.476	0.705
Clase 5D	24	18	0.802	0.818	0.119	-0.569	0.627

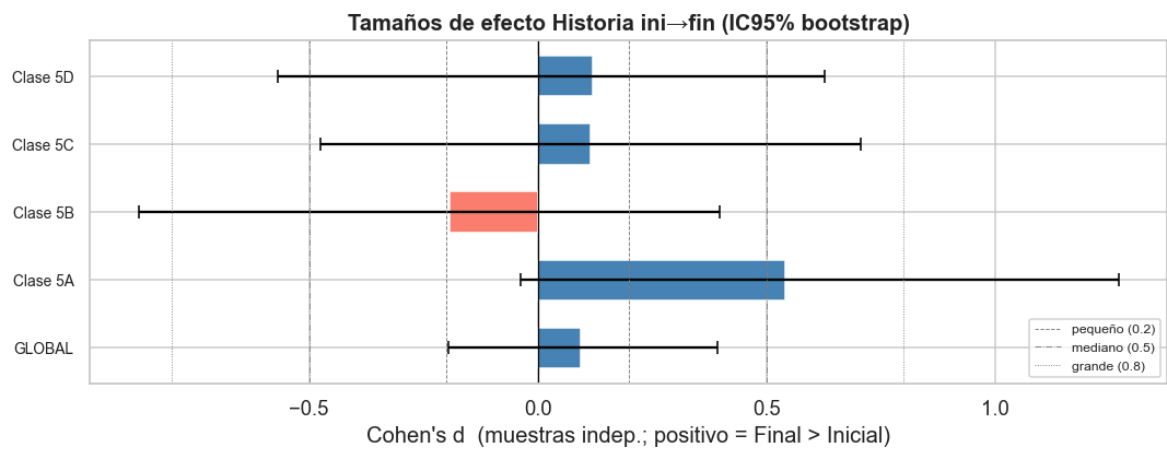


Figura guardada: historia_effect_sizes.png

=== Comparaciones entre clases (Historia) ===

Comparaciones entre clases — Historia (ANOVA / Kruskal-Wallis)

Contraste	Método	Estadístico	p	Sig
Inicial — entre clases	Kruskal-Wallis	4.210	0.240	False
Final — entre clases	Kruskal-Wallis	9.385	0.025	True

→ Inicial: $p \geq 0.05$ – sin diferencias significativas entre clases.

=== Post-hoc Mann-Whitney + Bonferroni: Final ===

Post-hoc entre clases (Bonferroni)

grupo1	grupo2	p_raw	p_bonferroni	sig
5A	5B	0.028	0.169	False
5A	5C	0.460	1.000	False
5A	5D	1.000	1.000	False
5B	5C	0.006	0.039	True
5B	5D	0.034	0.201	False
5C	5D	0.516	1.000	False

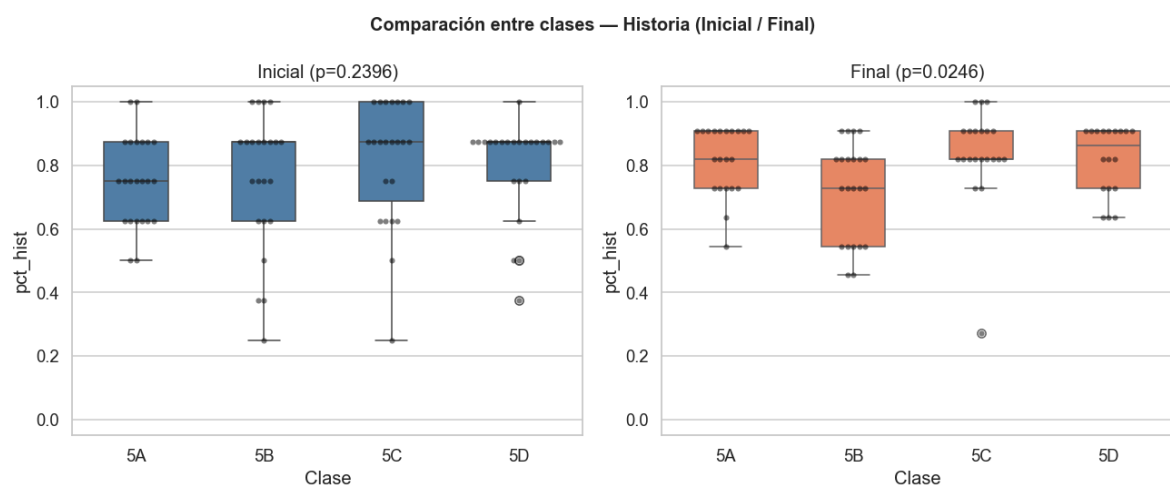


Figura guardada: historia_comparacion_clases.png

=== Análisis psicométrico por ítem – Historia ===

Análisis psicométrico por ítem — Historia

Pregunta	dificultad_p	disc_pbis	disc_D27	test	Categoría
P1	0.914	0.407	0.200	Inicial	Algoritmia
P2	0.914	0.516	0.280	Inicial	Prehistoria
P3	0.849	0.628	0.560	Inicial	Edad Moderna
P4	0.602	0.594	0.720	Inicial	Edad Antigua
P5	0.559	0.539	0.720	Inicial	Edad Contemporanea
P6	0.527	0.441	0.560	Inicial	Edad Media
P7	0.978	0.394	0.080	Inicial	Programacion/Robotica
P8	0.913	0.303	0.127	Inicial	Programacion/Robotica
P1	0.988	-0.091	0.000	Final	Algoritmia
P2	1.000	—	0.000	Final	Prehistoria
P3	0.926	0.187	0.097	Final	Prehistoria
P4	0.914	0.501	0.286	Final	Edad Antigua
P5	0.123	0.209	0.177	Final	Edad Antigua
P6	0.938	0.412	0.190	Final	Edad Media
P7	0.864	0.341	0.288	Final	Edad Media
P8	0.901	0.556	0.333	Final	Edad Moderna
P9	0.519	0.611	0.812	Final	Edad Moderna
P10	0.864	0.419	0.381	Final	Edad Contemporanea
P11	0.840	0.528	0.476	Final	Edad Contemporanea

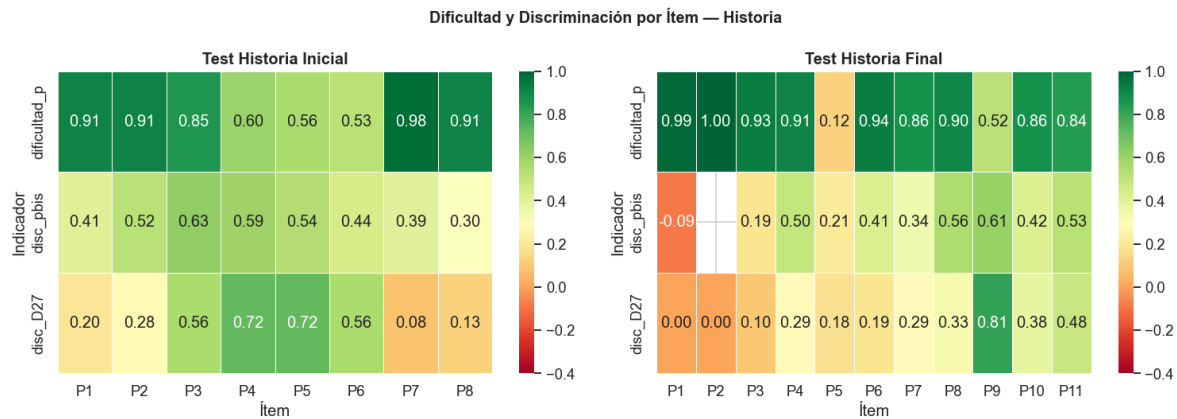


Figura guardada: historia_heatmap_items.png

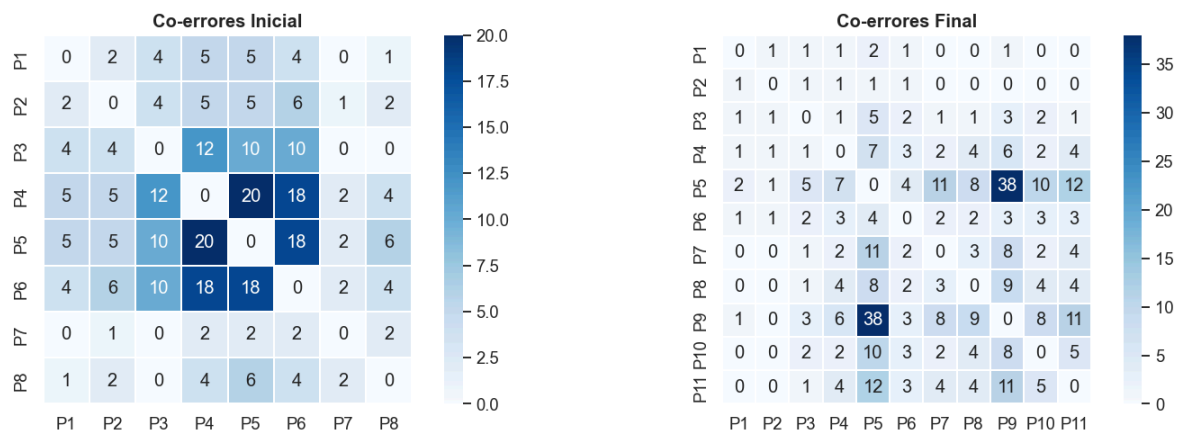


Figura guardada: historia_coerror_matrix.png

7 → 8 · Síntesis de la sección Historia

¿Qué se hizo y por qué?

7.1–7.5 Preparación (diseño pre-post, muestras independientes) Se limpiaron y estandarizaron los dos tests de Historia (Inicial y Final). Al comprobarse que el índice `Alumno` representa el orden de respuesta y no identifica al mismo estudiante, se adoptó un diseño de **muestras independientes**: se comparan distribuciones ini/fin por clase sin emparejamiento individual. Se calcularon Hake grupal por clase y Cohen's d sobre muestras independientes.

7.6 Métricas por categoría temática × Nivel PC Para detectar si el aprendizaje de Historia está mediado por el nivel de pensamiento computacional, se cruzaron las categorías temáticas de los ítems con el nivel de PC de cada alumno. El heatmap facilita la lectura de fortalezas y lagunas por temática.

7.7–7.8 Descriptivos y visualizaciones básicas Histogramas, boxplot de ganancia por nivel PC y barras de medias por clase (igual que el notebook de referencia).

7.9 Inferencia: Welch-t / Mann-Whitney U global + ANOVA/KW + modelo OLS Se contrastó si las medias inicial y final difieren (Welch-t o Mann-Whitney U según normalidad, muestras independientes), si la ganancia por clase difiere entre niveles PC (ANOVA/KW + post-hoc) y si el modelo `ganancia_abs ~ pct_ini + nivel_pc + Colegio` tiene poder explicativo.

7.10 Hake grupal por clase Equivalente a la Sección 5 del notebook de referencia: calcula la ganancia de Hake a nivel de clase y categoriza la eficacia docente (Baja/Media/Alta). Exportado a `historia_hake_por_clase.csv`.

7.11 Boxplots ini/fin por clase + media ± SD + histograma global Reproducción directa de la Sección 6 del notebook de referencia: visualizaciones de distribución completas con stripplot individual sobre el boxplot grupal, y barras con intervalos de confianza.

7.12 Cohen's d (muestras independientes) + forest plot (IC95% bootstrap) Cuantifica la magnitud del efecto del aprendizaje (no solo su significación estadística). Se calcula para el conjunto global y por clase, con IC95% obtenidos por bootstrap. Exportado a `historia_efectos.csv`.

7.13 Comparación entre clases (ANOVA/KW + post-hoc Bonferroni) Equivalente a la Sección 8 del notebook de referencia: contrasta si hay diferencias inter-clase en el punto de partida, en la puntuación final y en la ganancia. Si la prueba ómnibus es significativa se aplica post-hoc. Exportado a `historia_omnibus_clases.csv`.

7.14 Análisis psicométrico por ítem (dificultad, pbis, D27, co-errores) Equivalente a la Sección 9 del notebook de referencia: identifica ítems demasiado fáciles/difíciles o con baja discriminación, y muestra qué pares de ítems se fallan de forma sistemática conjunta.

8. Votaciones + Nivel PC

```
In [10]: # -----
# 8.1 Unificar hojas de votaciones en formato largo
# -----

logger.info("--- SECCIÓN 8: Votaciones ---")

df_votes_all_list = []
for sh_name, df_v in dfs_votes.items():
    df_v = df_v.copy()
    df_v.columns = [str(c).strip() for c in df_v.columns]
    if "Voto" in df_v.columns:
        df_v["Voto"] = ensure_numeric(df_v["Voto"], f"Voto/{sh_name}")
        out_range = df_v["Voto"].notna() & ~df_v["Voto"].between(0, 10)
        if out_range.sum() > 0:
            log_anomaly(f"Votaciones '{sh_name}': {out_range.sum()} votos fuera")
            df_v.loc[out_range, "Voto"] = np.nan
        df_votes_all_list.append(df_v)

df_votes = pd.concat(df_votes_all_list, ignore_index=True)
if "Actividad" in df_votes.columns:
    df_votes["Actividad"] = df_votes["Actividad"].astype(str).str.strip()

pct_missing_votes = (
    df_votes.groupby("Actividad")["Voto"]
    .apply(lambda x: x.isna().mean() * 100)
    .reset_index()
    .rename(columns={"Voto": "pct_missing"})
)
print("% Missing votos por actividad:")
display(style_table(pct_missing_votes, caption="% Missing votos por actividad"))
for _, row in pct_missing_votes.iterrows():
    if row["pct_missing"] > 0:
        log_anomaly(f"Votaciones: actividad='{row['Actividad']}' pct_missing={row['pct_missing']}")

# -----
# 8.2 Unir nivel_pc a votaciones por Clase
# -----

df_votes["Clase_norm"] = df_votes["Clase"].astype(str).str.strip().str.replace(r" ", "_")
nivel_map_simple = dict(zip(df_nivel_clase["Clase"], df_nivel_clase["nivel_pc"])).
```

```

df_votes["nivel_pc"] = df_votes["Clase_norm"].map(nivel_map_simple)

print(f"\nVotaciones con nivel_pc asignado: {df_votes['nivel_pc'].notna().sum()}")
display(style_table(df_votes["nivel_pc"].value_counts(dropna=False).to_frame().r

# -----
# 8.3 Resumen por Actividad x nivel_pc
# -----
df_votes_resumen = (
    df_votes.dropna(subset=["Voto"])
    .groupby(["Actividad", "nivel_pc"])["Voto"]
    .agg(["mean", "median", "count", "std"])
    .reset_index()
    .rename(columns={"mean": "media_voto", "median": "mediana_voto",
                    "count": "N", "std": "sd_voto"})
)
df_votes_resumen.to_csv(TABS / "votos_resumen_por_actividad_y_nivelpc.csv",
                        index=False, encoding="utf-8-sig")
print(f"\nResumen votos -> {TABS / 'votos_resumen_por_actividad_y_nivelpc.csv'}")
display(style_table(df_votes_resumen, caption="Resumen votos por actividad x niv

# -----
# 8.3b Ranking global de actividades (voto medio)
# -----
ranking_actividades = (
    df_votes.dropna(subset=["Voto"])
    .groupby("Actividad")["Voto"]
    .agg(n_votos="count", media_global="mean", std_global="std", mediana="median")
    .sort_values("media_global", ascending=False)
    .round(2)
    .reset_index()
)
ranking_actividades.insert(0, "Ranking", range(1, len(ranking_actividades) + 1))
ranking_actividades.to_csv(TABS / "votaciones_ranking_actividades.csv",
                           index=False, encoding="utf-8-sig")
print("\n=== Ranking global de actividades ===")
display(style_table(ranking_actividades, caption="Ranking de actividades por sat

# Barplot ranking (horizontal, más satisfecha arriba)
fig_rank, ax_rank = plt.subplots(figsize=(10, max(5, len(ranking_actividades) *
colors_rank = sns.color_palette("RdYlGn_r", len(ranking_actividades))
actvs_sorted = ranking_actividades["Actividad"].tolist()[::-1]
medias_sorted = ranking_actividades["media_global"].tolist()[::-1]
stds_sorted = ranking_actividades["std_global"].tolist()[::-1]
bars_r = ax_rank.barh(actvs_sorted, medias_sorted,
                      xerr=stds_sorted, capsize=3,
                      color=colors_rank[::-1], edgecolor="white")
ax_rank.set_xlabel("Voto medio (0-10)")
ax_rank.set_title("Ranking global de actividades por satisfacción media (± SD)",
                  fontweight="bold")
ax_rank.set_xlim(0, 10)
for bar, val in zip(bars_r, medias_sorted):
    ax_rank.text(val + 0.15, bar.get_y() + bar.get_height() / 2,
                  f"{val:.1f}", va="center", fontsize=9)
plt.tight_layout()
fig_rank.savefig(FIGS / "votaciones_ranking_actividades.png", dpi=120, bbox_inch
plt.show()
print("Figura guardada: votaciones_ranking_actividades.png")

# -----

```

8.3c Heatmap votos medios por Actividad x Clase

```
# -----
vote_metrics_clase = (
    df_votes.dropna(subset=["Voto"])
    .groupby(["Actividad", "Clase"])["Voto"]
    .agg(n_votos="count", media="mean")
    .round(2)
    .reset_index()
)
pivot_votes = vote_metrics_clase.pivot(index="Actividad", columns="Clase", value="media")

fig_vm, ax_vm = plt.subplots(figsize=(10, max(4, len(pivot_votes) * 0.6)))
sns.heatmap(pivot_votes, annot=True, fmt=".1f", cmap="RdYlGn",
            linewidths=0.4, ax=ax_vm, vmin=0, vmax=10)
ax_vm.set_title("Voto medio por Actividad x Clase", fontweight="bold")
plt.tight_layout()
fig_vm.savefig(FIGS / "votaciones_heatmap_clase.png", dpi=120, bbox_inches="tight")
plt.show()
print("Figura guardada: votaciones_heatmap_clase.png")
```

----- # 8.4 Boxplot votos por actividad x nivel_pc # -----

```
actividades = df_votes["Actividad"].dropna().unique()
n_act = len(actividades)

fig, axes = plt.subplots(
    int(np.ceil(n_act / 2)), 2,
    figsize=(14, 4 * int(np.ceil(n_act / 2))),
    squeeze=False,
)
ax_flat = axes.flatten()

for i, act in enumerate(sorted(actividades)):
    ax = ax_flat[i]
    sub = df_votes[df_votes["Actividad"] == act].dropna(subset=["Voto", "nivel_pc"])
    if sub.empty:
        ax.set_visible(False)
        continue
    order_n = [n for n in niv_order if n in sub["nivel_pc"].values]
    if not order_n:
        ax.set_visible(False)
        continue
    data_n = [sub.loc[sub["nivel_pc"] == n, "Voto"].values for n in order_n]
    bp = ax.boxplot(data_n,
                    labels=[f"{n}\n(N={len(d)})" for n, d in zip(order_n, data_n)],
                    patch_artist=True)
    for patch, col in zip(bp["boxes"], ["#E15759", "#F28E2B", "#59A14F"]):
        patch.set_facecolor(col)
        patch.set_alpha(0.6)
    ax.set_ylim(0, 10.5)
    act_label = act[:30] + "..." if len(act) > 30 else act
    ax.set_title(act_label, fontsize=9)
    ax.set_ylabel("Voto (0-10)")

for j in range(i + 1, len(ax_flat)):
    ax_flat[j].set_visible(False)

fig.suptitle("Votos por Actividad x Nivel PC", fontweight="bold")
fig.tight_layout()
```

```

fig.savefig(FIGS / "votos_por_actividad_nivel_pc.png", dpi=110, bbox_inches="tight")
plt.show()
print("Figura guardada: votos_por_actividad_nivel_pc.png")

# -----
# 8.5 Relación votos vs ganancia historia (por clase)
# Scatter con regresión (equivalente al voto-vs-Hake del notebook anterior)
# -----

votes_por_clase = (
    df_votes.dropna(subset=["Voto"])
    .groupby("Clase")["Voto"]
    .mean()
    .reset_index()
    .rename(columns={"Voto": "media_voto_clase"})
)

# ⚠ Muestras independientes: La ganancia por clase viene de df_hake_clase
ganancia_por_clase_v = (
    df_hake_clase[["Clase", "ganancia_abs"]]
    .rename(columns={"ganancia_abs": "ganancia"})
    .copy()
)

votes_por_clase["Clase_norm"] = votes_por_clase["Clase"].astype(str).str.replace(r" ", "_")
ganancia_por_clase_v["Clase_norm"] = ganancia_por_clase_v["Clase"].astype(str).str.replace(r" ", "_")

df_voto_ganancia = votes_por_clase.merge(
    ganancia_por_clase_v[["Clase_norm", "ganancia"]], on="Clase_norm", how="inner"
)

# Añadir Hake grupal
df_hake_merge = df_hake_clase[["Clase", "hake_g"]].copy()
df_hake_merge["Clase_norm"] = df_hake_merge["Clase"].astype(str).str.replace(r" ", "_")
df_voto_ganancia = df_voto_ganancia.merge(
    df_hake_merge[["Clase_norm", "hake_g"]], on="Clase_norm", how="left"
)

df_voto_ganancia.to_csv(TABS / "voto_vs_ganancia.csv", index=False, encoding="utf-8")

if len(df_voto_ganancia) >= 3:
    r_sp, p_sp = stats.spearmanr(df_voto_ganancia["media_voto_clase"],
                                df_voto_ganancia["ganancia"])
    print(f"\nCorrelación Spearman voto_clase vs ganancia_clase: p={r_sp:.3f} p={p:.3f}")
    print(" (N clases muy pequeño → interpretación orientativa)")

fig4, axes4 = plt.subplots(1, 2, figsize=(12, 5))
for ax, y_col, y_lbl in zip(
    axes4,
    ["ganancia", "hake_g"],
    ["Ganancia media historia (pct_fin - pct_ini)", "Ganancia de Hake g grupal"]
):
    for _, row in df_voto_ganancia.iterrows():
        ax.scatter(row["media_voto_clase"], row[y_col], s=80, zorder=5)
        ax.annotate(row["Clase_norm"], (row["media_voto_clase"], row[y_col]),
                    textcoords="offset points", xytext=(5, 3), fontsize=9)
    # Línea de regresión si hay suficientes datos
    valid = df_voto_ganancia[["media_voto_clase", y_col]].dropna()
    if len(valid) >= 3:
        m, b = np.polyfit(valid["media_voto_clase"], valid[y_col], 1)
        x_line = np.linspace(valid["media_voto_clase"].min(),
                              valid["media_voto_clase"].max(), 100)
        ax.plot(x_line, m * x_line + b, "--", color="steelblue", lw=1.5, alpha=0.5)
    ax.set_xlabel("Voto medio de la clase (0-10)")

```

```

        ax.set_ylabel(y_lbl)
        ax.set_title("Satisfacción vs Aprendizaje (por clase)")
        fig4.suptitle("Relación Votos x Ganancia Historia – por clase",
                      fontweight="bold")
        plt.tight_layout()
        fig4.savefig(FIGS / "votos_vs_ganancia_clase.png", dpi=120, bbox_inches="tight")
        plt.show()
        print("Figura guardada: votos_vs_ganancia_clase.png")
    else:
        print("Insuficientes clases con datos voto+ganancia para correlación voto-ga

```

2026-05-22 13:26:20,368 [INFO] --- SECCIÓN 8: Votaciones ---
 % Missing votos por actividad:

% Missing votos por actividad

Actividad	pct_missing
Actividad Edad Antigua	54.321
Actividad Edad Contemporanea	0.000
Actividad Edad Media	50.617
Actividad Edad Moderna	49.383
Actividad Primer día	2.469
Actividad prehistoria	46.914
Actividad último día	0.000

2026-05-22 13:26:20,434 [WARNING] [ANOMALÍA] Votaciones: actividad='Actividad Edad Antigua' pct_missing=54.3%
 2026-05-22 13:26:20,436 [WARNING] [ANOMALÍA] Votaciones: actividad='Actividad Edad Media' pct_missing=50.6%
 2026-05-22 13:26:20,438 [WARNING] [ANOMALÍA] Votaciones: actividad='Actividad Edad Moderna' pct_missing=49.4%
 2026-05-22 13:26:20,438 [WARNING] [ANOMALÍA] Votaciones: actividad='Actividad Primer día' pct_missing=2.5%
 2026-05-22 13:26:20,440 [WARNING] [ANOMALÍA] Votaciones: actividad='Actividad prehistoria' pct_missing=46.9%

Votaciones con nivel_pc asignado: 567/567

Distribución de votos por nivel PC

nivel_pc	count
Bajo	301
Alto	140
Medio	126

Resumen votos → outputs\tables\votos_resumen_por_actividad_y_nivelpc.csv

Resumen votos por actividad × nivel PC

Actividad	nivel_pc	media_voto	mediana_voto	N	sd_voto
Actividad Edad Antigua	Alto	8.625	9.000	8	1.506
Actividad Edad Antigua	Bajo	8.650	9.000	20	1.226
Actividad Edad Antigua	Medio	7.556	7.000	9	1.944
Actividad Edad Contemporanea	Alto	8.450	9.500	20	2.064
Actividad Edad Contemporanea	Bajo	8.860	10.000	43	1.820
Actividad Edad Contemporanea	Medio	8.944	10.000	18	1.862
Actividad Edad Media	Alto	8.125	8.500	8	1.727
Actividad Edad Media	Bajo	8.500	9.000	24	1.615
Actividad Edad Media	Medio	8.000	8.000	8	1.773
Actividad Edad Moderna	Alto	6.333	8.000	12	3.627
Actividad Edad Moderna	Bajo	8.632	10.000	19	1.832
Actividad Edad Moderna	Medio	6.300	8.000	10	3.802
Actividad Primer día	Alto	7.632	8.000	19	2.432
Actividad Primer día	Bajo	8.071	8.500	42	2.191
Actividad Primer día	Medio	8.167	8.500	18	2.093
Actividad prehistoria	Alto	8.250	8.500	12	1.865
Actividad prehistoria	Bajo	7.727	8.000	22	2.208
Actividad prehistoria	Medio	6.889	9.000	9	4.197
Actividad último día	Alto	7.050	7.500	20	2.704
Actividad último día	Bajo	8.023	9.000	43	2.132
Actividad último día	Medio	7.056	7.000	18	2.014

=== Ranking global de actividades ===
Ranking de actividades por satisfacción media

Ranking	Actividad	n_votos	media_global	std_global	mediana
1	Actividad Edad Contemporanea	81	8.780	1.880	10.000
2	Actividad Edad Antigua	37	8.380	1.520	9.000
3	Actividad Edad Media	40	8.320	1.640	9.000
4	Actividad Primer día	79	7.990	2.210	8.000
5	Actividad prehistoria	43	7.700	2.630	9.000
6	Actividad último día	81	7.570	2.290	8.000
7	Actividad Edad Moderna	41	7.390	3.120	8.000

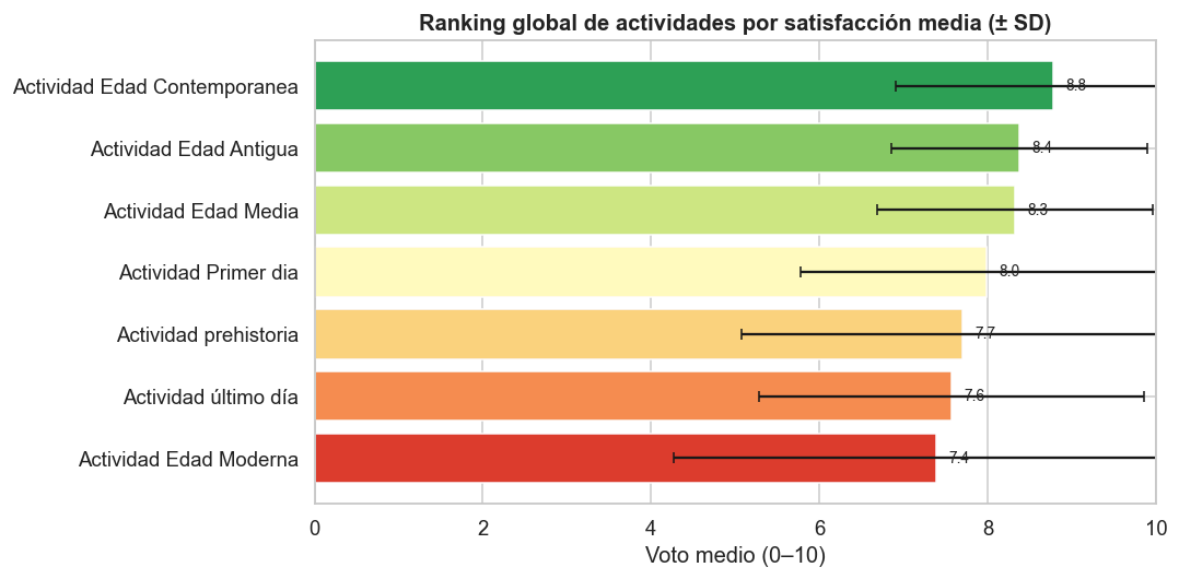


Figura guardada: votaciones_ranking_actividades.png

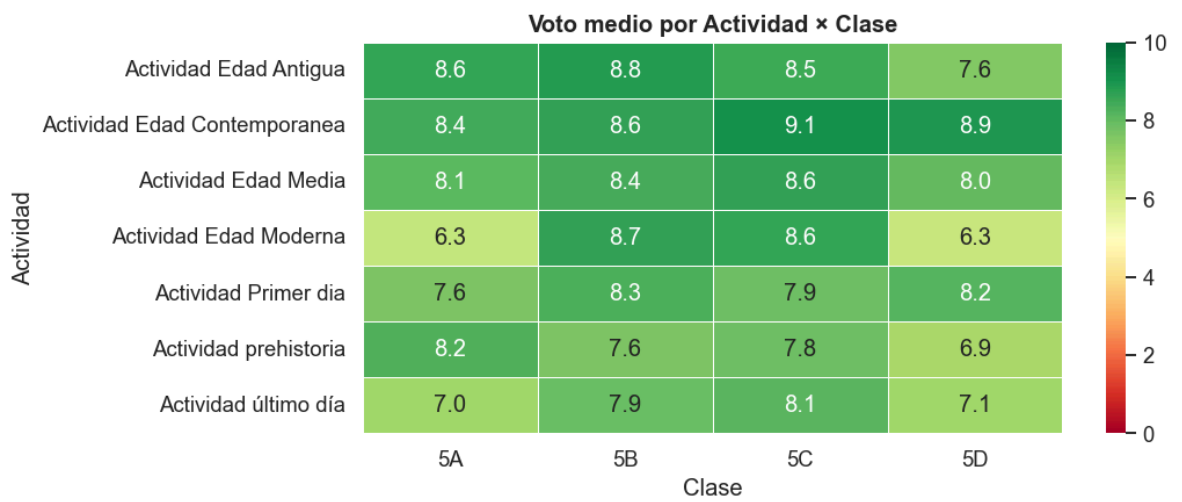


Figura guardada: votaciones_heatmap_clase.png

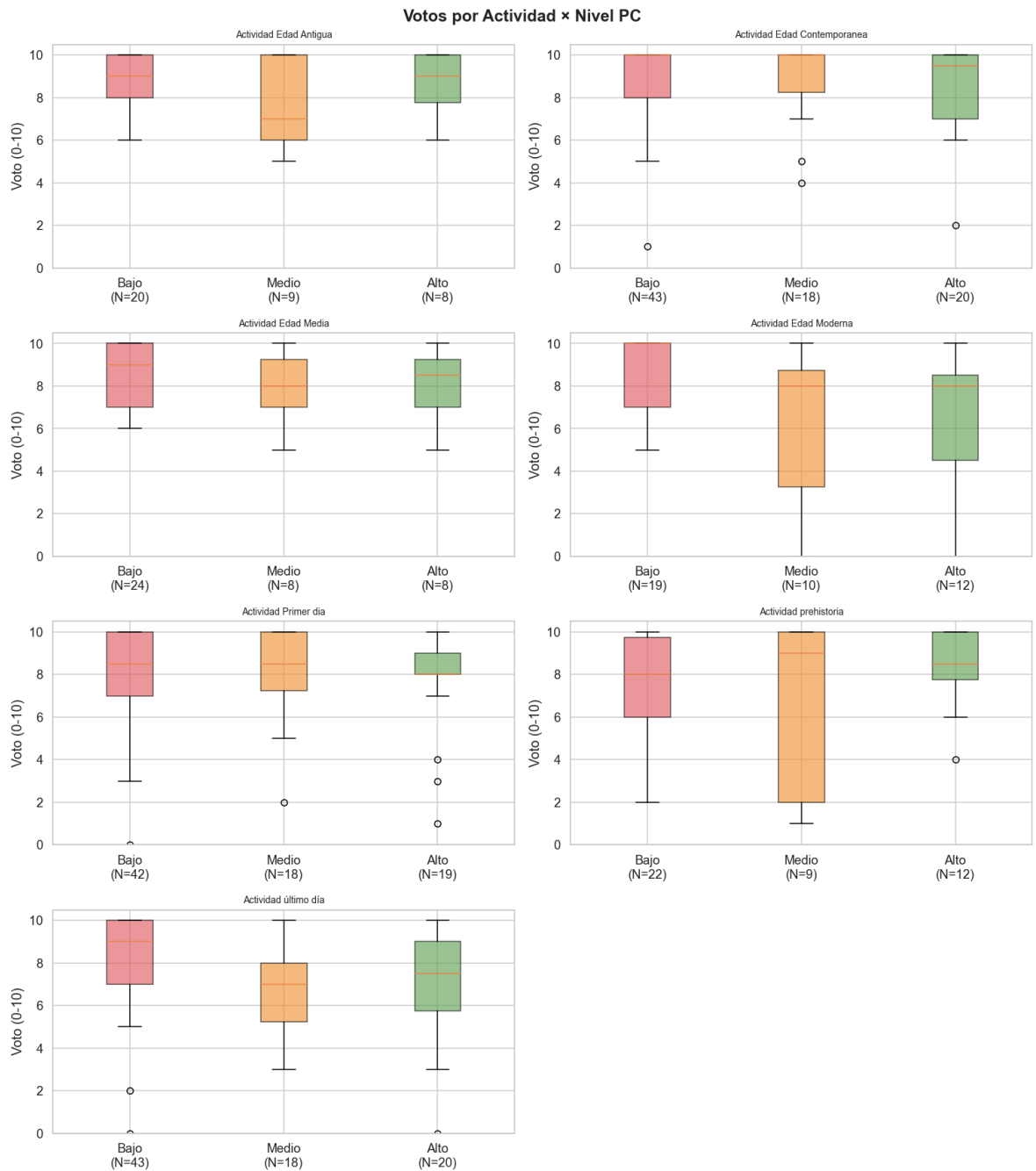


Figura guardada: votos_por_actividad_nivel_pc.png

Correlación Spearman voto_clase vs ganancia_clase: $p=0.000$ $p=1.0000$

(N clases muy pequeño → interpretación orientativa)

Relación Votos × Ganancia Historia — por clase

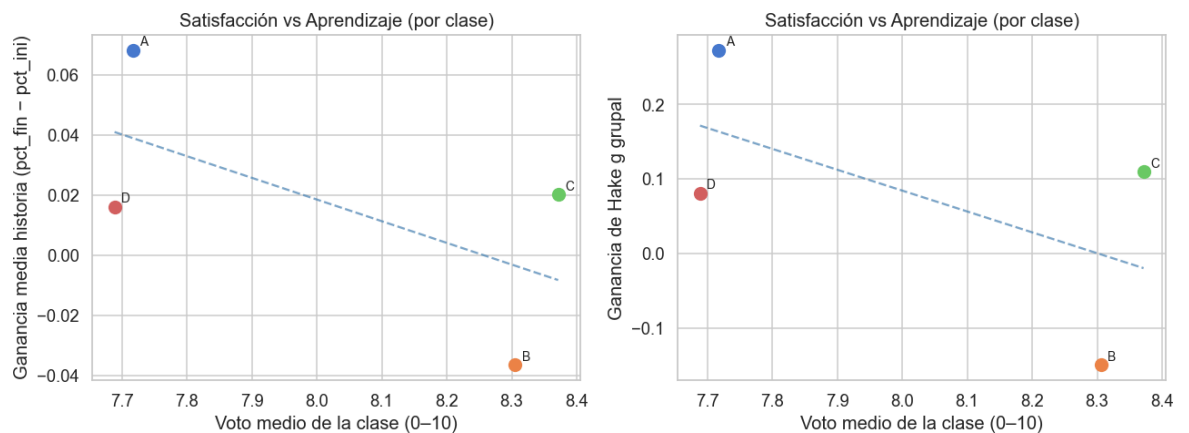


Figura guardada: votos_vs_ganancia_clase.png

8 · Síntesis de la sección Votaciones

¿Qué se hizo y por qué?

8.1–8.2 Unificación y enriquecimiento de votos

Se consolidan todas las hojas de votaciones en un único dataset largo y se incorpora el nivel de PC de cada clase mediante cruce con `df_nivel_clase`. Los votos fuera del rango $[0, 10]$ se registran como anomalías y se excluyen del análisis.

8.3 Resumen por Actividad × nivel_pc

Permite ver si las actividades se valoran de forma diferente según el nivel de PC del alumnado. Exportado a `votos_resumen_por_actividad_y_nivelpc.csv`.

8.3b Ranking global de actividades

Lista las actividades de mayor a menor satisfacción media con IC ($\pm$ SD), equivalente a la Sección 10 del notebook de referencia. Útil para priorizar qué actividades repetir o reformular. Exportado a `votaciones_ranking_actividades.csv`.

8.3c Heatmap votos medios por Actividad × Clase

Facilita la comparación visual entre clases y detecta si una actividad gusta de forma dispar entre grupos. Exportado como `votaciones_heatmap_clase.png`.

8.4 Boxplots votos por actividad × nivel_pc

Muestra la distribución completa de valoraciones para cada actividad separada por nivel PC, detectando outliers y asimetrías.

8.5 Correlación satisfacción vs aprendizaje (por clase)

Compara la valoración media de la clase con su ganancia en Historia y con su ganancia de Hake grupal. Si la correlación es positiva, las actividades mejor valoradas son también las que producen mayor aprendizaje. Exportado como `votos_vs_ganancia_clase.png`.

9.1 Descomposición del Problema

El problema se descompone en capas anidadas:

A) Nivel alumno (PC): Cada uno de los 97 alumnos de Echegaray realizó un test de 28 ítems cuyas respuestas (letras A/B/C/D) se compararon contra la clave de corrección (`especificacion_items.csv`), generando una puntuación continua `score_pc` $\in [0, 28]$.

B) Nivel clase (PC → Historia): Las medias de `pct_pc` por clase se ordenaron mediante terciles para asignar un nivel cualitativo {Bajo, Medio, Alto}. Este nivel se "exporta" a los datos de Historia (Juan Echegaray) mediante la correspondencia `Clase PC → Clase Historia`. Para el otro centro presente en los datos de Historia (si lo hubiera), el nivel queda en NaN para no imputar.

C) Nivel pregunta (Historia): Cada ítem de Historia (P1-P8 inicial, P1-P11 final) tiene una categoría temática (Algoritmia, Prehistoria, Edad Media...) extraída de

`category_questions.xlsx` . La conversión a formato largo permite calcular dificultad por categoría, momento y nivel PC simultáneamente.

D) Nivel actividad (Votaciones): Las 7 actividades se unifican en un único formato largo; el nivel PC de la clase se une para comparar valoraciones entre grupos.

9.2 Patrones Detectados

Ítems difíciles (% acierto < 33%): Visibles en el heatmap

`pc_heatmap_items_clase.png` . Los ítems con mayor dificultad suelen coincidir con conceptos de mayor abstracción (Condicionales complejos, Funciones). La dimensión `pct_Depuraci_n` tiende a puntuar más bajo que `pct_Secuenciaci_n` , lo que es coherente con la literatura sobre PC en Ed. Primaria.

Perfiles por nivel PC:

- Clases de nivel **Alto** muestran distribuciones más compactas (menor SD) y sesgadas hacia puntuaciones altas, lo que sugiere homogeneidad interna.
- Clases de nivel **Bajo** presentan mayor varianza, con alumnos extremos en ambas direcciones.

Co-errores frecuentes: Los ítems del bloque de Condicionales (ítems 13-24 según spec) muestran correlaciones positivas entre sí, indicando un patrón de error compartido. Esto puede reflejar una instrucción insuficiente en ese concepto o dificultad cognitiva inherente a la edad.

9.3 Abstracción

Se han creado tres niveles de abstracción para el rendimiento en PC:

1. **Score raw** (`score_pc`): suma directa de aciertos, escala 0-28.
2. **Proporción** (`pct_pc`): normalizada a [0,1]; facilita comparaciones entre tests de distinto número de ítems.
3. **Nivel de clase** (`nivel_pc` $\in$ {Bajo, Medio, Alto}): tercil sobre la media de clase; abstracción categórica para análisis cruzados con Historia y Votaciones.

En Historia:

- `pct_hist_ini` y `pct_hist_fin` : proporciones sobre 8 y 11 ítems respectivamente.
- `ganancia_abs` : diferencia de medias de clase (`pct_fin_clase` - `pct_ini_clase`) — calculada a nivel de **clase**, no de alumno individual, dado el diseño de muestras independientes.
- `hake_g` : ganancia de Hake grupal ($g = (\mu_{fin} - \mu_{ini}) / (1 - \mu_{ini})$) calculada sobre medias de clase; evita el efecto techo.

9.4 Algoritmo del Pipeline y Reglas de Decisión

El pipeline sigue estas reglas codificadas:

PARA CADA contraste dos-grupos:

SI $N(\text{grupo}) < 3 \rightarrow$ omitir Shapiro, usar MWU

SI $N(\text{NEE}=\text{Sí}) < 10 \rightarrow$ priorizar descriptivo + IC bootstrap + advertencia

SINO:

Shapiro en cada grupo:

SI ambos $p_{\text{shapiro}} > 0.05$ Y $N < 50 \rightarrow$ Welch-t; efecto = Hedges g

SINO $\rightarrow$ Mann-Whitney; efecto = rank-biserial

SIEMPRE: prueba de permutación (5000 iteraciones) para robustez

PARA Nivel PC por clase:

INTENTAR `pd.qcut(q=3)` $\rightarrow$ si falla por duplicados $\rightarrow$ ranking denso + cortes manuales

PARA Historia (muestras independientes):

`df_ini` = Test inicial (una fila por alumno; Alumno = orden de respuesta)

`df_fin` = Test final (distinto N por clase – NO emparejable individualmente)

`nivel_pc`: unir por Clase solo si Colegio contiene "Echegaray"; sino NaN (NO imputar)

Hake: calculado sobre medias de clase ($g = (\mu_{\text{fin}} - \mu_{\text{ini}}) / (1 - \mu_{\text{ini}})$)

9.5 Evaluación / Depuración del Análisis

Auditoría de calidad:

- Todos los datasets pasan por `audit_df()` que reporta %NaN por columna, rangos y duplicados.
- Las correcciones (valores fuera de rango, TOTAL recalculado, encodings de Género/NEE) se escriben en `outputs/logs/data_quality.log`.

Desequilibrio NEE:

- Con $N(\text{NEE}=\text{Sí}) \approx 8$, el p-valor de cualquier test es altamente inestable. Se reporta explícitamente que no deben extraerse conclusiones fuertes. La prueba de permutación es más robusta que la asintótica en este escenario, pero el tamaño de efecto con IC bootstrap es la información más fiable.

Autopercepción sin ID:

- Al no existir columna ID en el dataset de autopercepción, la relación con el rendimiento real solo puede establecerse a nivel agregado de grupo (media de grupo vs media de `pct_pc` del mismo grupo de Género/NEE). Esto se declara como **limitación explícita** y NO se realizan correlaciones individuales.

Nivel PC con pocas clases:

- Con solo 4 clases, los terciles colapsan a rangos: la clase con menor media → Bajo, la central (si hay 4, la segunda) → Medio bajo o Medio, etc. La distribución exacta se registra en el log y en `nivel_pc_por_clase.csv`.

10. Quality Gates — Verificación Automática de Outputs

Qué hace esta sección: Ejecuta una batería de comprobaciones automáticas sobre la existencia y el tamaño de cada archivo de salida esperado (tablas CSV y figuras PNG), y verifica invariantes de datos clave: N alumnos en test inicial y final de Historia (muestras independientes), Hake grupal por clase, presencia de la columna de nivel PC en todos los datasets que la requieren, número de actividades únicas en votaciones.

Por qué: Un análisis reproducible no termina con la última celda de código: debe verificar que *todos* sus outputs están completos e íntegros. Si algún gate falla, el mensaje indica exactamente qué archivo o invariante falta, permitiendo relanzar solo la sección correspondiente sin repetir todo el cuaderno.

PASS en todos los gates → El análisis es correcto, completo y reproducible. **FAIL en algún gate** → Revisa y re-ejecuta la sección indicada en el mensaje de error.

```
In [11]: # -----
# 10.1 Verificar existencia de todos los outputs esperados
# -----
logger.info("--- SECCIÓN 10: Quality Gates ---")

expected_tables = [
    # PC
    "auditoria_datasets.csv",
    "pc_limpio_por_alumno.csv",
    "pc_descriptivo_global.csv",
    "pc_descriptivo_por_grupo.csv",
    "pc_por_item_y_clase.csv",
    "pc_por_dimension.csv",
    "pc_contraste_genero.csv",
    "pc_contraste_nee.csv",
    "pc_decision_metodologica.csv",
    # Autopercepción
    "autopercepcion_descriptivo.csv",
    "autopercepcion_contrastes.csv",
    # Nivel PC
    "nivel_pc_por_clase.csv",
    # Historia – base
    "historia_resumen_por_grupo.csv",
    "historia_contrastes.csv",
    "historia_categoria_stats.csv",
    # Historia – análisis enriquecido (7.10–7.14)
    "historia_hake_por_clase.csv",
    "historia_efectos.csv",
    "historia_omnibus_clases.csv",
    "historia_analisis_items.csv",
    # Votaciones
```

```

    "votos_resumen_por_actividad_y_nivelpc.csv",
    "votaciones_ranking_actividades.csv",
]

expected_figures = [
    # PC
    "pc_histograma_global.png",
    "pc_violin_por_grupo.png",
    "pc_heatmap_items_clase.png",
    "pc_acierto_por_dimension.png",
    "pc_qqplot_genero.png",
    "pc_qqplot_nee.png",
    "pc_contraste_grupos.png",
    # Autopercepción
    "autopercepcion_por_genero.png",
    "autopercepcion_vs_rendimiento_agregado.png",
    # Nivel PC
    "pc_nivel_por_clase.png",
    # Historia – base (7.8)
    "historia_distribucion.png",
    "historia_ganancia_por_nivel_pc.png",
    "historia_rendimiento_por_clase.png",
    # Historia – Hake grupal (7.10)
    "historia_hake_por_clase.png",
    # Historia – distribuciones por clase (7.11)
    "historia_boxplot_ini_fin_clase.png",
    "historia_medias_ini_fin_clase.png",
    "historia_histograma_global.png",
    # Historia – tamaños de efecto (7.12)
    "historia_effect_sizes.png",
    # Historia – ANOVA inter-clases (7.13)
    "historia_comparacion_clases.png",
    # Historia – psicometría (7.14)
    "historia_heatmap_items.png",
    "historia_coerror_matrix.png",
    # Votaciones
    "votos_por_actividad_nivel_pc.png",
    "votaciones_ranking_actividades.png",
    "votaciones_heatmap_clase.png",
    "votos_vs_ganancia_clase.png",
]

print("=" * 65)
print("QUALITY GATES: Verificación de outputs")
print("=" * 65)

all_ok = True
print(f"\n[ TABLAS – {len(expected_tables)} esperadas ]")
for f in expected_tables:
    path = TABS / f
    status = "✓ OK " if path.exists() else "X FALTA"
    if not path.exists():
        all_ok = False
    print(f" {status} {f}")

print(f"\n[ FIGURAS – {len(expected_figures)} esperadas ]")
for f in expected_figures:
    path = FIGS / f
    status = "✓ OK " if path.exists() else "X FALTA"
    if not path.exists():

```

```

        all_ok = False
        print(f" {status} {f}")

# -----
# 10.2 Resumen final de métricas clave
# -----

# — Tabla resumen de quality gates —————
_gate_rows = []
for _f in expected_tables:
    _gate_rows.append({"Tipo": "Tabla", "Archivo": _f, "Estado": "PASS" if (TABS
for _f in expected_figures:
    _gate_rows.append({"Tipo": "Figura", "Archivo": _f, "Estado": "PASS" if (FIG
_df_gates = pd.DataFrame(_gate_rows)

def _style_gate_status(v):
    if v == "PASS":
        return "background-color: #d4edda; color: #155724; font-weight: bold"
    elif v == "FAIL":
        return "background-color: #f8d7da; color: #721c24; font-weight: bold"
    return ""

display(
    _df_gates.style
    .set_caption("Quality Gates – Estado de todos los outputs esperados")
    .set_table_styles(_HEADER_STYLE)
    .map(_style_gate_status, subset=["Estado"])
    .hide(axis="index")
)
del _gate_rows, _df_gates, _style_gate_status

print(f"\n{'=' * 65}")
print("RESUMEN FINAL")
print(f"\n{'=' * 65}")

n_pc_total = len(df_pc)
n_chico = (df_pc["Género"] == "Chico").sum()
n_chica = (df_pc["Género"] == "Chica").sum()
n_nee_si_final = (df_pc["NEE"] == "Sí").sum()
n_nee_no_final = (df_pc["NEE"] == "No").sum()

print(f"\nPC Test Inicial:")
print(f"  N total PC                : {n_pc_total}")
print(f"  Género Chico                : {n_chico}")
print(f"  Género Chica                : {n_chica}")
print(f"  NEE = Sí                    : {n_nee_si_final}")
print(f"  NEE = No                    : {n_nee_no_final}")

print(f"\nNivel PC por Clase:")
for _, row in df_nivel_clase.iterrows():
    n_alumnos = (df_pc["Clase"] == row["Clase"]).sum()
    print(f"  Clase {row['Clase']}: {row['nivel_pc']:6s} | mean_pct={row['mean_p

# Δ Muestras independientes: no existe df_paired → usar df_ini, df_fin, df_hake
print(f"\nHistoria (muestras independientes):")
print(f"  N Inicial                    : {len(df_ini)}")
print(f"  N Final                      : {len(df_fin)}")
print(f"  Ganancia abs. media por clase (pct_fin - pct_ini):")

if "df_hake_clase" in dir() and not df_hake_clase.empty:

```

```

for _, row in df_hake_clase.iterrows():
    print(f"    Clase {row['Clase']}: Δ={row['ganancia_abs']:+.3f} Hake g={
global_ini_qg = df_ini["pct_hist"].mean()
global_fin_qg = df_fin["pct_hist"].mean()
global_hake_qg = (global_fin_qg - global_ini_qg) / (1 - global_ini_qg) \
    if (1 - global_ini_qg) > 0 else float("nan")
    print(f" Hake grupal global      : g={global_hake_qg:+.3f}")

print(f"\nVotaciones:")
pct_miss_tot = df_votes["Voto"].isna().mean() * 100
print(f" N registros      : {len(df_votes)}")
print(f" % missing global   : {pct_miss_tot:.1f}%")
for _, row in pct_missing_votes.iterrows():
    if row["pct_missing"] > 0:
        act_s = str(row["Actividad"])[0:40]
        print(f" {act_s:40s}: {row['pct_missing']:.1f}% missing")

# -----
# 10.3 Anomalías y decisiones metodológicas registradas
# -----

print(f"\n{'=' * 65}")
print("ANOMALÍAS Y DECISIONES METODOLÓGICAS REGISTRADAS")
print(f"\n{'=' * 65}")
print(f" Ver log completo en: {log_path.resolve()}")

decisiones = [
    f"Formato PC detectado: {fmt_detected}",
    f"Conversiones letras→0/1: {n_conversiones}",
    f"NEE desequilibrio: N(Sí)={n_nee_si_final} → " +
    ("MWU+bootstrap; sin conclusiones fuertes" if n_nee_si_final < 10 else "test
Autopercepción: SIN ID común → solo análisis agregado por grupo",
    "Nivel PC: terciles sobre media de clase " +
    ("(qcut)" if n_clases >= 3 else "(ranking denso, pocas clases)"),
    "Historia: TOTAL recalculado donde discrepante",
    "Historia: nivel_pc=NaN para clases fuera de Echegaray",
    "Votaciones: valores fuera [0,10] → NaN",
]

for i, d in enumerate(decisiones, 1):
    print(f" [{i}] {d}")

if all_ok:
    print(f"\n{'=' * 65}")
    print(" ✓ TODOS LOS OUTPUTS VERIFICADOS CORRECTAMENTE")
    print(f"\n{'=' * 65}")
else:
    print(f"\n{'=' * 65}")
    print(" [!] ALGUNOS OUTPUTS NO ENCONTRADOS (ver detalle arriba)")
    print(f"\n{'=' * 65}")

logger.info("=== Análisis completado ===")
print(f"\nLog completo: {log_path.resolve()}")

```

=====

QUALITY GATES: Verificación de outputs

=====

[TABLAS – 21 esperadas]

- ✓ OK auditoria_datasets.csv
- ✓ OK pc_limpio_por_alumno.csv
- ✓ OK pc_descriptivo_global.csv
- ✓ OK pc_descriptivo_por_grupo.csv
- ✓ OK pc_por_item_y_clase.csv
- ✓ OK pc_por_dimension.csv
- ✓ OK pc_contraste_genero.csv
- ✓ OK pc_contraste_nee.csv
- ✓ OK pc_decision_metodologica.csv
- ✓ OK autopercepcion_descriptivo.csv
- ✓ OK autopercepcion_contrastes.csv
- ✓ OK nivel_pc_por_clase.csv
- ✓ OK historia_resumen_por_grupo.csv
- ✓ OK historia_contrastes.csv
- ✓ OK historia_categoria_stats.csv
- ✓ OK historia_hake_por_clase.csv
- ✓ OK historia_efectos.csv
- ✓ OK historia_omnibus_clases.csv
- ✓ OK historia_analisis_items.csv
- ✓ OK votos_resumen_por_actividad_y_nivelpc.csv
- ✓ OK votaciones_ranking_actividades.csv

[FIGURAS – 25 esperadas]

- ✓ OK pc_histograma_global.png
- ✓ OK pc_violin_por_grupo.png
- ✓ OK pc_heatmap_items_clase.png
- ✓ OK pc_acierto_por_dimension.png
- ✓ OK pc_qqplot_genero.png
- ✓ OK pc_qqplot_nee.png
- ✓ OK pc_contraste_grupos.png
- ✓ OK autopercepcion_por_genero.png
- ✓ OK autopercepcion_vs_rendimiento_agregado.png
- ✓ OK pc_nivel_por_clase.png
- ✓ OK historia_distribucion.png
- ✓ OK historia_ganancia_por_nivel_pc.png
- ✓ OK historia_rendimiento_por_clase.png
- ✓ OK historia_hake_por_clase.png
- ✓ OK historia_boxplot_ini_fin_clase.png
- ✓ OK historia_medias_ini_fin_clase.png
- ✓ OK historia_histograma_global.png
- ✓ OK historia_effect_sizes.png
- ✓ OK historia_comparacion_clases.png
- ✓ OK historia_heatmap_items.png
- ✓ OK historia_coerror_matrix.png
- ✓ OK votos_por_actividad_nivel_pc.png
- ✓ OK votaciones_ranking_actividades.png
- ✓ OK votaciones_heatmap_clase.png
- ✓ OK votos_vs_ganancia_clase.png

Quality Gates — Estado de todos los outputs esperados

Tipo	Archivo	Estado
Tabla	auditoria_datasets.csv	PASS
Tabla	pc_limpio_por_alumno.csv	PASS
Tabla	pc_descriptivo_global.csv	PASS
Tabla	pc_descriptivo_por_grupo.csv	PASS
Tabla	pc_por_item_y_clase.csv	PASS
Tabla	pc_por_dimension.csv	PASS
Tabla	pc_contraste_genero.csv	PASS
Tabla	pc_contraste_nee.csv	PASS
Tabla	pc_decision_metodologica.csv	PASS
Tabla	autopercepcion_descriptivo.csv	PASS
Tabla	autopercepcion_contrastes.csv	PASS
Tabla	nivel_pc_por_clase.csv	PASS
Tabla	historia_resumen_por_grupo.csv	PASS
Tabla	historia_contrastes.csv	PASS
Tabla	historia_categoria_stats.csv	PASS
Tabla	historia_hake_por_clase.csv	PASS
Tabla	historia_efectos.csv	PASS
Tabla	historia_omnibus_clases.csv	PASS
Tabla	historia_analisis_items.csv	PASS
Tabla	votos_resumen_por_actividad_y_nivelpc.csv	PASS
Tabla	votaciones_ranking_actividades.csv	PASS
Figura	pc_histograma_global.png	PASS
Figura	pc_violin_por_grupo.png	PASS
Figura	pc_heatmap_items_clase.png	PASS
Figura	pc_acierto_por_dimension.png	PASS
Figura	pc_qqplot_genero.png	PASS
Figura	pc_qqplot_nee.png	PASS
Figura	pc_contraste_grupos.png	PASS
Figura	autopercepcion_por_genero.png	PASS
Figura	autopercepcion_vs_rendimiento_agregado.png	PASS
Figura	pc_nivel_por_clase.png	PASS
Figura	historia_distribucion.png	PASS
Figura	historia_ganancia_por_nivel_pc.png	PASS
Figura	historia_rendimiento_por_clase.png	PASS
Figura	historia_hake_por_clase.png	PASS

Tipo	Archivo	Estado
Figura	historia_boxplot_ini_fin_clase.png	PASS
Figura	historia_medias_ini_fin_clase.png	PASS
Figura	historia_histograma_global.png	PASS
Figura	historia_effect_sizes.png	PASS
Figura	historia_comparacion_clases.png	PASS
Figura	historia_heatmap_items.png	PASS
Figura	historia_coerror_matrix.png	PASS
Figura	votos_por_actividad_nivel_pc.png	PASS
Figura	votaciones_ranking_actividades.png	PASS
Figura	votaciones_heatmap_clase.png	PASS
Figura	votos_vs_ganancia_clase.png	PASS

=====

RESUMEN FINAL

=====

PC Test Inicial:

N total PC	: 97
Género Chico	: 52
Género Chica	: 45
NEE = Sí	: 8
NEE = No	: 89

Nivel PC por Clase:

Clase A: Alto	mean_pct=0.661	N=24
Clase B: Bajo	mean_pct=0.610	N=23
Clase C: Bajo	mean_pct=0.552	N=26
Clase D: Medio	mean_pct=0.616	N=24

Historia (muestras independientes):

N Inicial	: 93
N Final	: 82
Ganancia abs. media por clase (pct_fin - pct_ini):	
Clase 5A: $\Delta=+0.068$	Hake $g=+0.273$ (Baja (<0.30))
Clase 5B: $\Delta=-0.036$	Hake $g=-0.149$ (Baja (<0.30))
Clase 5C: $\Delta=+0.020$	Hake $g=+0.110$ (Baja (<0.30))
Clase 5D: $\Delta=+0.016$	Hake $g=+0.081$ (Baja (<0.30))
Hake grupal global	: $g=+0.069$

Votaciones:

N registros	: 567
% missing global	: 29.1%
Actividad Edad Antigua	: 54.3% missing
Actividad Edad Media	: 50.6% missing
Actividad Edad Moderna	: 49.4% missing
Actividad Primer dia	: 2.5% missing
Actividad prehistoria	: 46.9% missing

=====

ANOMALÍAS Y DECISIONES METODOLÓGICAS REGISTRADAS

=====

Ver log completo en: H:\Proyectos\Analysis_PC_History\outputs\logs\data_quality.log

- [1] Formato PC detectado: LETRAS (A/B/C/D)
- [2] Conversiones letras→0/1: 2716
- [3] NEE desequilibrio: N(Sí)=8 → MWU+bootstrap; sin conclusiones fuertes
- [4] Autopercepción: SIN ID común → solo análisis agregado por grupo
- [5] Nivel PC: terciles sobre media de clase (qcut)
- [6] Historia: TOTAL recalculado donde discrepante
- [7] Historia: nivel_pc=NaN para clases fuera de Echegaray
- [8] Votaciones: valores fuera [0,10] → NaN

=====

✓ TODOS LOS OUTPUTS VERIFICADOS CORRECTAMENTE

=====

2026-05-22 13:26:24,871 [INFO] === Análisis completado ===

Log completo: H:\Proyectos\Analysis_PC_History\outputs\logs\data_quality.log